

Formation Angular Avancé Introduction

AYMEN SELLAOUTI

Plan du Cours

1. Injection de dépendances
2. Router Module
3. RxJS et HTTP
4. Reactive Form
5. Modules
6. Zone et Change Detection
7. Migration vers Ng20
8. Signaux
9. Standalone Component
10. Nouveau Control Flow

Répo de la formation

<https://github.com/aymensellaouti/advancedNg30326>

Angular Service et injection de dépendances



AYMEN SELLAOUTI

Objectifs

1. INJECTION DE DÉPENDANCES (IOC)
 1. Principes
 2. Configurer son application
 3. L'injection de dépendances : type-based et hiérarchique
 4. Différents types de providers
2. Les services fournis
3. Injection de service

Qu'est ce qu'un service ?



- Un service est une classe qui permet d'exécuter un traitement.
- Il permet d'encapsuler des fonctionnalités redondantes permettant ainsi d'éviter la redondance de code.

Component 1

```
f(){};  
g(){};  
k(){};
```

Component 2

```
f(){};  
g(){};  
l(){};
```

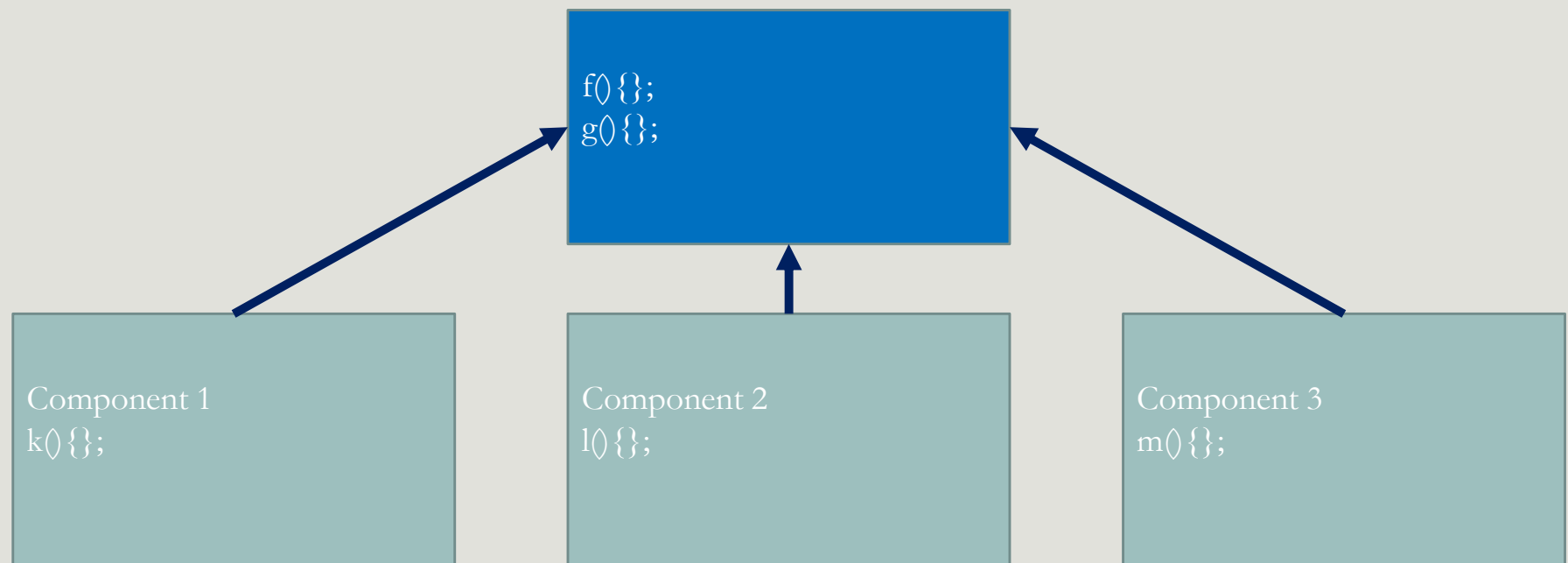
Component 3

```
f(){};  
g(){};  
m(){};
```

Redondance de code

Maintenabilité difficile

Qu'est ce qu'un service ?



Qu'est ce qu'un service ?



- Un service peut :
 - Interagir avec les données (fournit, supprime et modifie)
 - Interaction entre classes et composants
 - Tout traitement métier (calcul, tri, extraction ...)

Création d'un service

- Via CLI
 - `ng generate service nomDuService`
 - `ng g s nomDuService`

Premier Service

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class FirstService {

  constructor() { }

}
```

Comment fonctionne un restaurant ?



Injection de dépendance (DI)



- L'injection de dépendance est un patron de conception.

```
Classe A1 {  
  ClasseB b;  
  ClasseC c;  
  ...  
}
```

```
Classe A2 {  
  ClasseB b;  
  ...  
}
```

```
Classe A3 {  
  ClasseC c;  
  ...  
}
```

- Que se passera t-il si on change quelque chose dans le constructeur de B ou C ?
- Qui va modifier l'instanciation de ces classes dans les différentes classes qui en dépendent?

Injection de dépendance (DI)



- Déléguer cette tâche à une entité tierce.

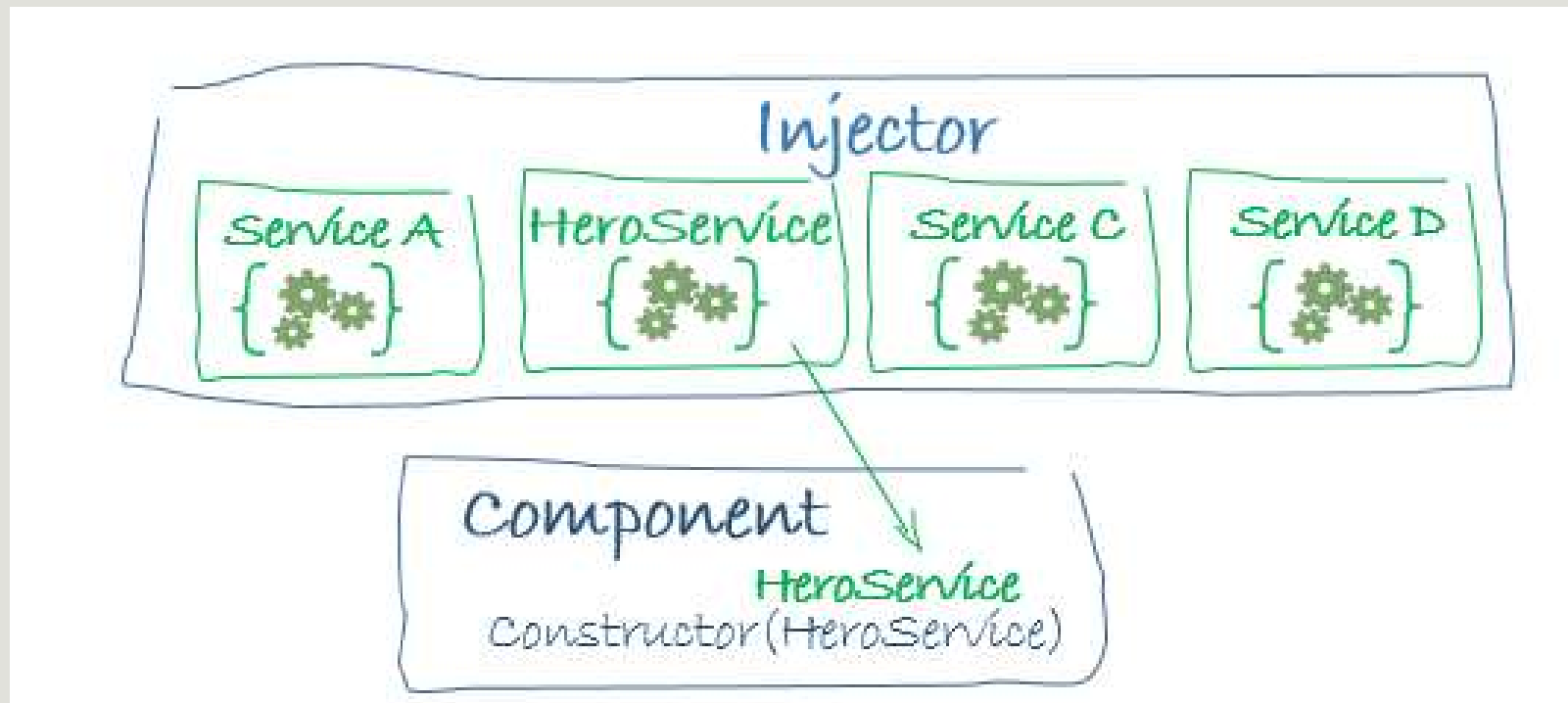
```
Classe A1 {  
  Constructor(B b, C c)  
  ...  
}
```

```
Classe A2 {  
  Constructor(B b)  
  ...  
}
```

```
Classe A3 {  
  Constructor(C c)  
  ...  
}
```

INJECTOR

Injection de dépendance (DI)



Comment fonctionne un restaurant (système d'injection de dépendances) ?

Providers

Injector

IOC

Injection



Injection de dépendance (DI)

- Comment les injecter ?
- Comment spécifier à l'injecteur quel service et ou est-il visible ?

Injection de dépendance (DI)

- L'injection de dépendance utilise les étapes suivantes :
 - **Provider les dépendances** (**Préparer notre menu, définir quels plats nous pouvons fournir**) via **l'annotation @Injectable et son attribut providedIn** (ici les services), dans le **provider du module ou du composant** (**C'est la préparation de votre menu**).
 - **Injecter le service (commander le plat souhaité)** en le passant comme **paramètre du constructeur** de la **classe (composant ou service) qui en a besoin**.

Injection de dépendance (DI)

Provider les dépendances

```
@NgModule({  
  providers: [CvService],  
  bootstrap: [AppComponent],  
})  
export class AppModule {}
```

```
@Injectable({  
  providedIn: 'root',  
})  
export class CvService {}
```

Provider

Injector

```
@Component({  
  selector: 'app-cv',  
  templateUrl: './cv.component.html',  
  styleUrls: ['./cv.component.css'],  
  providers: [CvService]  
})  
export class CvComponent {}
```

```
export class CvComponent {  
  constructor(  
    private cvService: CvService  
  ) {}  
}
```

Chargement automatique du service

- A partir de Angular 6 vous pouvez ne plus utiliser le provider du module afin de charger votre service mais le faire directement au niveau du service à travers l'annotation **@Injectable** et sa propriété **providedIn**. Vous pouvez charger le service dans toute l'application via le mot clé **root**. Ceci permet **d'optimiser votre code** via :
- **Lazy loading** : N'instancie un service que s'il est injecté au moins une fois
- Permettre le **Tree-Shaking** des services non utilisés : Si le service n'est jamais utilisé, son **code sera entièrement retiré du build final**.

```
@Injectable({  
  providedIn: 'root',  
})  
export class CvService {}
```

@Injectable

- C'est un décorateur permettant de rendre une classe injectable
- Une classe est dite injectable si on peut y injecter des dépendances
- @Component, @Pipe, et @Directive sont des sous classes de @Injectable(), ceci explique le fait qu'on peut y injecter directement des dépendances.
- Si vous n'aller injecter aucun service dans votre service, cette annotation n'est plus nécessaire.
- **Remarque** : Angular conseille de toujours mettre cette annotation.

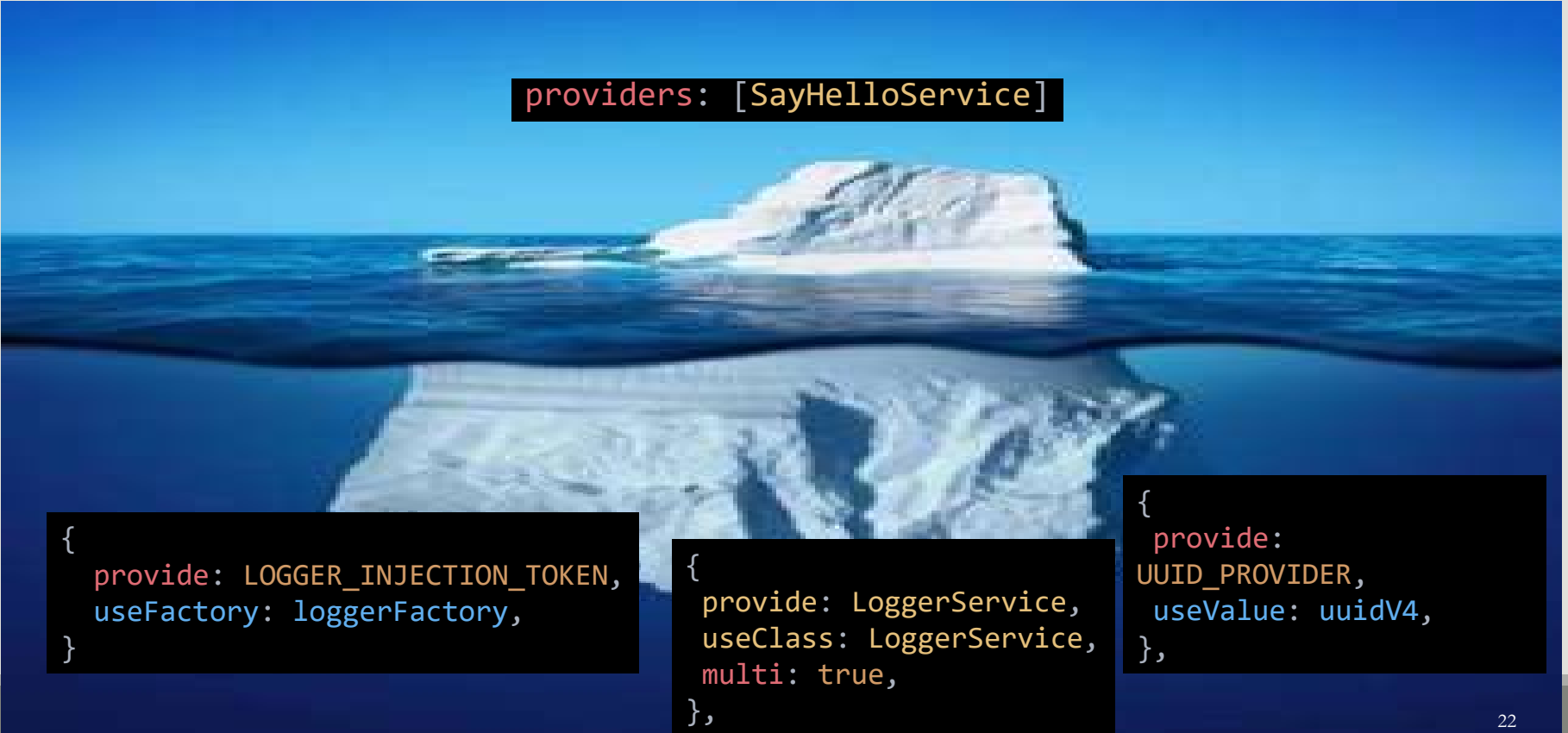
Injection de dépendance (DI)

Avantages

- L'intérêt de l'injection de dépendance est donc :
 - **Couplage lâche**
 - Facilement **remplacer une implémentation** d'une dépendance à des fins de test
 - Prendre en charge **plusieurs environnements** d'exécution
 - Fournir de **nouvelles versions d'un service** à un tiers qui utilise votre service dans sa base de code, etc.

Injection de dépendance (DI)

Deep Dive



```
providers: [SayHelloService]
```

```
{  
  provide: LOGGER_INJECTION_TOKEN,  
  useFactory: loggerFactory,  
}
```

```
{  
  provide: LoggerService,  
  useClass: LoggerService,  
  multi: true,  
},
```

```
{  
  provide:  
    UUID_PROVIDER,  
    useValue: uuidV4,  
},
```

Injection de dépendance (DI)

Que peut on injecter

Que peut on injecter ?

- Toute dépendance de votre classe, à savoir :
 - Des instances de classes
 - Des constantes

Injection de dépendance (DI)

Aller plus loin : Comment ça fonctionne

- Reprenons l'exemple de notre restaurant. Pour préparer son menu, **le chef de la cuisine a une recette associé à chacune des plats du menu**. Mais imaginons qu'un client ait envie d'un plat qui n'est pas dans le menu. Que doit-il faire ?
- **Fournir sa propre recette au chef.**
- **C'est ce qu'on appelle une providerFactory**



Injection de dépendance (DI)

Les providersFactory

- Une **providerFactory** est la **recette** permettant à votre système d'inversion de control de créer les dépendances
- Ceci implique que **pour toute dépendance** de votre application quelque soit son type, il **existe une Provider Factory** qui **sait comment la créer** et qui le fait.
- Le moyen permettant de spécifier au système d'injection de dépendance d'Angular comment créer la dépendance est la fonction **Provider factory**.
- Une **Provider factory** est simplement une **fonction** simple qu'**Angular** peut **appeler** afin de **créer une dépendance**.
- Cette fonction peut être **créée implicitement par Angular** en utilisant quelques conventions simples (c'est une recette du menu connues par le chef, c'est le cas le plus répandue) ou **par vous-même (votre propre recette)**.

Injection de dépendance (DI)

Comment ça fonctionne

- Injection de **dépendance** => Il nous faut donc une **dépendance**
- Afin de **Lier** cette **dépendance** au **système d'injection de dépendance d'Angular** nous devons répondre à deux questions
 - **Comment Angular va créer la dépendance ?**
 - **Quand est ce qu'Angular doit utiliser cette dépendance ?**
- Le moyen permettant de spécifier au système d'injection de dépendance d'Angular comment créer la dépendance est la fonction ***Provider factory***.
- Une ***Provider factory*** est simplement une **fonction** simple qu'**Angular** peut **appeler** afin de **créer une dépendance**.

Injection de dépendance (DI)

Comment ça fonctionne

- Cette fonction peut être *créée implicitement par Angular* en utilisant quelques conventions simples (le cas le plus répondu) ou *par vous même*.
- Ceci implique que **pour toute dépendance** de votre application, quelque soit son type, il **existe une Provider Factory** qui **sait comment la créer** et qui le fait.

Injection de dépendance (DI)

Comment ça fonctionne

```
function todoServiceProviderFactory(): TodoService {  
    return new TodoService();  
}  
function todoServiceProviderFactory(http:HttpClient): TodoService {  
    return new TodoService(http);  
}
```

Injection de dépendance (DI)

Associer votre Factory Provider à Angular Token

- Maintenant il reste à dire à Angular **quand utiliser ce provider**.
- Donc, nous devons répondre à cette interrogation : **Comment Angular sait-il quoi injecter**, et donc **quelle Provider factory appeler pour créer quelle dépendance ?**
- Pour ce faire, vous pouvez utiliser des **Tokens**.
- Un Token peut avoir plusieurs formes et il a pour **rôle d'identifier une Provider Factory**.

Injection de dépendance (DI)

Associer votre Factory Provider à Angular

Angular injection token

- La première forme de Token est l'**Angular injection token**
- C'est une instance de la class **InjectionToken**
- Son rôle est **d'identifier le service** dans le système d'injection de dépendance

```
export const TODOS_SERVICE_TOKEN = new InjectionToken<TodoService>("TODO_SERVICE_TOKEN");
```

Injection de dépendance (DI)

Associer votre Factory Provider à Angular

Configurer le Provider

- Maintenant que nous avons notre Provider Factory et notre Token, nous devons **configurer** Angular pour qu'ils les prennent en considération.
- Ceci sera fait à travers le **Provider** qui n'est autre qu'un **objet de configuration**.
- Il peut prendre en paramètres 3 clés (pas que):
 - **provide**: qui est notre Token
 - **useFactory**: qui est notre Factory
 - **deps**: un tableau des dépendances de votre factory

Injection de dépendance (DI)

Associer votre Factory Provider à Angular

Configurer le Provider

```
export const TODOS_SERVICE_TOKEN =  
  new InjectionToken<TodoService>("TODO_SERVICE_TOKEN");  
function todoServiceProviderFactory(http:HttpClient): TodoService {  
  return new TodoService(http);  
}
```

```
providers: [  
  {  
    provide: TODOS_SERVICE_TOKEN,  
    useFactory: todoServiceProviderFactory,  
    deps: [HttpClient]  
  }  
]
```


Injection de dépendance (DI)

Associer votre Factory Provider à Angular

Injecter la factory

- Il reste une dernière étape, à savoir **comment injecter notre dépendance dans notre classe**.
- On utilise le décorateur **@Inject** au niveau du **constructeur** et on lui passe le **Token du Provider Factory** que nous voulons injecter.

```
constructor(  
  @Inject(TODOS_SERVICE_TOKEN) private todoService: TodoService  
) {}
```

Les providers personnalisés

Les autres formes de Tokens

- Comme nous l'avons présenté, le Token peut avoir plusieurs formes. Parmi elles, le **nom de la classe**.
- Le token peut être aussi une **chaîne de caractères**, mais ceci est déconseillé afin d'éviter les collisions de noms.
- Le **TOKEN** **doit** être **unique pour éviter toute collision**, les **Provider factory** sont **stockés dans une map**, et si le provider est simple et qu'il a le même nom, la map ne contiendra que **le dernier provider** défini.

Les providers personnalisés

Les autres formes de Tokens

```
providers: [  
  {  
    provide: TodoService,  
    useFactory: todoServiceProviderFactory,  
    deps: [HttpClient]  
  }  
],
```

```
constructor(  
  @Inject(TodoService) private todoService: TodoService  
) {}
```

Les providers personnalisés

useClass

- Une autre option s'offre à vous. Au lieu de spécifier la Fonction du Provider Factory avec `useFactory`, vous pouvez utiliser la clé **`useClass`**.
- En utilisant **`useClass`**, Angular saura que la valeur que nous transmettons est un **`constructeur`**, qu'Angular peut simplement **`appeler en utilisant l'opérateur new`**.

Les providers personnalisés

useClass

```
providers: [  
  {  
    provide: TodoService,  
    useClass: TodoService,  
    deps: [HttpClient]  
  }  
],
```

```
constructor(  
  @Inject(TodoService) private todoService: TodoService  
) {}
```

Les providers personnalisés

useClass

- Une autre fonctionnalité très pratique de **useClass** est que pour ce type de dépendances, Angular **essaiera de déduire le Token d'injection au moment de l'exécution** en fonction de la **valeur des annotations de type Typescript**.
- Cela signifie qu'avec les dépendances useClass, nous n'avons même **plus besoin du décorateur Inject**, ce qui explique pourquoi vous le voyez rarement.

Les providers personnalisés

useClass

```
providers: [  
  {  
    provide: TodoService,  
    useClass: TodoService,  
    deps: [HttpClient]  
  }  
],
```

Le Token est déterminé par
Angular en utilisant le Type
TodoService

```
constructor(private todoService: TodoService) {}
```

@Injectable

- C'est un décorateur permettant de rendre une classe injectable
- Une classe est dite injectable si on peut y injecter des dépendances
- `@Component`, `@Pipe`, et `@Directive` sont des sous classes de `@Injectable()`, ceci explique le fait qu'on peut y injecter directement des dépendances.
- Si vous n'aller injecter aucun service dans votre service, cette annotation n'est plus nécessaire.
- **Remarque** : Angular conseille de toujours mettre cette annotation.

Les providers personnalisés

useClass

- En utilisant le décorateur **@Injectable**, votre Provider devient encore plus simple puisqu'**on n'a plus à spécifier les dépendances qui seront directement déterminées au niveau du constructeur** par le Système d'Injection de dépendance d'Angular

```
providers: [  
  {  
    provide: TodoService,  
    useClass: TodoService,  
    deps: [HttpClient]  
  },  
],
```



```
providers: [  
  {  
    provide: TodoService,  
    useClass: TodoService,  
  },  
],
```



```
providers: [  
  {  
    TodoService,  
  },  
],
```

Les providers personnalisés

useClass

- La syntaxe **useClass** est aussi utile pour injecter dynamiquement une classe.
- Imaginez que le service de log dépend de l'environnement de développement.

```
@Module({  
  providers: [  
    {  
      provide: LoggerService,  
      useClass:  
        config.env === 'development'  
          ? DevelopmentLoggerService  
          : ProductionLoggerService,  
    },  
  ],  
}) export class AppModule {}
```


- 43

Les providers personnalisés multi

- La plupart des dépendances de notre système correspondront à **une seule valeur**, comme par exemple une classe.
- Cependant, il y a des occasions où nous voulons avoir **plusieurs instances pour le même provider**.
- Pour ce faire, ajouter la clé **multi** et mettez la à **true**.
- Au lieu de recevoir **une instance**, vous recevrez **un tableau d'instance**.

```
const AuthenticationInterceptorProvider = {  
  provide: HTTP_INTERCEPTORS,  
  useClass: AuthenticationInterceptor,  
  multi: true  
};
```

Injection de dépendance (DI)

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class CvService {
  // Ce service est visible pour tout le monde
  constructor() { }
}
```

Injection de dépendance (DI)

```
import { BrowserModule, } from '@angular/platform-browser';
import { CUSTOM_ELEMENTS_SCHEMA, NgModule } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { HttpClientModule } from '@angular/http';
import { AppComponent } from './app.component';
import { CvService } from './cv.service';
@NgModule({
  declarations: [
    AppComponent,
  ],
  imports: [
    BrowserModule,
    FormsModule,
    HttpClientModule
  ],
  providers: [CvService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

```
import { Injectable } from
 '@angular/core';

@Injectable()
export class CvService {

  constructor() { }

}
```

Injection de dépendance (DI)

```
import { Component, OnInit } from '@angular/core';
import { Cv } from './cv';
import { CvService } from "../cv.service";
@Component({
  selector: 'app-cv',
  templateUrl: './cv.component.html',
  styleUrls: ['./cv.component.css'],
  providers: [CvService] // on peut aussi l'importer ici
})
export class CvComponent implements OnInit {
  selectedCv : Cv;
  constructor(private monPremierService: CvService) { }
  ngOnInit() {
  }
}
```

Chargement automatique du service

- A partir de Angular 6 vous pouvez ne plus utiliser le provider du module afin de charger votre service mais le faire directement au niveau du service à travers l'annotation `@Injectable` et sa propriété `providedIn`. Vous pouvez charger le service dans toute l'application via le mot clé `root`.
- Si vous voulez charger le service dans un module particulier vous l'importer et vous le mettez à la place de `'root'`.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class CvService {
  constructor() { }
}
```


Chargement automatique du service `providedIn`

- La clé **`providedIn`** peut prendre les valeurs suivantes :
- **`root`** : L'injecteur au niveau de l'application, c'est ce que vous trouvez dans la plupart des applications.
- **`platform`** : Un injecteur de plateforme singleton spécial partagé par toutes les applications de la page.
- **`any`** : Fournit une **instance unique** dans chaque module chargé d'une manière *lazy* tandis que tous les modules chargés en *eager* partagent une instance. Cette option est obsolète.

```
providedIn?: Type<any> | 'root' | 'platform' | 'any' | null;
```

Avantage de l'utilisation du providedIn

- Permettre le **Tree-Shaking** des services non utilisés : Si le **service n'est jamais utilisé**, son **code** ne sera **entièrement retiré du build final**.

Autres providers

- Dans certains cas d'utilisation, l'utilisation standard des providers ne convient pas, imaginer l'un des cas suivants :
 - Vous souhaitez créer une instance personnalisée au lieu de laisser le container le faire pour vous.
 - Vous voulez **injecter une bibliothèque externe**
 - Vous voulez **mock une classe pour le teste**
 - Vous voulez injecter des **instances différentes** selon **le contexte** ...
- Angular nous permet de définir des providers particuliers selon votre besoin.

Les providers personnalisés

useValue

- La syntaxe **useValue** est utile pour injecter
 - Une valeur constante,
 - Une bibliothèque externe
 - Remplacer une implémentation réelle par un objet fictif.

```
providers: [  
  {  
    useValue: [{ lundi: 'Angular' }, { mardi: 'Still Angular' }],  
    provide: 'TODO_LIST',  
  },  
  TodoService,  
],
```

Les providers personnalisés

useValue

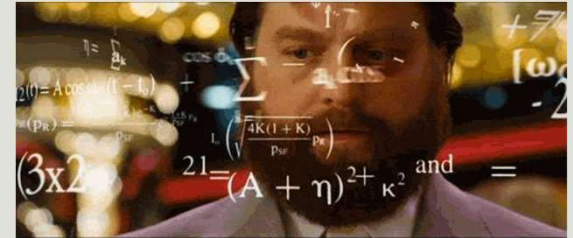
- Si vous injecter une classe, l'utilisation de l'injection via le constructeur reste d'actualité.
- Sinon, pour injecter ce provider utiliser la syntaxe **@Inject**, qui prend en paramètre le Token.

```
providers: [  
  {  
    provide: 'TODOS_LIST',  
    useValue: [{ lundi: 'nestJs' }, { mardi: 'Still NestJs' }],  
  },  
  TodoService,  
],
```

```
constructor(  
  @Inject('TODOS_LIST') todoList,  
) {  
  console.log('Fake Todo List', todoList);  
}
```

```
@Controller('todo')  
export class TodoController {  
  todoList = inject('TODOS_LIST');  
  constructor() {}  
}
```

Exercice



- Provider la fonction uuid
- Faite en sorte de l'utiliser dans le TodoService

Les providers personnalisés la fonction inject (avant Angular 14)

- La fonction inject vous permet d'injecter un injectable.
- **Avant Angular 14** et à partir **d'Angular 9**, la fonction **inject** pouvait être utilisé uniquement dans la **factory** de **l'InjectionToken** ou dans le factory du **@Injectable**.

```
import {inject, InjectionToken, PLATFORM_ID} from "@angular/core";

export const WINDOW = new InjectionToken<Window>('Window bject', {
  providedIn: 'root',
  factory: () => {
    const platformId = inject(PLATFORM_ID);
    return platformId === 'browser' ? window : {} as Window;
  }
})
```

```
@Injectable({
  providedIn: 'root',
  useFactory: () => {
    const platformId = inject(PLATFORM_ID);
    return platformId === 'browser' ? window : {} as Window;
  }
})
export class WindowService{}
```

Les providers personnalisés la fonction inject (après Angular 14)

- A partir d'Angular 14, cette fonction **n'est plus limitée aux factory**.
- Vous pouvez maintenant **l'utiliser dans vos composants, directives et pipes**.
- Le premier intérêt est le **type safety**
- Il **facilite aussi l'héritage** en externalisant les dépendances de la classe.

DI Hiérarchique

- Dans Angular vous disposez de plusieurs endroits où vous pouvez définir les providers pour vos dépendances :
 - Module
 - Composant
 - Directive !

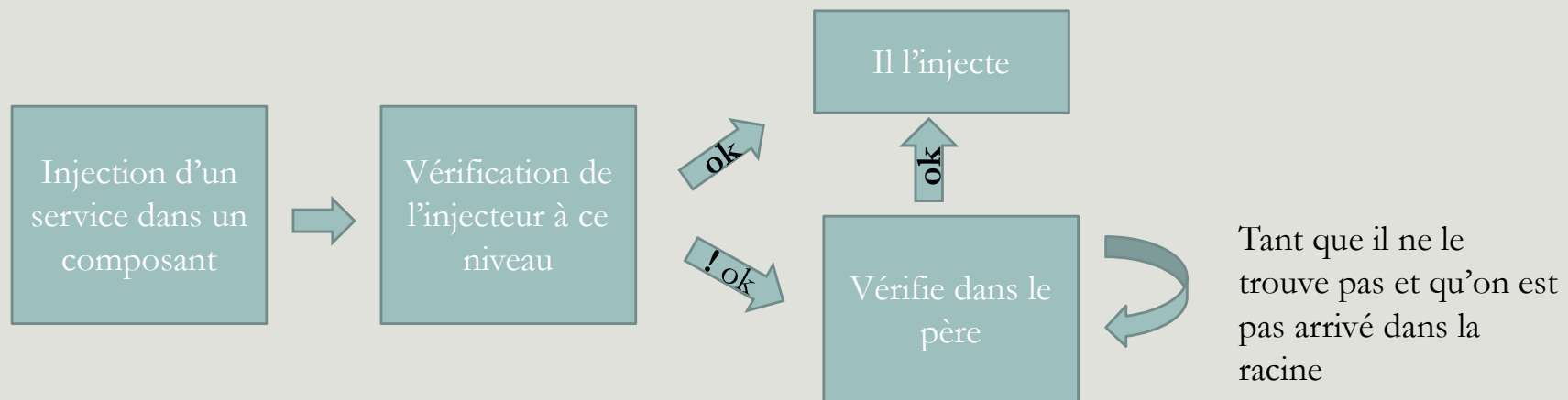
DI Hiérarchique

- Le système d'injection de dépendance d'Angular est **hiérarchique**
- Il possède **deux hiérarchie d'injecteur**
 - Une **hiérarchie d'injecteur niveau composant** (element Injector **Hierarchy**)
 - Une hiérarchie d'injecteur **niveau module**.
- La **hiérarchie composant** (appelé aussi **Node Injector Hierarchy**) est la plus prioritaire

DI Hiérarchique

Hiérarchie d'injecteur niveau composant

- Le système d'injection de dépendance d'Angular est hiérarchique.
- Un arbre d'injecteur est créé. Cet arbre est // à l'arbre de composant.
- L'algorithme suivant permet la détection de l'injecteur adéquat :

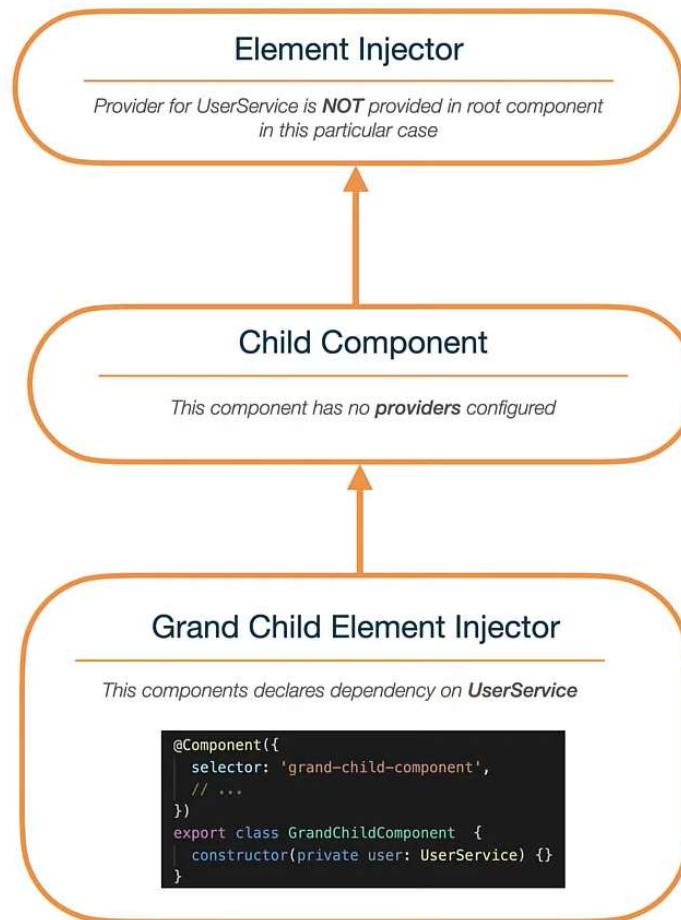


DI Hiérarchique

Hiérarchie d'injecteur niveau Module

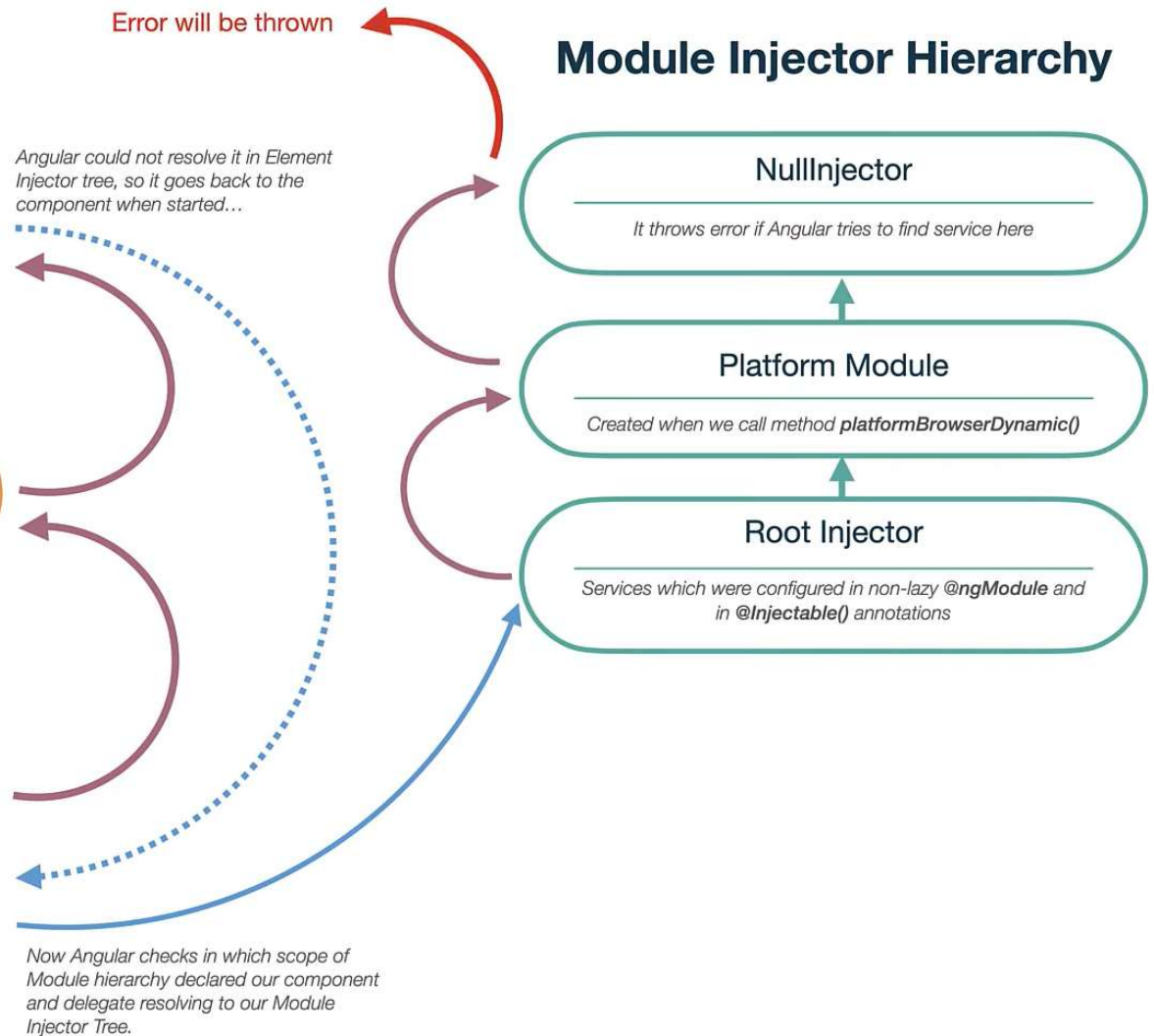
- Si Angular **ne trouve pas de provider** au niveau de la **hiérarchie des composants**, il va aller voir dans la **hiérarchie de module**.
- Le ModuleInjector peut être configuré de deux manières en utilisant :
 - La propriété @Injectable pour référencer un **NgModule**, ou 'root'
 - Le tableau des providers dans @NgModule()
- Le moduleInjector **identifie les providers disponibles** en effectuant un **aplatissement de tous les tableaux de providers** qui peuvent être atteints en suivant **les NgModule.imports de manière récursive**.

Element Injector Hierarchy



Angular could not resolve it in Element Injector tree, so it goes back to the component when started...

Module Injector Hierarchy



Angular Routing

AYMEN SELLAOUTI

Récupérer les paramètres d'une route

ActivatedRoute

- Afin de récupérer les paramètres d'une route au niveau d'un composant, Angular nous fournit un **service** qui gère la **route active**, c'est le **ActivatedRoute**
- L'objet **ActivatedRoute** dans Angular est un objet qui contient des **informations sur la route actuellement activée**.
- Il est généralement utilisé pour **accéder aux informations de route** telles que **les paramètres de route**, **les données de route** et **les paramètres de requête**.
- Il est également utilisé **pour souscrire aux changements de route**.

Récupérer les paramètres d'une route

ActivatedRoute

- **params**: Retourne un observable des paramètres de route actuels.
- **queryParams**: Retourne un observable des paramètres de requête actuels.
- **data**: Retourne un observable des données de route associées à la route actuelle.
- **snapshot**: Retourne un instantané de la route actuelle.
- **url**: Retourne un observable de l'URL de la route actuelle.
- **parent**: Retourne l'instance de `ActivatedRoute` de la route parente.
- **firstChild**: Retourne l'instance de `ActivatedRoute` du premier enfant de la route actuelle.

Récupérer les paramètres d'une route

ActivatedRoute

- **children**: Retourne un tableau d'instances de `ActivatedRoute` des enfants de la route actuelle.
- **paramMap**: Retourne un observable qui contient les paramètres de route sous forme de map.
- **queryParamsMap**: Retourne un observable qui contient les paramètres de requête sous forme de map.

Récupérer les paramètres d'une route ActivatedRoute

```
activatedRoute                                details-cv.component.ts:33
                                              details-cv.component.ts:34
ActivatedRoute {url: BehaviorSubject, params: BehaviorSubject, queryParams: BehaviorSubject, fragment: BehaviorSubject, data: BehaviorSubject, ...} ⓘ
  ▶ component: class DetailsCvComponent
  ▶ data: BehaviorSubject {closed: false, currentObservers: Array(0), ...}
  ▶ fragment: BehaviorSubject {closed: false, currentObservers: null, outlet: "primary"}
  ▶ params: BehaviorSubject {closed: false, currentObservers: null, ...}
  ▶ queryParams: BehaviorSubject {closed: false, currentObservers: null, ...}
  ▶ snapshot: ActivatedRouteSnapshot {url: Array(1), params: {...}, ...}
  ▶ title: AnonymousSubject {closed: false, currentObservers: null, ...}
  ▶ url: BehaviorSubject {closed: false, currentObservers: null, ...}
  ▶ _futureSnapshot: ActivatedRouteSnapshot {url: Array(1), params: {...}}
  ▶ _routerState: RouterState {_root: TreeNode, snapshot: RouterStateSnapshot, ...}
    children: (...)
    firstChild: (...)
    paramMap: (...)
    parent: (...)
    pathFromRoot: (...)
    queryParamMap: (...)
    root: (...)
    routeConfig: (...)
  ▶ [[Prototype]]: Object
```

Récupérer les paramètres d'une route

ActivatedRoute / snapshot

- La propriété **snapshot** de l'objet **ActivatedRoute** contient un instantané de l'état de la route actuelle.
- Elle contient des **informations** telles que **l'URL actuelle**, **les paramètres de route actuels**,...
- Elle est généralement utilisée pour **accéder aux informations de route** dans un composant **lors de son initialisation**.
- Il est important de noter que **l'instantané de la route ne change pas lorsque la route change**. Il représente un **état figé** de la route lors de son instantiation.

Récupérer les paramètres d'une route

ActivatedRoute / snapshot

- Voici quelques propriétés courantes de l'API snapshot :
 - **url**: Retourne l'URL de la route actuelle sous forme de tableau de segments d'URL.
 - **params**: Retourne un objet qui contient les paramètres de route actuels.
 - **queryParams**: Retourne un objet qui contient les paramètres de requête actuels.
 - **fragment**: Retourne la partie de l'URL après le symbole "#".
 - **data**: Retourne les données de route associées à la route actuelle.
 - **outlet**: Retourne le nom de l'outlet de route actuel.
 - **component**: Retourne le composant de route actuel.
 - **routeConfig**: Retourne la configuration de la route actuelle.

Récupérer les paramètres d'une route ActivatedRoute / snapshot

```
▼ snapshot: ActivatedRouteSnapshot
  ▶ component: class DetailsCvComponent
  ▶ data: {cv: {...}}
  fragment: null
  outlet: "primary"
  ▶ params: {id: '27'}
  ▶ queryParams: {}
  ▼ routeConfig:
    ▶ component: class DetailsCvComponent
    path: ":id"
    ▶ resolve: {cv: f}
    ▶ [[Prototype]]: Object
  ▶ url: [UrlSegment]
  _lastPathIndex: 1
  ▶ _paramMap: ParamsAsMap {params: {...}}
  ▶ _resolve: {cv: f}
  ▶ _resolvedData: {cv: {...}}
  ▶ _routerState: RouterStateSnapshot {_root: TreeNode, url: '/cv/2'}
  ▶ _urlSegment: UrlSegmentGroup {segments: Array(2), children: {...}}
  ▶ children: Array(0)
  firstChild: null
  ▼ paramMap: ParamsAsMap
    ▶ params: {id: '27'}
    keys: (...)
    ▶ [[Prototype]]: Object
    parent: (...)
```

Récupérer les paramètres d'une route

ActivatedRoute / snapshot

- Donc pour accéder à votre propriété, passez par l'objet **snapshot**
- Avec **snapshot**, vous avez deux méthodes pour récupérer les paramètres:
- Via la **propriété params** qui retourne un tableau d'objet des paramètres
- Via la propriété **paramMap**
 - Appeler sa méthode **get**
 - Passez lui le nom de la propriété souhaitée.

```
this.activatedRoute.snapshot.paramMap.get('id')
```

Route Fils

- Certains composants ne sont visible qu'à l'intérieur d'autres composants.
- Prenons l'exemple d'un objet Personne. En accédant à la route `/personne/:id` nous avons l'affichage de la personne et nous aimerions avoir deux boutons. Un pour éditer la Personne (route `/personne/:id/editer`). L'autre pour afficher ces détails (route `/personne/:id/aperçu`).
- L'idée est de **préfixer** nos routes.

Route Fils

- Afin de mettre en place ce processus nous procédons comme suit :
 - Nous définissons le préfixe avec la propriété **path**.
 - Nous y ajoutons la propriété **children** qui contiendra le tableau des routes. Chaque route de ce tableau sera préfixé avec la route définie dans **path**.

Route Fils

```
const CV_ROUTE: Routes = [
  {
    path: 'cv',
    children: [
      {path: '', component: CvComponent },
      {path: 'detail/:id', component: DetailCvComponent },
      {path: 'addPersonne', component: FormPersonneComponent },
    ]
  }
];
```

Route fils / définition dans un parent

- Supposons que nous voulons avoir un Template central avec des données fixe et des parties variables dans le même template.
- En changeant les routes, le contenu principal doit rester le même et la partie variable doit changer selon la route.

Route Fils

- Afin de mettre en place ce processus nous procédons comme suit :
 - Nous définissons le préfixe avec la propriété **path**. On lui associe le composant Père.
 - Nous y ajoutons la propriété **children** qui contiendra le tableau des routes. Chaque route de ce tableau sera préfixé avec la route définie dans **path**.
 - Nous ajoutons la balise `<router-outlet></router-outlet>` dans le Template père.

Route Fils

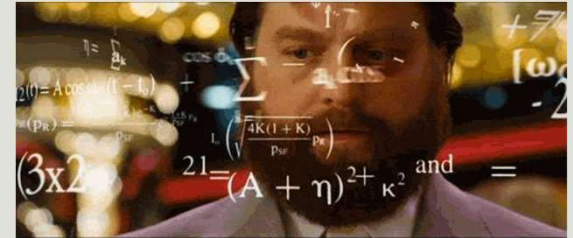
```
const CV_ROUTE: Routes = [  
  {  
    path: 'cv',  
    component: CvComponent,  
    children: [  
      {path: 'detail/:id', component: DetailCvComponent },  
      {path: 'addPersonne', component: FormPersonneComponent },  
    ]  
  }  
];
```

Master Detail Implementation

- Le pattern "**master-detail**" dans Angular est un modèle d'architecture utilisé pour afficher des données dans une interface utilisateur.
- Il consiste à avoir **une vue "maître"** qui affiche une **liste d'éléments**, et une vue **"détail"** qui affiche les **détails d'un élément** sélectionné dans la vue "maître".
- Cela permet aux utilisateurs de naviguer facilement entre les différents éléments et de voir les détails correspondants sans avoir à charger une nouvelle page.
- Afin de l'implémenter, il faut utiliser les routes fils.

Exercice

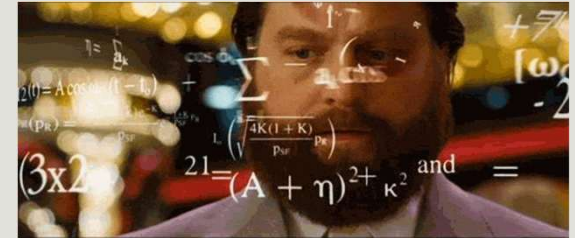
Master Detail Implementation



- Nous voulons implémenter le pattern Master Details pour notre liste de cvs.
- Créer un nouveau composant MasterDetailsCv.
- Il devra permettre l'affichage de la liste des cvs. Au click, un détail du cv sélectionné devra apparaitre.

Exercice

Master Detail Implementation



cvStartingAdvProject

localhost:4200/cv/list

todo goodies gk Barre de favoris TCPC2022 Rem4U Routeur - Accueil enseignement utilities enseignement balis selection rxjs ChatGPT

cvStartingAdvProject Home Cv List Add Cv Todo Word Color Logout

container works!

resolution-modifiers works!

master-detail-cv works!

aymen sellauti

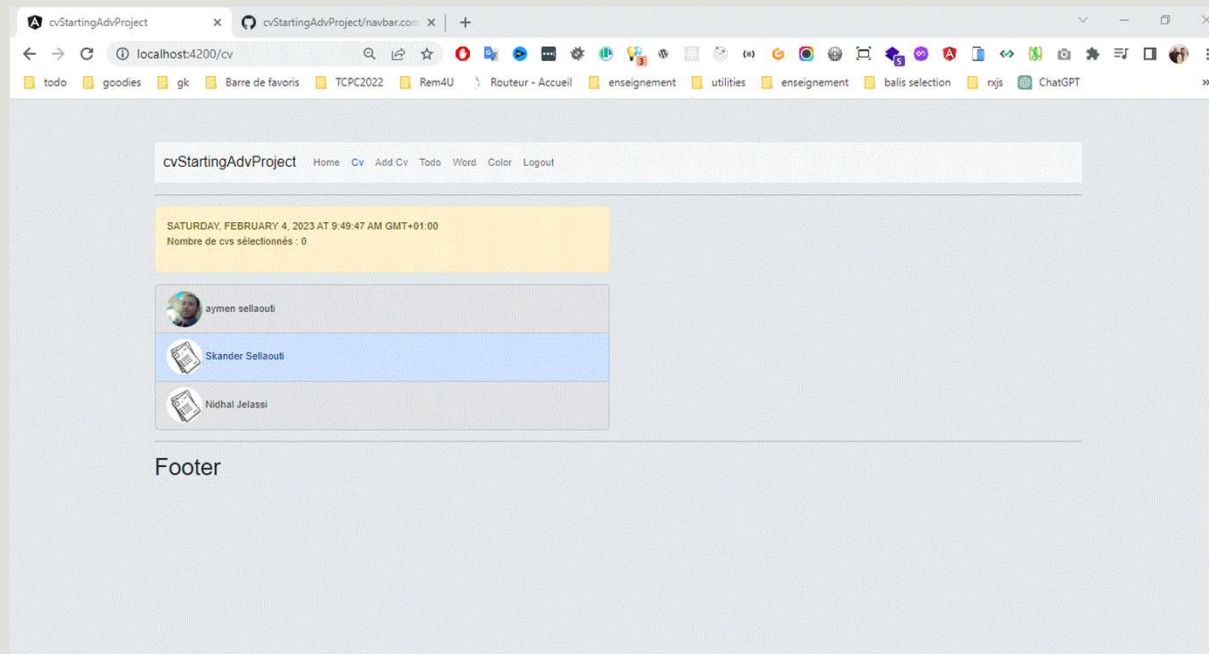
Skander Sellaoui

Nidhal Jelassi

Footer

Router Resolver

➤ Analysons la page DetailsCv



Router Resolver

- Le **Router Resolver** d'Angular est un mécanisme qui permet de **résoudre des données avant qu'une route ne soit chargée**.
- Il permet de **charger les données nécessaires** pour afficher une vue spécifique en **utilisant un service qui est lié à la route**.
- Il est utilisé lorsque vous avez besoin de **charger des données avant d'afficher une vue**, comme pour afficher des données d'un utilisateur avant d'afficher sa page de profil.
- Il est également utilisé pour **éviter les erreurs de chargement de vue** en **garantissant** que les données nécessaires sont **chargées avant de naviguer vers une route**.

Router Resolver

- Le **Router Resolver** est soit une fonction (à partir d'Angular 15) soit une classe qui implémente l'interface **Resolve** et qui est **générique**. Vous devez donc spécifier ce que le Resolver va fournir comme données.
- Vous devez implémenter la **fonction Resolve** qui devra **retourner** un **objet de T** ou une **Promise<T>** ou un **Observable<T>**.
- Elle prend en paramètre un objet de type **ActivatedRouteSnapshot** qui vous permet de **récupérer les paramètre de votre route** à travers le paramètre **paramMap** et sa méthode **get**.

Router Resolver

```
import { ResolveFn } from '@angular/router';

export const firstResolver: ResolveFn<boolean> = (route, state) => {
  return true;
};
```

Router Resolver

```
@Injectable({  
  providedIn: 'root',  
})  
export class CvResolver implements Resolve<Cv> {  
  resolve(  
    route: ActivatedRouteSnapshot,  
    state: RouterStateSnapshot  
  ): Observable<Cv> | Cv {  
    const cvId = route.paramMap.get('id');  
  }  
}
```

Router Resolver

- Maintenant, afin de passer le résultat de votre resolver à votre route, ajoutez une propriété **resolve** à votre route dans votre fichier de routing et passez lui un **object** avec comme clé le nom que vous voulez donner à votre propriété et comme valeur le resolver.

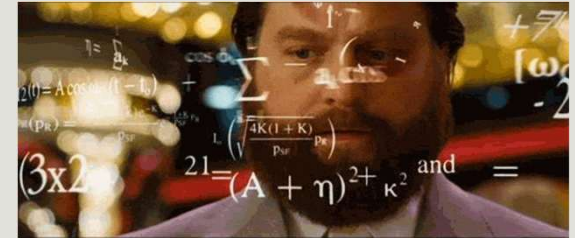
```
{  
  path: ':id',  
  component: DetailsCvComponent,  
  resolve: {  
    cv: CvResolver  
  }  
},
```

Router Resolver

- Maintenant, dans votre composant, injecter le service **ActivatedRoute**.
- Pour accéder à la valeur de retour du resolver (si c'est une Promise ou un Observable, il attendra la valeur émise), accéder à la propriété **snapshot** qui contient une **propriété data** qui **contiendra le champ**.
- Si vous voulez un accès **dynamique**, utilisez **l'Observable data de l'ActivatedRoute**.

```
this.activatedRoute.snapshot.data['cv'];
```

Exercice



- Appliquez le resolver pour le composant MasterDetailsComponent

Route Provider

A partir d'Angular 14

- A partir d'Angular 14, la clé provider a été introduite dans l'objet route.
- Ceci permettra de provider des provider pour la route et ses enfants

```
{  
  path: 'routerProvider',  
  component: RouterPoviderComponent,  
  providers: [LoggerService]  
},
```

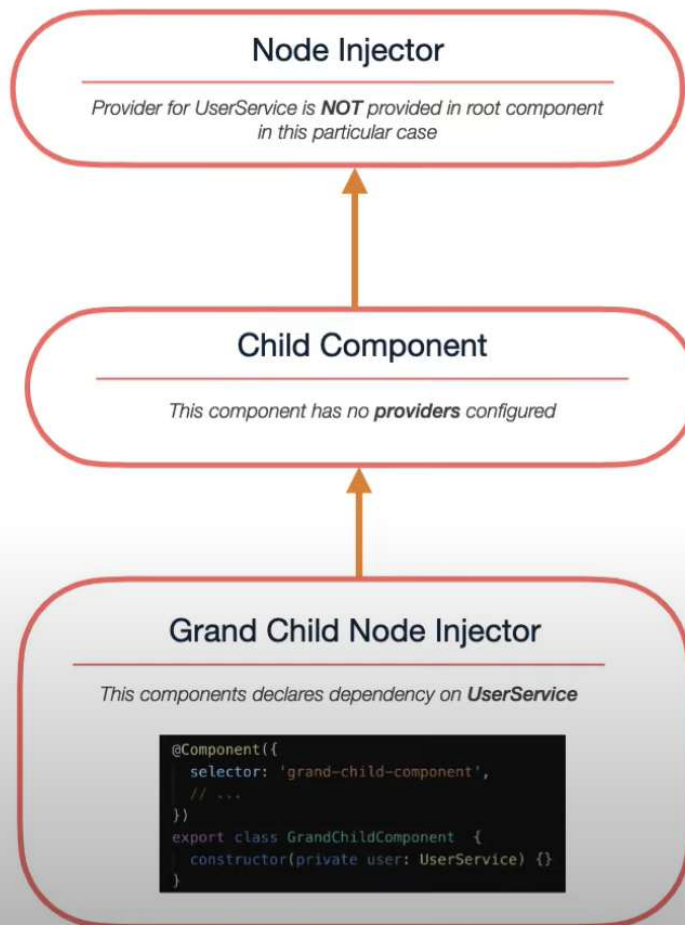

Route Provider

A partir d'Angular 14

- Ceci va créer un nouvel *Injector* qui sera appelé juste après *l'element Injector*

```
{  
  path: 'routerProvider',  
  component: RouterPoviderComponent,  
  providers: [ LoggerService ]  
},
```

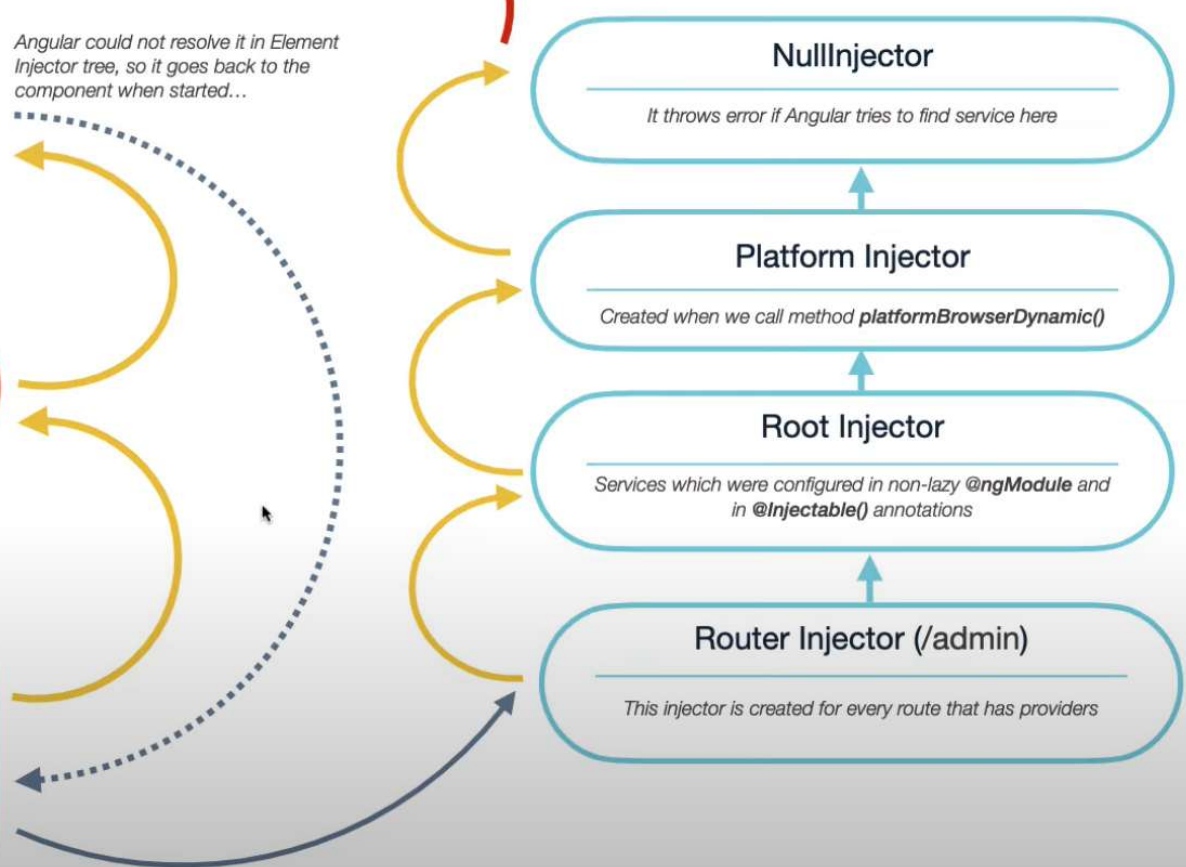
Node Injector Hierarchy



Angular could not resolve it in Element Injector tree, so it goes back to the component when started...

Error will be thrown

Environment Injector Hierarchy



Guard

- Ce sont des classes qui permettent de gérer l'accès à vos routes.
- Un guard informe sur la validité ou non de la continuation du process de navigation en retournant un booléen, une promesse d'un booléen ou un observable d'un booléen.
- Le routeur supporte plusieurs types de guards, par exemple :
 - `CanActivate` permettre ou non l'accès à une route.
 - `CanActivateChild` permettre ou non l'accès aux routes filles.
 - `CanDeactivate` permettre ou non la sortie de la route.

Guard / canActivate

- Afin d'utiliser le guard `canActivate` (de même pour les autres), vous devez créer une classe qui implémente l'interface `CanActivate` et donc qui doit implémenter la méthode `canActivate` de sorte qu'elle retourne un booléen permettant ainsi l'accès ou non à la route cible. 1
- Vous devez ensuite provider cette classe. 2
- Finalement pour l'appliquer à une route, ajouter la dans la propriété `canActivate`. Cette propriété prend un tableau de guard. Elle ne laissera l'accès à la route que si la totalité des guard retourne true. 3
- Vous pouvez utiliser la méthode : `ng g g nomGuard`

Guard / canActivate

```
import { Injectable } from '@angular/core';
import { ActivatedRouteSnapshot, CanActivate, RouterStateSnapshot } from '@angular/router';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {
  constructor() {
    // route contient la route appelé
    // state contiendra la futur état du routeur de l'application qui devra passer la validation du guard
    // https://vsavkin.com/routeur-angular-comprendre-l%C3%A9tat-du-routeur-5e15e729a6df
    canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): Observable<boolean> |
    Promise<boolean> | boolean {
      if (// your condition) {
        return true;
      }
      return false;
    }
  }
}
```

Guard / canActivate

- A partir d'angular 16, les classes Guards sont devenues deprecated et on s'oriente vers l'utilisation des Fonctionnal Guards.

```
export const firstGuard: CanActivateFn = (route, state) => {  
  return true;  
};
```

```
export declare type CanActivateFn =  
(route: ActivatedRouteSnapshot, state: RouterStateSnapshot) => Observable<boolean | UrlTree> |  
Promise<boolean | UrlTree> | boolean | UrlTree;
```

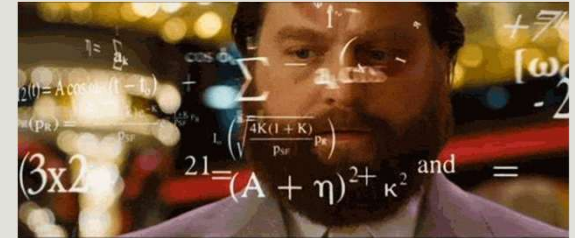
Guard / canActivate

```
{  
  path: 'lampe',  
  component: ColorComponent,  
  canActivate: [AuthGuard]  
},
```

Guard / canActivateChild

- Le guard **canActivateChild** est un service Angular qui permet de protéger les routes enfants en vérifiant si l'utilisateur a les autorisations appropriées pour accéder à ces routes.
- Il retourne true si l'utilisateur a les autorisations requises et false sinon. Cela permet de contrôler l'accès aux pages enfants d'une route donnée.

Exercice



- Toutes les routes commençant par /cv ne doivent être accessible que pour les personnes authentifiées.

Guard / canDeactivate

- Le guard **canDeactivate** est un service Angular qui permet de bloquer la sortie d'une route.
- Il retourne true si l'utilisateur a les autorisations requises pour **quitter la route** et false sinon.
- Il est généralement utilisé pour s'assurer de ne pas quitter une page au cas où il y a des données non encore enregistrées ou s'il y a un upload en cours et que vous ne voulez pas quitter la page tant que ce n'est pas encore terminé.
- Il **ne s'applique pas aux 'children Routes'**.

Guard / canDeactivate

➤ Il est **fortement couplé** au composant

```
@Component({
  selector: 'app-form',
  template: `
    <form>
      <input [(ngModel)]="formData" name="formData">
    </form>
    <button (click)="save()">Save</button>
  `
})
export class FormComponent implements OnInit,
CanDeactivate<FormComponent> {
  formData: string;
  saved = false;
  save() {
    this.saved = true;
  }
}
```

```
canDeactivate(
  component: FormComponent,
  currentRoute: ActivatedRouteSnapshot,
  currentState: RouterStateSnapshot,
  nextState?: RouterStateSnapshot
): Observable<boolean> | Promise<boolean> | boolean {
  if (!this.saved && this.formData !== '') {
    return confirm('Are you sure you want to leave? You have
unsaved changes.');
```

Guard / canDeactivate

```
export const deactivateGuard: CanDeactivateFn<unknown> =  
(component, currentRoute, currentState, nextState) => {  
  return true;  
};
```


RxJs

AYMEN SELLAOUTI

Qu'est ce que la programmation réactive

1. Nouvelle manière d'appréhender les appels asynchrones
2. Programmation avec des flux de données asynchrones

Programmation réactive =
Flux de données (observable) + écouteurs d'événements(observer).

C'est un ensemble de réaction suite à des événements.

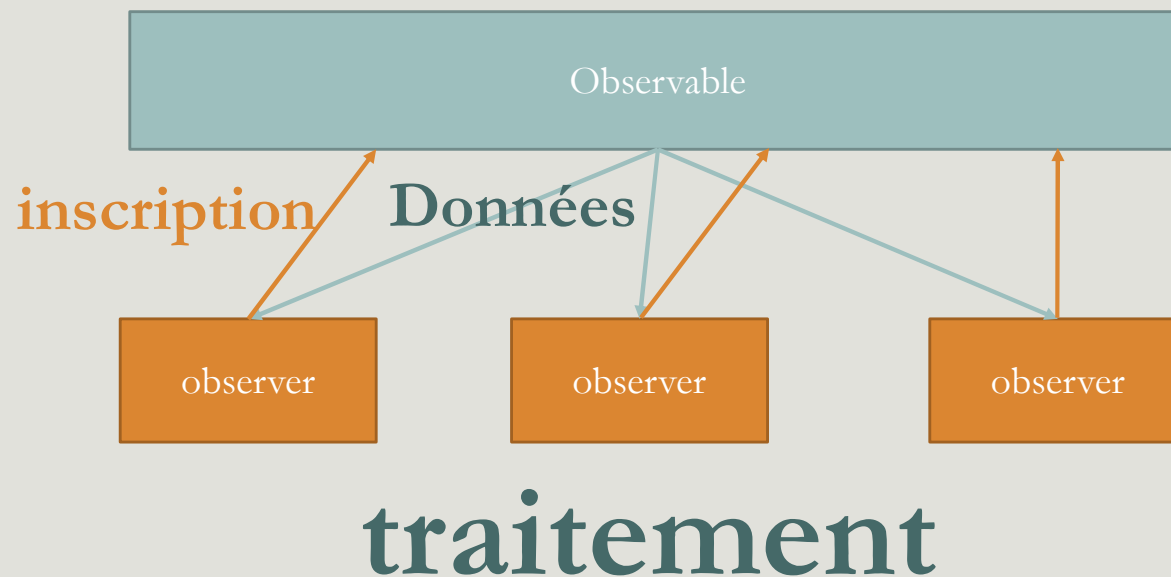
Qu'est ce que la programmation réactive

- La programmation réactive nous permet donc de **réagir à divers événements** tel qu'une écoute sur un **click, un timer ou un interval** qui déclenche un traitement, une **requête http**.
- Elle va nous permettre **d'uniformiser la gestion de ses événements**.
- Actuellement pour une requête **http** c'est un **fetch**, pour un **event** dans le navigateur c'est un **addEventListener**, ...
- L'idée est donc de **normaliser tout ça**.
- Ceci nous permettra de les **fusionner**, les **combiner** ...

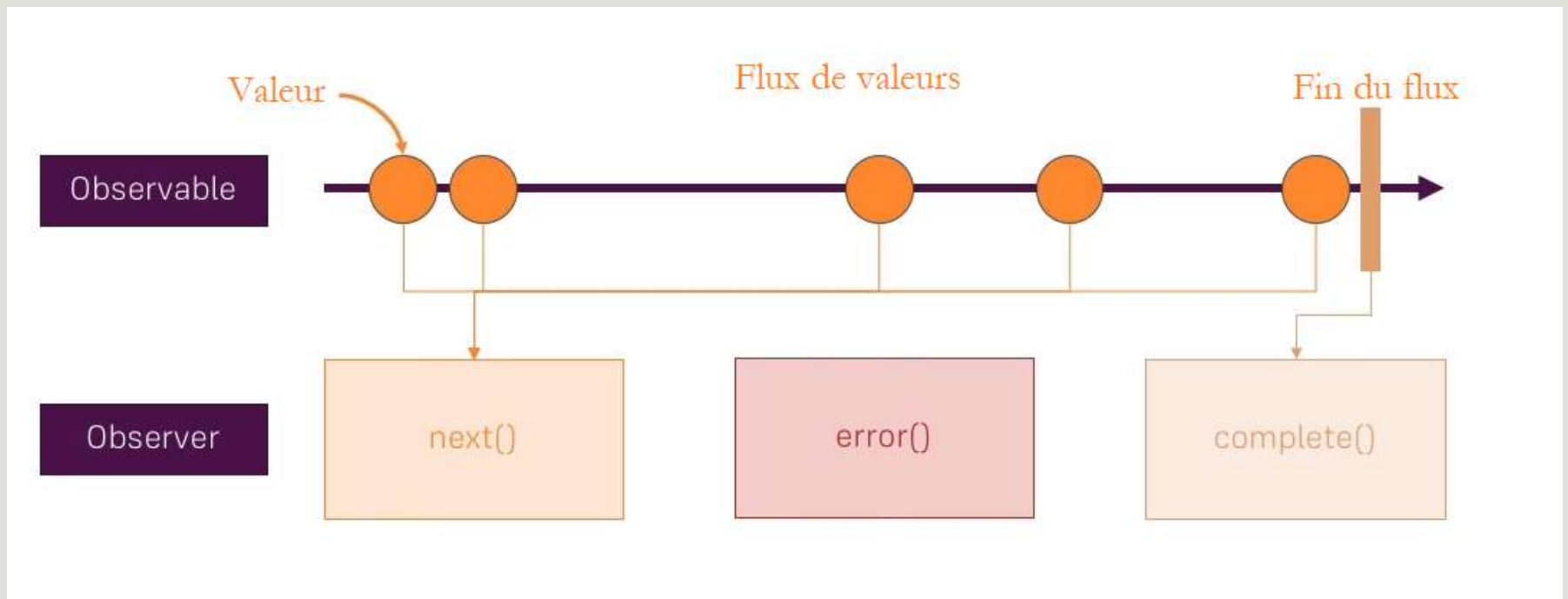
Le pattern « Observer »

- Le patron de conception **observer** permet à un objet de garder la trace d'autres objets, intéressés par l'état de ce dernier.
- Il définit une relation entre objets de type **un-à-plusieurs**.
- Lorsque l'état de cet objet **change**, il **notifie** ces **observateurs**.

Observables, Observers et subscriptions



Fonctionnement



Promesse Vs Observable

Promesse	Observable
Un promesse gère un seul événement	Un observable gère un « flux » d'événements.
Non annulable.	Annulable.
Traitement immédiat.	Lazy : le traitement n'est déclenché qu'à la première utilisation du résultat.
Deux méthodes uniquement (then/catch).	Une centaine d'opérateurs de transformation natifs (map, reduce, merge, filter, ...).
	Opérateurs tels que retry

S'inscrire à un observable

```
myObservable$.subscribe({  
  next: (data) => {  
    //TODO: implementez le comportement quand vous recevez les données  
  },  
  error: (error) => {  
    //TODO: implementez le comportement quand vous avez une erreur  
  },  
  complete: () => {  
    //TODO: implementez le comportement à la fin du flux  
  },  
});
```

Exemple d'un Observable

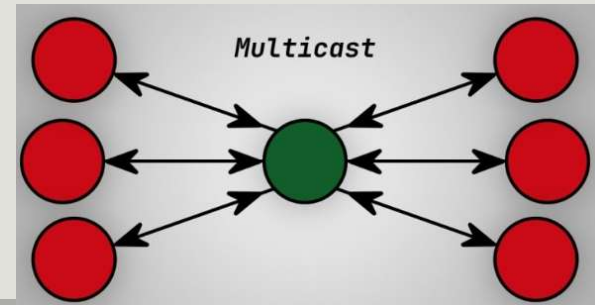
Création et inscription

```
export class TestObservableComponent {  
  myObservable$: Observable<number>;  
  constructor() {  
    this.myObservable$ = new Observable((observer) => {  
      let i = 5;  
      const intervalIndex = setInterval(() => {  
        if (!i) {  
          observer.complete();  
          clearInterval(intervalIndex);  
        }  
        observer.next(i--);  
      }, 1000);  
    });  
    this.myObservable$.subscribe({  
      next: (val) => {  
        console.log(val);  
      },  
    });  
  }  
}
```



Hot Vs Cold Observable

- Les **Cold Observables** commencent à émettre des valeurs uniquement quand on s'y inscrit. Les **Hot observables**, par contre émettent toujours.
- Les **Cold Observables** diffusent un flux par inscrit, ils sont **unicast**. Chaque nouvelle inscription crée un **nouveau contexte d'exécution**.
- Les **Hot observables**, sont **multicast**, le **même flux est partagé par tous les inscrits**.
- Dans les **Cold Observables**, la **source de données est à l'intérieur** de l'observable.
- Dans les **HotObservables**, la **source de données est à l'extérieur** de l'observable.



Quelques exemples de Hot Observable

- `fromEvent`
- Les Subject RxJs
- Quelques opérateurs telque `share()` et `shareReplay()`

Fonction Pure

- Une fonction est dite pure si elle :
 - Ne provoque pas de side effect.
 - Renvoie la même valeur pour les mêmes paramètres.
- Le meilleur exemple illustrant l'intérêt des fonction pure dans Angular est l'exemple des pipes.

Les operateurs de l'observable

- Les opérateurs sont des fonctions. Il y a deux types d'opérateurs :
- **Les opérateurs de création**, elles permettent de créer un Observable. Par exemple : `of(1, 2, 3)` crée un observable qui va émettre 1, 2, et 3, l'un après l'autre.
- **Un opérateur pipeable** est une fonction qui prend un observable comme entrée et renvoie un autre observable. C'est **une opération pure** : le précédent Observable reste inchangé.
 - Syntaxe : `monObservable.pipe(opertaeur1(), operateur2(), ...)`.

Les operateurs de création

- Les opérateurs de création sont utilisés pour créer de nouveaux observables.
- Ils sont divisés en **opérateurs de création** et en **opérateurs de création de jointure**.
- La principale différence entre eux réside dans le fait que les opérateurs de création de **jointure créent des observables à partir d'autres observables**, alors que les **opérateurs de création** créent des observables **à partir d'objets qui diffèrent des observables**.

Les operateurs de création

from

- **from** est utilisé pour **convertir des types d'objets JavaScript** comme un **tableau** ou **un objet itérable** en une séquence **observable** de valeurs.
- L'opérateur émet également une **chaîne** sous forme de **séquence de caractères**.

```
from([1, 2, 3]).subscribe({  
  next: (data) => console.log('[from]', data),  
  complete: () => console.log('[from] complete'),  
});
```

[from]	1
[from]	2
[from]	3
[from]	complete

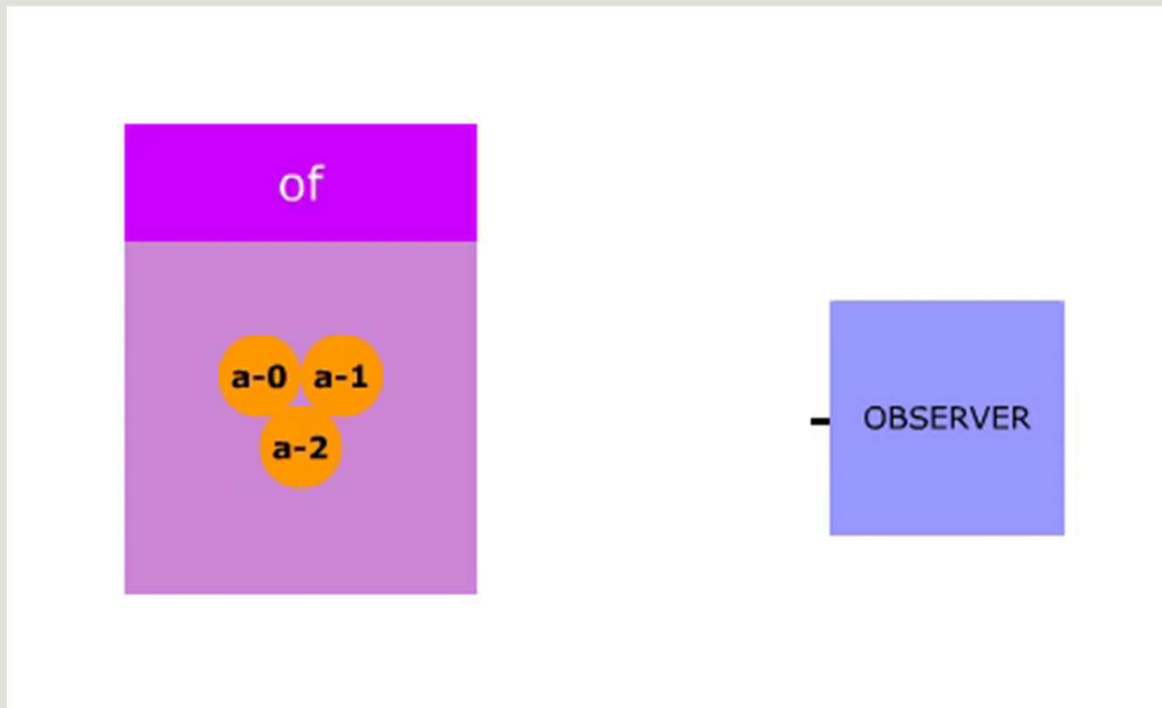
Les operateurs de création of

- l'opérateur **of** est utilisé afin de **convertir un argument en un observable**
- il ne fait **aucun aplatissement ou conversion** et **émet** chaque argument sous le **même type qu'il reçoit**
- of est couramment utilisé lorsque vous avez simplement **besoin de renvoyer une valeur là où un observable est attendu** ou de **démarrer une chaîne observable**.

```
of([1, 2, 3], new Date(), {  
  name: 'sellaouti',  
  firstname: 'aymen',  
}).subscribe({  
  next: (data) => console.log('[of]', data),  
})
```

```
[of] ▶ (3) [1, 2, 3]  
[of] Tue Jan 03 2023 13:46:43 GMT+0100 (West Africa Standard Time)  
[of] ▶ {name: 'sellaouti', firstname: 'aymen'}  
[of] complete
```

Les opérateurs de création of



Les operateurs de création

tap

- L'opérateur **tap** est utilisé pour gérer les effets de bord.
- En effet, il n'a aucun effet sur le flux de données et il vous permet de faire des opérations mais sans toucher au flux.

```
from([1, {name: 'aymen'}, 3])  
  .pipe(  
    tap((data) => console.log(data))  
  )  
)
```

Les operateurs de création timer

- L'opérateur **timer** est un opérateur de création utilisé pour créer un observable qui commence à émettre les valeurs après un délai d'attente, et la valeur continuera d'augmenter après chaque appel.
- Il prend en **entrée** en **premier paramètre quand déclencher** l'événement.
- En **second paramètre** en cas de volonté de répétition chaque **combien de millisecondes reprendre l'émission**.

```
timer(1000).subscribe((val) => console.log('[timer] : '+ val));
```

```
[timer] : 0
```

```
timer(1000, 1000).subscribe((val) => console.log('[timer] : '+ val));
```

```
[timer] : 0
```

```
[timer] : 1
```

```
[timer] : 2
```

```
[timer] : 3
```

```
[timer] : 4
```


Les operateurs de création fromEvent

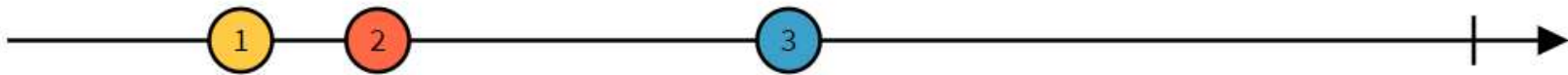
- L'opérateur **fromEvent** est utilisé pour émettre un observable en se basant sur un événement.
- Il prend en paramètre l'élément cible puis l'événement à écouter.

```
export class FromEventComponent implements AfterViewInit {  
  @ViewChild('btn') button!: ElementRef;  
  onClickButton$: Observable<Event>;  
  ngAfterViewInit() {  
    this.onClickButton$ = fromEvent(this.button.nativeElement, 'click');  
  }  
}
```

Quelques opérateurs utiles de l'Observable

opérateur pipable

map



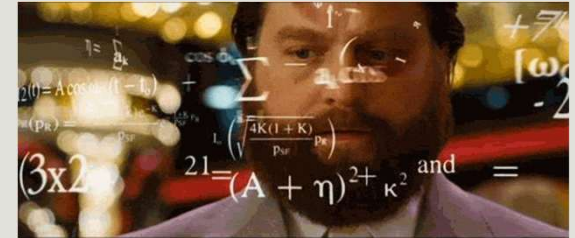
`map(x => 10 * x)`



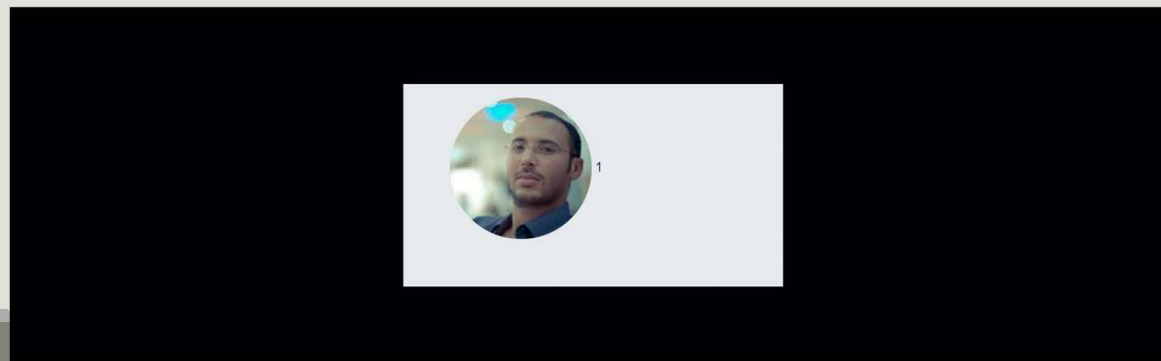
asyncPipe

- **async** Pipe est un pipe qui permet d'afficher directement les valeurs émises par un **observable**.
- `{{ valeurSourceAsynchrone | async }}`
- L'asyncPipe **s'inscrit automatiquement** à l'observable et affiche le **dernier résultat envoyé**.
- Quand le **composant** est **détruit** l'asyncPipe se **désinscrit** automatiquement de l'observable.

Exercice



- Ecrire un composant qui affiche une suite d'images non stop.
- Utiliser un observable comme source d'images.
- Vous devez uniquement utiliser des opérateurs pour faire le travail.
- La taille de l'image, la liste des images et le temps entre chaque image doit être paramétrable.



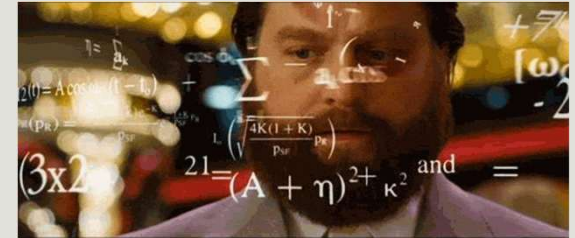
asyncPipe

L'opérateur as

- Si vous utilisez le même observable plusieurs fois dans votre template vous serez amené à réutiliser async plusieurs fois.
- Afin d'éviter ça, vous pouvez utiliser l'opérateur **as** qui vous permettra de récupérer le résultat émis par votre source observable dans une variable que vous pourrez utiliser plusieurs fois.

```
<ng-container *ngIf="data$ | async as data">  
  
</ng-container>
```

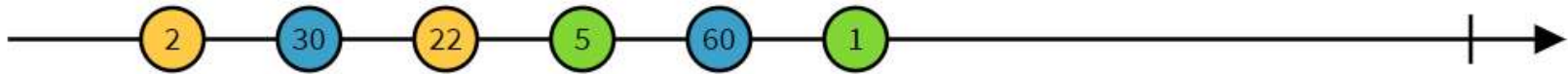
Exercice



- Utilisez le asyncPipe afin de réécrire le **DetailsCvComponent**
- **Ps : Ignorer pour le moment la partie Gestion des erreurs on y reviendra après.**

Quelques opérateurs utiles de l'Observable opérateur pipable

filter



```
filter(x => x > 10)
```

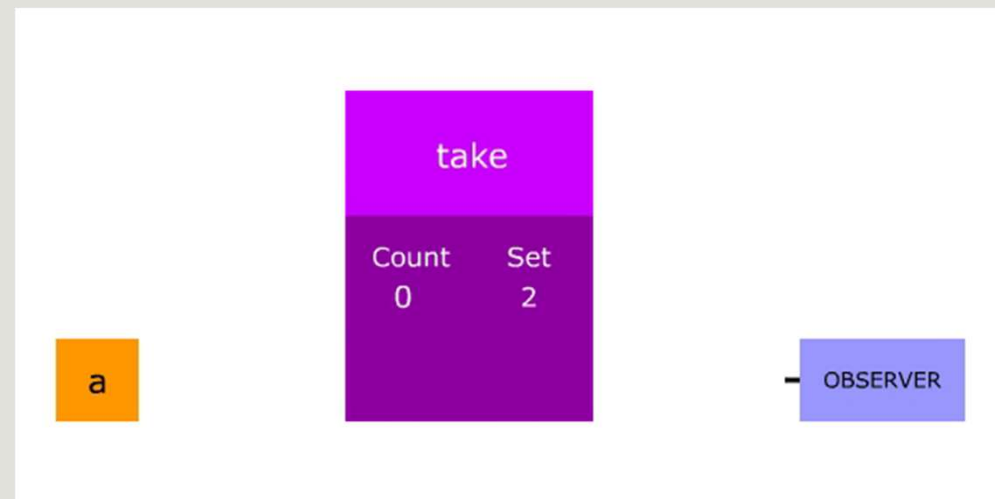


Quelques opérateurs utiles de l'Observable

opérateur pipable

take

- L'opérateur **take** prend des valeurs de la source observable **jusqu'à ce qu'il atteigne le seuil de valeurs défini** pour l'opérateur.
- Sur chaque valeur, **take** compare le nombre de valeurs qu'il a mises en miroir avec le seuil défini. Si le **seuil est atteint**, il termine le flux en se **désabonnant** de la source et en transmettant la notification **complete** à l'observateur.

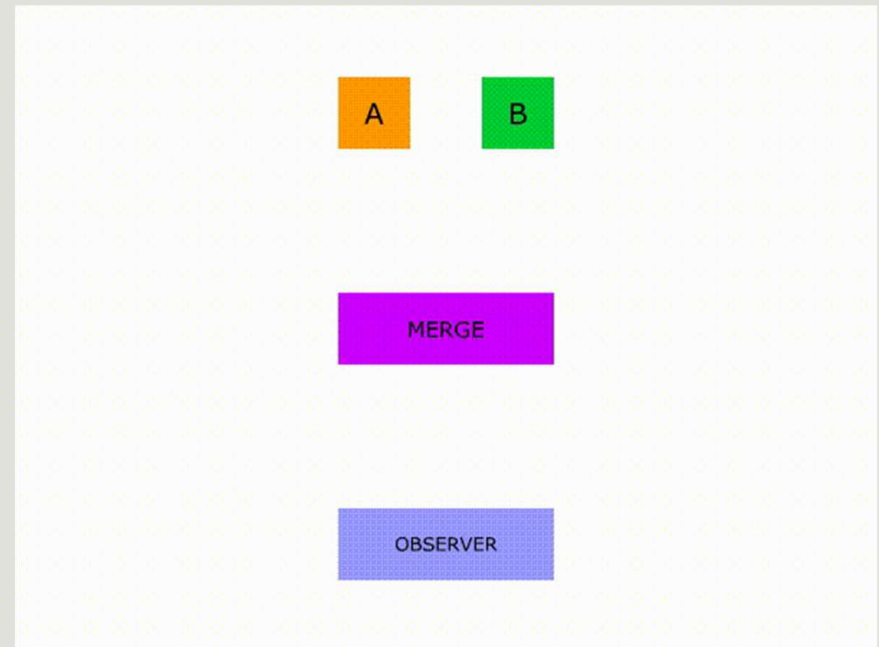


Quelques opérateurs utiles de l'Observable

opérateur pipable

merge

- **merge** combine un certain nombre de flux observables et émet simultanément toutes les valeurs de chaque flux d'entrée donné.
- Au fur et à mesure que les valeurs d'une séquence combinée sont produites, ces valeurs sont émises dans le flux résultant.
- Utilisez cet opérateur si vous n'êtes pas concerné par l'ordre des émissions de flux et si vous êtes simplement intéressé par toutes les valeurs provenant de plusieurs flux combinés comme si elles étaient produites par un seul flux.



Quelques opérateurs utiles de l'Observable

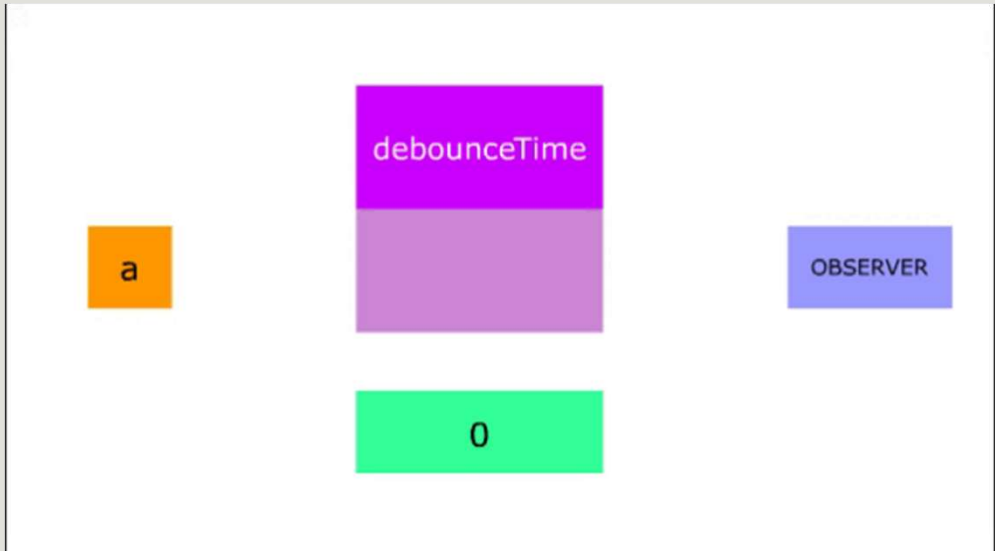
opérateur pipable

debounceTime

➤ **debounceTime** retarde les valeurs émises par une source pour le **temps d'échéance donné**. Si dans ce délai une **nouvelle valeur arrive**, la **valeur en attente précédente est supprimée** et l'intervalle est réinitialisé.

➤ De cette façon, **debounceTime** garde une trace de la **valeur la plus récente** et émet cette valeur la plus récente lorsque l'heure d'échéance donnée est dépassée.

➤ **L'autocomplete** est le **cas classique** du **debounceTime**



Les opérateurs d'aplatissement/ flattening operators

- Une erreur courante commise dans les applications Angular consiste à **imbriquer les abonnements observables**.
- Cette syntaxe **n'est pas recommandée** car elle est **difficile à lire et peut entraîner des bogues des effets secondaires inattendus**.
- Par exemple, cette syntaxe rend **difficile la désinscription** correcte de tous ces observables.
- De plus, si observable1 émet plus d'une fois dans un court laps de temps, **nous pourrions vouloir annuler l'abonnement précédent à observable2** et en démarrer un nouveau basé sur les nouvelles données reçues d'observable1.

```
this.activatedRoute.params.subscribe(  
  (params) => {  
    this.cvService.findPersonneById(params.id)  
      .subscribe(  
        (personne) => {  
          this.personne = personne;  
        },  
        (erreur) => {  
          if (!this.personne) {  
            this.router.navigate(['cv']);  
          }  
        })  
      );  
  })  
);
```

Les opérateurs d'aplatissement/ flattening operators

- L'opérateur **d'aplatissement** sont des opérateur qui permettent d'émettre un flux à partir d'un autre.
- Ils permettent **d'éviter les inscriptions imbriquées avec subscribe**.
- Il en existe plusieurs variantes et qui permettent de spécifier comment gérer les flux récupérés :
 - Est-ce qu'on est encore intéressé par l'inscription précédente ?
 - Est-ce que l'ordre des inscriptions est important ?

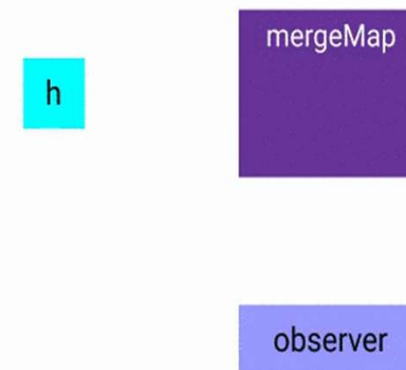
Les opérateurs d'applatissage/ flattening operators MergeMap

- L'opérateur **mergeMap** est essentiellement une **combinaison** de deux opérateurs - **merge** et **map**.
- La partie **map** vous permet de **mapper** une valeur d'une source observable à un **flux observable**. Ces flux sont souvent appelés flux internes.
- La partie **merge combine** tous les flux internes observables renvoyés par la map et **émet simultanément toutes les valeurs de chaque flux d'entrée**.

Les opérateurs d'applatissage/ flattening operators

MergeMap

- Au fur et à mesure que les valeurs de toute séquence combinée sont produites, ces valeurs sont émises dans le cadre de la séquence résultante.
- Utilisez cet opérateur **si vous n'êtes pas concerné par l'ordre des émissions** et que vous êtes simplement **intéressé par toutes les valeurs provenant de plusieurs flux combinés** comme si elles étaient produites par un seul flux.



Les opérateurs d'applatissage/ flattening operators

MergeMap

```
timerObs(timer: number, name: string, iteration = 4) {  
  return new Observable((observer: Observer<string>) => {  
    let i = 0;  
    const x = setInterval(() => {  
      if (i >= iteration) {  
        observer.complete();  
        clearInterval(x);  
      }  
      observer.next(`observable ${name} ${++i}`);  
    }, timer);  
  });  
}
```

observable	obs1	1
observable	obs2	1
observable	obs1	2
observable	obs1	3
observable	obs2	2
observable	obs1	4
observable	obs2	3
observable	obs2	4

```
params = [  
  {name: 'obs1', timer: 1000, iteration: 4 },  
  {name: 'obs2', timer: 1500, iteration: 4 },  
];  
constructor(private rxjsService: RxjsService) {  
  from(this.params).pipe(  
    mergeMap((param) => this.rxjsService.timerObs(param.timer,  
    param.name))  
  ).subscribe(  
    (data) => console.log(data)  
  )  
}
```

Les opérateurs d'applatissage/ flattening operators

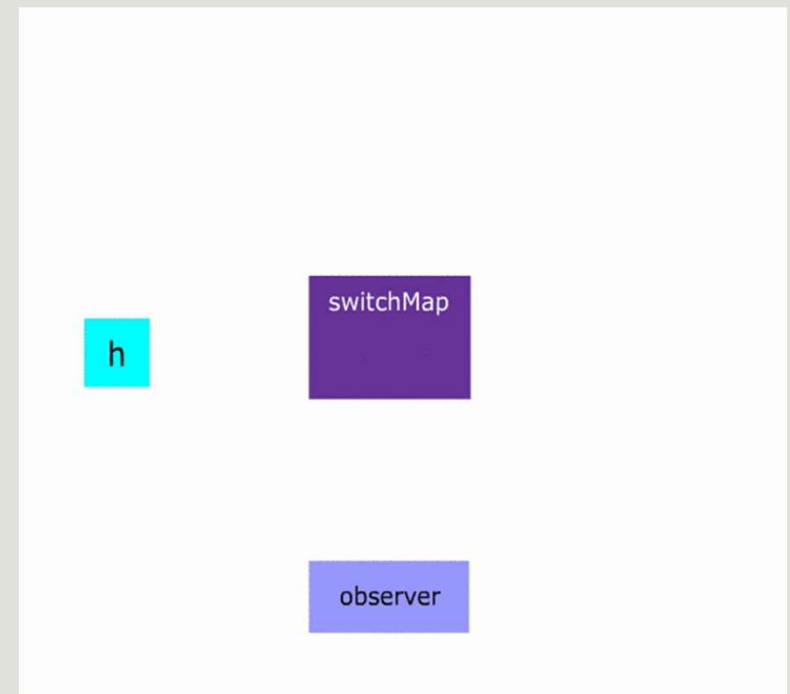
SwitchMap

- L'opérateur **switchMap** est essentiellement une combinaison de deux opérateurs - **switchAll** et **map**.
- La partie de **map** vous permet de mapper une valeur d'une source observable d'ordre supérieur à un flux observable interne.
- La partie **switch** s'abonne à l'observable interne fourni le plus récemment émis par un observable d'ordre supérieur, en se désabonnant de tout observable interne précédemment souscrit.
- L'opérateur **switchMap** effectue toutes les actions suivantes :
 - **Annule et se désabonne automatiquement** du second observable lorsque le premier émet une nouvelle valeur.
 - Se **désabonne automatiquement du second observable** si nous nous **désinscrivons du premier**.
 - S'assure que les deux observables se **produisent en séquence**, l'un après l'autre.

Les opérateurs d'applatissage/ flattening operators

SwitchMap

- **switchMap** n'a **qu'un seul abonnement actif à la fois** à partir duquel les valeurs sont transmises à un observateur.
- Une fois que l'observable d'ordre supérieur **émet une nouvelle valeur, switchMap exécute la fonction pour obtenir un nouveau flux observable interne et commute les flux.** Il se désabonne du flux actuel et s'abonne au nouvel observable interne.



Les opérateurs d'applatissage/ flattening operators

SwitchMap

- Une erreur courante commise dans les applications Angular consiste à **imbriquer les abonnements observables**.
- Cette syntaxe **n'est pas recommandée** car elle est **difficile à lire et peut entraîner des bogues et des effets secondaires inattendus**.
- Par exemple, cette syntaxe rend **difficile la désinscription** correcte de tous ces observables.
- De plus, si observable1 émet plus d'une fois dans un court laps de temps, **nous pourrions vouloir annuler l'abonnement précédent à observable2** et en démarrer un nouveau basé sur les nouvelles données reçues d'observable1.

```
this.activatedRoute.params.subscribe(
  (params) => {
    this.cvService.findPersonneById(params.id)
      .subscribe(
        (personne) => {
          this.personne = personne;
        },
        (erreur) => {
          if (!this.personne) {
            this.router.navigate(['cv']);
          }
        }
      );
  }
);
```

Les opérateurs d'applatissage/ flattening operators SwitchMap

```
timerObs(timer: number, name: string, iteration = 4) {  
  return new Observable((observer: Observer<string>) => {  
    let i = 0;  
    const x = setInterval(() => {  
      if (i >= iteration) {  
        observer.complete();  
        clearInterval(x);  
      }  
      observer.next(`observable ${name} ${++i}`);  
    }, timer);  
  });  
}
```

```
params = [  
  {name: 'obs1', timer: 1000, iteration: 4 },  
  {name: 'obs2', timer: 1500, iteration: 4 },  
];  
constructor(private rxjsService: RxjsService) {  
  from(this.params).pipe(  
    switchMap((param) => this.rxjsService.timerObs(param.timer,  
param.name))  
  ).subscribe(  
    (data) => console.log(data)  
  )  
}
```

```
observable obs2 1  
observable obs2 2  
observable obs2 3  
observable obs2 4
```

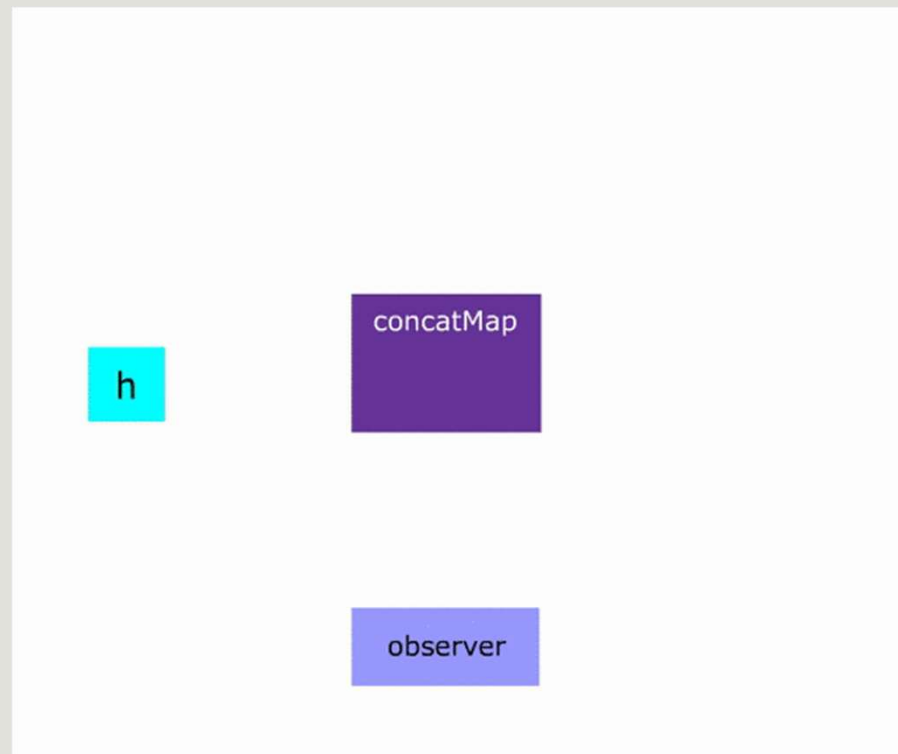
Les opérateurs d'applatissage/ flattening

operators

ConcatMap

- L'opérateur **concatMap** est essentiellement une combinaison de deux opérateurs - **concat** et **map**.
- La partie **map** vous permet de mapper une valeur d'une source observable à un flux observable. Ces flux sont souvent appelés flux internes.
- La partie **concat combine** tous les flux internes observables renvoyés par la **map** et **émet séquentiellement toutes les valeurs** de chaque flux d'entrée.
- Utilisez cet opérateur **si l'ordre des émissions est important** et que vous souhaitez d'abord voir les valeurs émises par les flux qui passent par l'opérateur en premier.

Les opérateurs d'applatissage/ flattening operators ConcatMap



Les opérateurs d'applatissage/ flattening operators ConcatMap

```
timerObs(timer: number, name: string, iteration = 4) {  
  return new Observable((observer: Observer<string>) => {  
    let i = 0;  
    const x = setInterval(() => {  
      if (i >= iteration) {  
        observer.complete();  
        clearInterval(x);  
      }  
      observer.next(`observable ${name} ${++i}`);  
    }, timer);  
  });  
}
```

observable	obs1	1
observable	obs1	2
observable	obs1	3
observable	obs1	4
observable	obs2	1
observable	obs2	2
observable	obs2	3
observable	obs2	4

```
params = [  
  {name: 'obs1', timer: 1000, iteration: 4 },  
  {name: 'obs2', timer: 1500, iteration: 4 },  
];  
constructor(private rxjsService: RxjsService) {  
  from(this.params).pipe(  
    concatMap((param) => this.rxjsService.timerObs(param.timer,  
param.name))  
  ).subscribe(  
    (data) => console.log(data)  
  )  
}
```

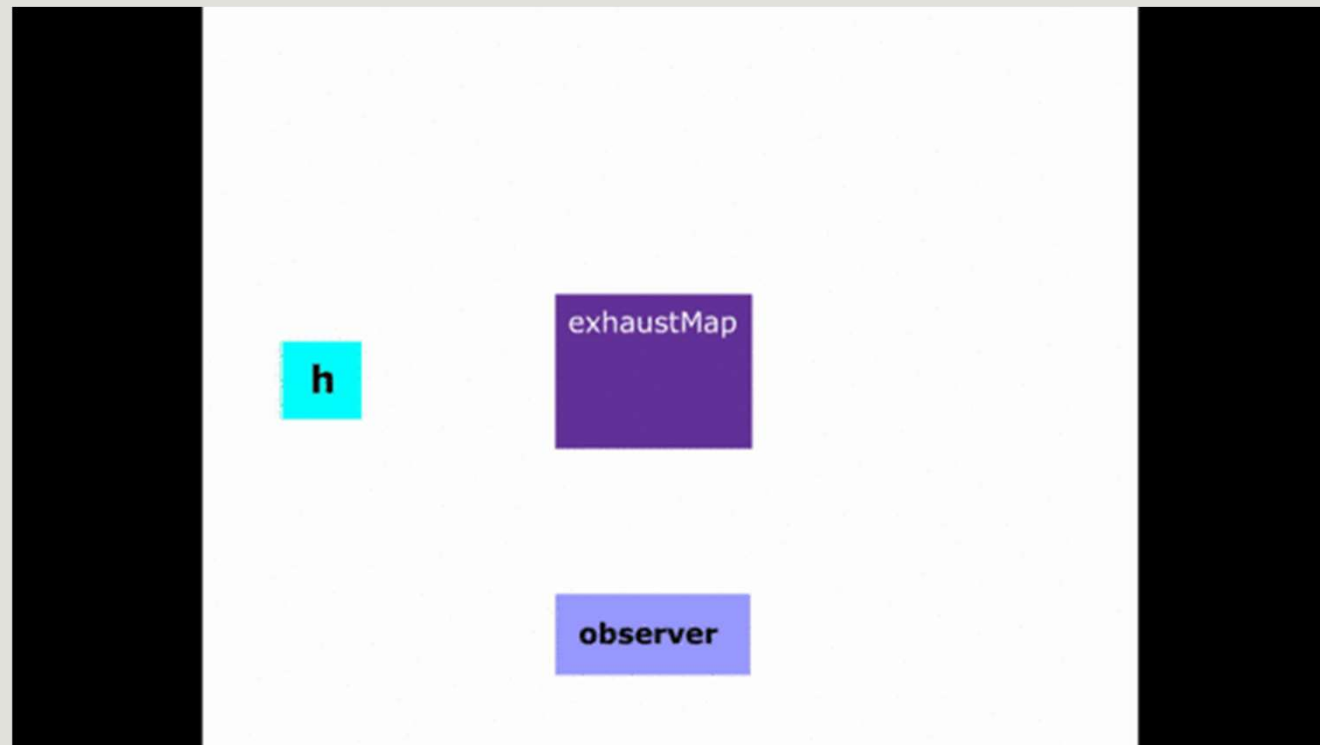
Les opérateurs d'applatissage/ flattening operators

ExhaustMap

- L'opérateur **exhaustMap** est essentiellement une combinaison de deux opérateurs - **exhaust** et **map**.
- La partie **map** vous permet de mapper une valeur d'une source observable d'ordre supérieur à un flux observable interne.
- La partie **exhaust** s'abonne ensuite à un observable interne et transmet des valeurs à un observateur s'il n'y a pas déjà d'abonnement actif, sinon il ignore simplement les nouveaux observables internes.
- **exhaustMap** n'a **qu'un seul abonnement actif à la fois** à partir duquel les valeurs sont transmises à un observateur. Lorsque l'observable d'ordre supérieur émet un nouveau flux observable interne, **si le flux actuel n'est pas terminé, ce nouvel observable interne est abandonné**.
- Une fois le flux actif en cours terminé, l'opérateur attend qu'un autre observable interne s'abonne en ignorant les observables internes précédents.

Les opérateurs d'applatissage/ flattening operators

ExhaustMap



Les opérateurs d'applatissage/ flattening operators ExhaustMap

```
timerObs(timer: number, name: string, iteration = 4) {  
  return new Observable((observer: Observer<string>) => {  
    let i = 0;  
    const x = setInterval(() => {  
      if (i >= iteration) {  
        observer.complete();  
        clearInterval(x);  
      }  
      observer.next(`observable ${name} ${++i}`);  
    }, timer);  
  });  
}
```

```
params = [  
  {name: 'obs1', timer: 1000, iteration: 4 },  
  {name: 'obs2', timer: 1500, iteration: 4 },  
];  
constructor(private rxjsService: RxjsService) {  
  from(this.params).pipe(  
    exhaustMap((param) => this.rxjsService.timerObs(param.timer,  
param.name))  
  ).subscribe(  
    (data) => console.log(data)  
  )  
}
```

observable	obs1	1
observable	obs1	2
observable	obs1	3
observable	obs1	4

Les opérateurs d'applatissage/ flattening

operators

combineLatest

- **combineLatest** permet de fusionner plusieurs flux en prenant la valeur la plus récente de chaque entrée observable et en émettant ces valeurs à l'observateur sous forme de sortie combinée (généralement sous forme de tableau).
- Les opérateurs **mettent en cache la dernière valeur pour chaque observable d'entrée** et seulement une fois que **tous les observables d'entrée ont produit au moins une valeur**, il **émet les valeurs mises en cache combinées à l'observateur**.
- Le flux résultant **se termine lorsque tous les flux internes sont terminés** et génère une **erreur si l'un des flux internes génère une erreur**.
- D'autre part, si **un flux n'émet pas de valeur mais se termine**, le **flux résultant se terminera au même moment sans rien émettre**.
- De plus, si un flux d'entrée n'émet aucune valeur et ne se termine jamais, combineLatest n'émettra jamais et ne se terminera jamais.

Les opérateurs d'applatissage/ flattening operators combineLatest

```
const cv$ = this.cvService.loadCvById(cvId)
  .pipe(startWith(null));

const skills$ = this.cvService.loadAllCvSkills(cvId);

this.data$ = combineLatest([cv$, skills$])
  .pipe(
    map(([cv, skills]) => {
      return {
        cv,
        skills
      }
    })
  );
```



Les opérateurs d'applatissage/ flattening operators combineLatest

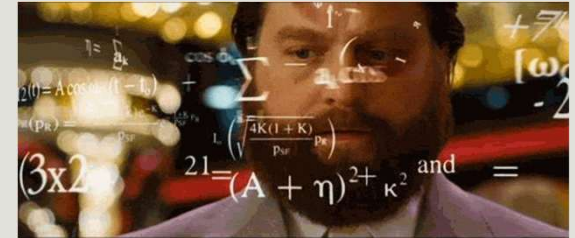
```
const cv$ = this.cvService.loadCvById(cvId)
  .pipe(startWith(null));

const skills$ = this.cvService.loadAllCvSkills(cvId)
  .pipe(startWith([]));

this.data$ = combineLatest([cv$, skills$])
  .pipe(
    map(([cv, skills]) => {
      return {
        cv,
        skills
      }
    })
  );
```

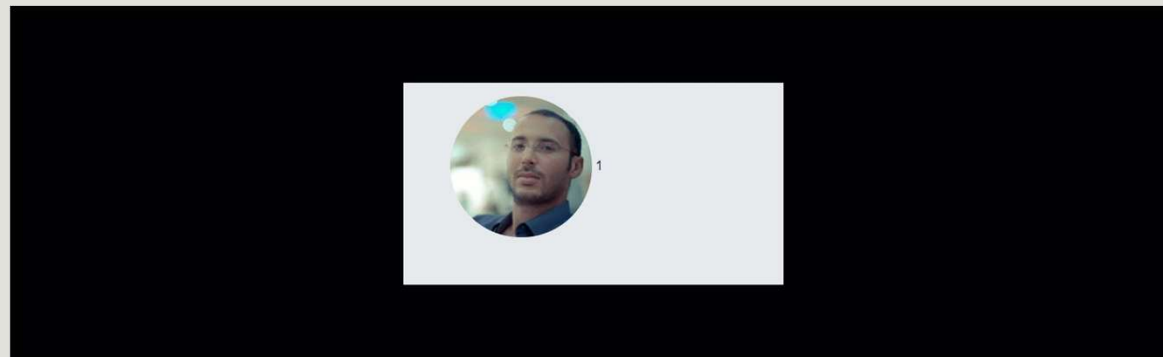


Exercice



- Reprenez votre slider et faites en sorte que la source de données à entrer soit un observable.
- Prenez l'exemple de l'api suivante :

<https://fakestoreapiserver.reactbd.com/photos>

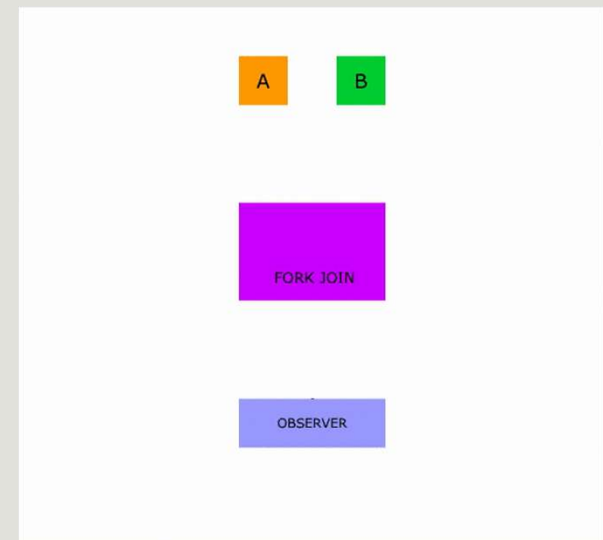


Les opérateurs d'applatissage/ flattening operators `combineLatest`

- **`combineLatest`** est utilisé par exemple pour implémenter un view model stream.
- Un View Model Stream est un observable qui représente l'état de votre vue et combine toutes les sources observables gérées dans votre vue.
- Ceci permet d'avoir une seule inscription au niveau de la vue.

Les opérateurs d'applatissage/ flattening operators forkJoin

- **forkJoin** peut être appliqué à n'importe quel nombre d'Observables. Lorsque **tous ces Observables sont terminés**, **forkJoin** émet la **dernière valeur de chacun d'eux**.
- Un cas d'utilisation courant pour **forkJoin** est lorsque nous devons **déclencher plusieurs requêtes HTTP en parallèle** avec le HttpClient puis **recevoir toutes les réponses en même temps dans un tableau de réponses**.



Les opérateurs d'applatissage/ flattening operators forkJoin

```
forkJoin({  
  users: http.get<User[]>('https://jsonplaceholder.typicode.com/users'),  
  posts: http.get<Post[]>('https://jsonplaceholder.typicode.com/posts'),  
}).subscribe((usersAndPosts) => {  
  this.posts = usersAndPosts.posts;  
  this.users = usersAndPosts.users;  
});
```

```
▼ Object ⓘ  
  ► posts: (100) [{...}, {...},  
  ► users: (10) [{...}, {...},
```

```
forkJoin([  
  http.get<User[]>('https://jsonplaceholder.typicode.com/users'),  
  http.get<Post[]>('https://jsonplaceholder.typicode.com/posts'),  
]).subscribe(([users, posts]) => {  
  this.posts = posts;  
  this.users = users;  
});
```

```
► (10) [{...}, {...}, {...}, {...}, {...},  
  
(100) [{...}, {...}, {...}, {...}, {...},
```


Les opérateurs RxJs

Gestion des erreur

catchError

- L'opérateur **catchError** de RxJs permet de **capturer les erreurs qui se produisent dans un observable** et de les traiter de manière appropriée.
- Il permet de **continuer à émettre des valeurs depuis un observable** même si une erreur se produit, plutôt que de stopper complètement l'émission de valeurs.
- Il prend en argument une fonction qui prend en entrée l'erreur et **retourne un observable** pour gérer cette erreur.
- Si vous voulez retourner l'erreur dans votre flux utilisez l'opérateur **throwError**.

Les opérateurs RxJs

Gestion des erreur

catchError

```
this.activatedRoute.params.pipe(  
  switchMap((param) => this.cvService.getCvById(param['id'])),  
  catchError((e) => {  
    console.log(e);  
    this.router.navigate([APP_ROUTES.cv]);  
    return throwError(() => new Error(e));  
  })  
);
```

Les opérateurs RxJs

Gestion des erreur

catchError

- Vous pouvez utiliser l'opérateur **EMPTY** de rxjs pour spécifier que vous **n'émettez rien** et qui émet ensuite une notification de **complétion** du flux.
- Vous pouvez aussi émettre ce que vous voulez afin de **continuer le flux**

```
catchError((e) => {  
    return EMPTY;  
}),
```

```
catchError((e) => {  
    return of("something went wrong");  
}),
```

Les opérateurs RxJs

Gestion des erreur

catchError / ignoreElements

- Si vous voulez émettre les erreurs en ignorant les éléments autres, vous pouvez utiliser **ignoreElements** et émettre l'erreur.
- L'opérateur **ignoreElements** supprime tous les éléments émis par la source observable, mais permet à sa notification de fin (**error** ou **complete**) de passer.
- Si vous **ne vous souciez pas des éléments émis** par un observable, mais que vous **souhaitez être averti lorsqu'il se termine** ou lorsqu'il **se termine par une erreur**, vous pouvez appliquer l'opérateur **ignoreElements** à l'observable

```
errors$ = this.service.getData()  
  .pipe(  
    ignoreElements(),  
    catchError(error => of(error))  
  )
```


- 157

Les opérateurs RxJs

Gestion des erreur

l'opérateur retry

- RxJS dispose d'un **opérateur de nouvelle tentative** très pratique, qui est utile dans les situations où une demande de données échoue.
- Par exemple, vous essayez de charger des données à partir d'une API et cela génère des erreurs.
- L'opérateur **retry**, va réessayer avant de déclencher une erreur.

```
return this.http.get<Cv[]>(API.cv).pipe(  
  retry(5)  
);
```

Les opérateurs RxJs

Gestion des erreurs

l'opérateur retry

- L'opérateur **retry** peut aussi accepter un objet avec les paramètres suivants:
 - **count** : le nombre maximal de tentative
 - **delay** : number | ((error: any, retryCount: number) => ObservableInput<any>)

Le nombre de secondes avant de relancer ou un notifier qui peut être un Subject et qui à chaque émission déclenchera un nouvel essai.

```
return this.http.get<Cv[]>(API.cv).pipe(  
  retry({  
    delay: 3000,  
    count: 3  
  })  
);
```

Quelques opérateurs utiles de l'Observable

<https://angular.io/guide/rx-library>

<http://reactivex.io/rxjs/manual/overview.html#operators>

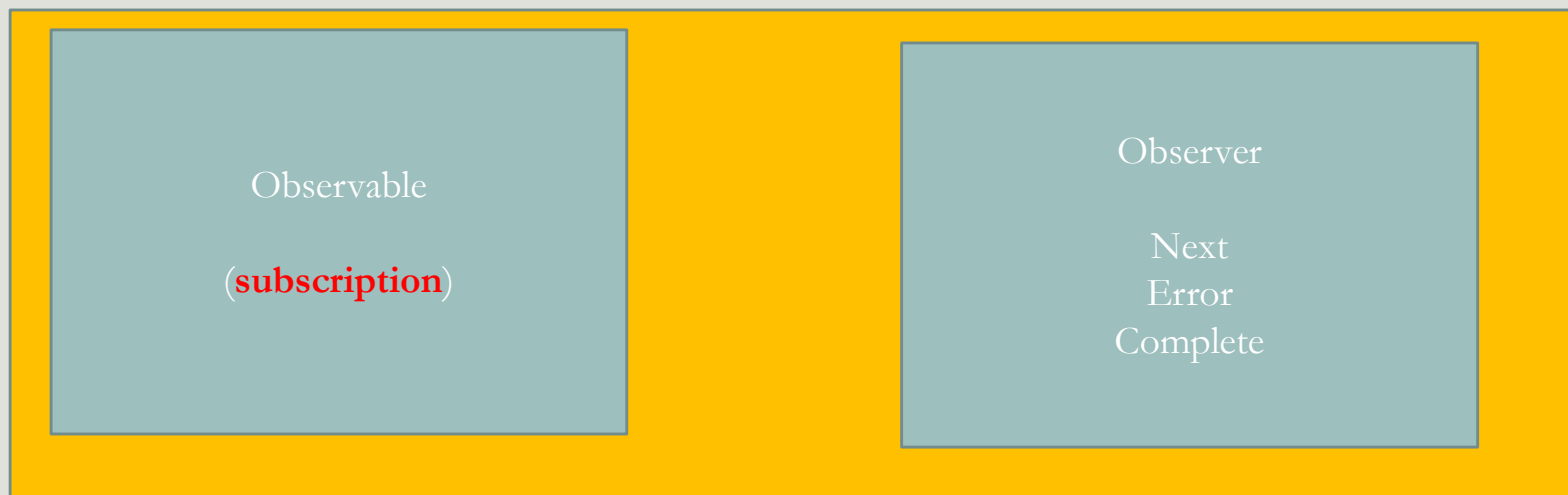
<http://rxmarbles.com/>

Les subjects

- Afin de créer des **flux chauds**, RxJs nous fournit une structure de données appelées **Subject**.
- Pensez au Subject comme à **un flux que vous créer au fur et à mesure** que vous recevez les données.
- C'est une **création en temps réel**, vous **n'avez aucune information au préalable** sur le contenu du flux.
- C'est comme la **transmission d'un match à la télé en direct**. Votre chaine ne sait pas quelles images elle va recevoir, elle ne les a pas en mémoire sous forme d'une vidéo, à chaque fois qu'elle recoit une image de la source, elle vous la transmet.

Les subjects

- Un **subject** est un type particulier d'observable. En effet Un **subject** est en même temps un **observable** et un **observer**, il possède donc les méthodes next, error et complete.
- Pour **broadcaster** une nouvelle valeur, il suffit d'appeler la méthode **next**, et elle sera diffusé aux Observateurs enregistrés pour écouter le Subject.



Les subjects

- Afin de créer un flux chaud commencer par **instancier un Subject**. Ceci vous permet d'avoir un flux vide auquel on peut s'inscrire.

```
nbUser$ = new Subject<Number>();
```

- Pour ajouter une valeur dans le flux, il suffit d'appeler la méthode next.

```
this.nbUser$.next(1);
```

1

```
this.nbUser$.next(2);
```

1

2

Les subjects

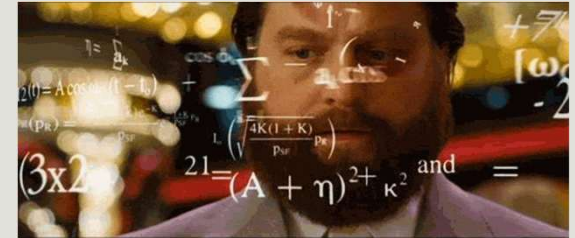
- Si vous êtes intéressé par ce flux, c'est un flux comme un autre, il vous suffit donc de vous inscrire.

```
nbUser$.subscribe({  
  next:(nb) => {},  
  error:(e) => {},  
  complete:() => {},  
})
```

Les subjects



Exercice



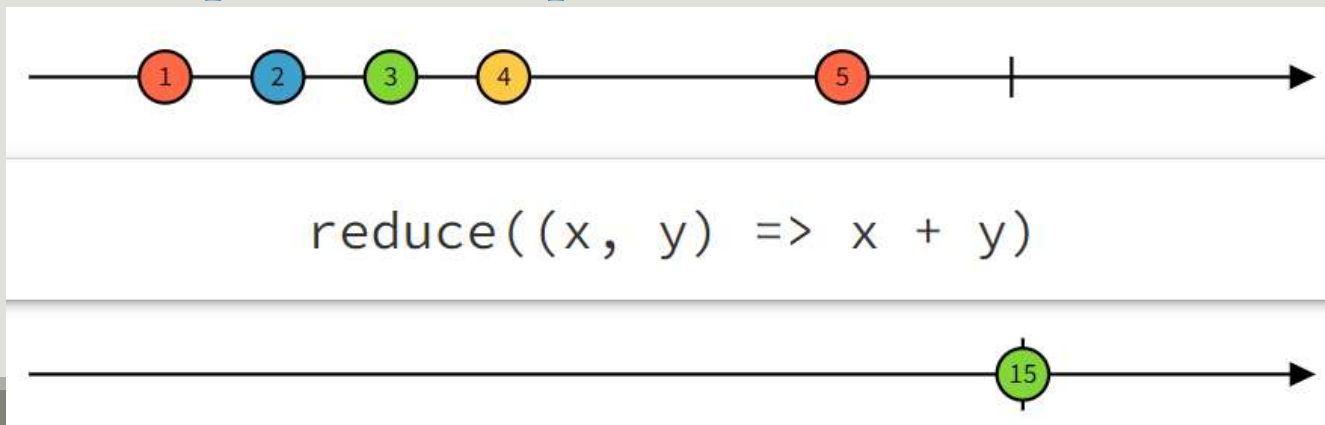
- Modifier l'affichage des détails d'une personne au click. Enlever tous les outputs et remplacer les par l'utilisation d'un subject.

Quelques opérateurs utiles de l'Observable

opérateur pipable

reduce

- **reduce** est un opérateur qui permet de réduire un flux en une valeur égale à l'opération passé en paramètre.
- Cette fonction prend en paramètre la valeur accumulé et la valeur actuelle.
- **Il émet uniquement lorsque le flux se termine.**

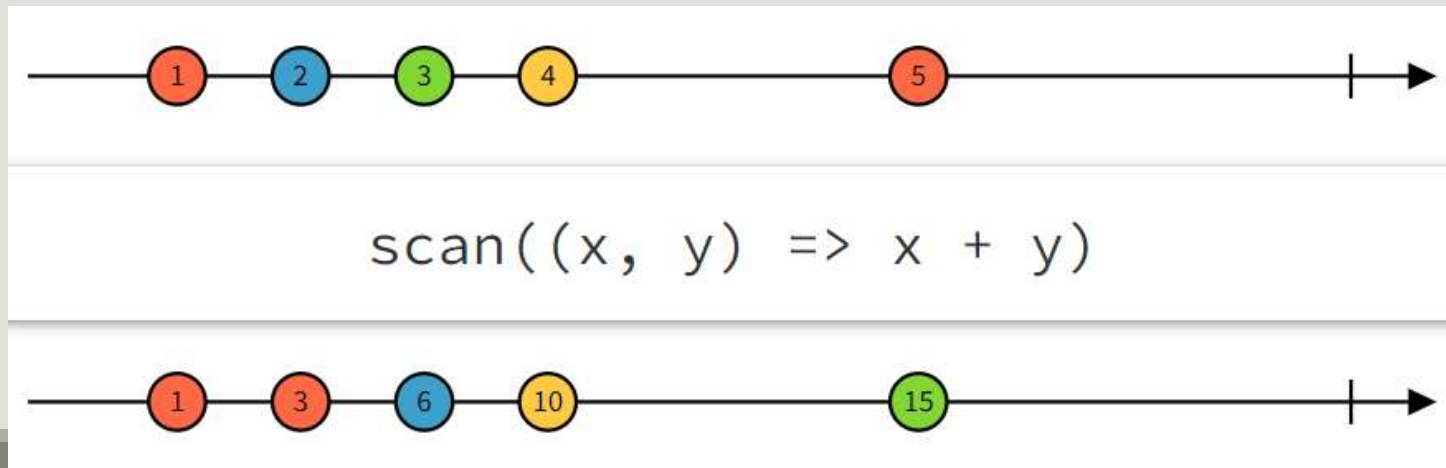


Quelques opérateurs utiles de l'Observable

opérateur pipable

scan

- scan est un opérateur qui ressemble à **reduce** sauf qu'il émet une valeur à chaque réception d'une nouvelle valeur reçu du flux.
- Cette fonction prend en paramètre la valeur accumulé et la valeur actuelle.



Les behaviourSubject

- Les **BehaviourSubject** sont une variante du **Subject**.
- Le **BehaviourSubject** a la particularité de **stocker la valeur "actuelle"**. Cela signifie que vous pouvez **toujours obtenir directement la dernière valeur émise** à partir du **BehaviourSubject**.
- Il existe deux façons d'obtenir cette dernière valeur émise. Vous pouvez soit **obtenir la valeur en accédant à la propriété value** sur le **BehaviourSubject**,
- Soit **vous y abonner**. Si vous vous y abonnez, le **BehaviourSubject** **émettra directement** la valeur actuelle à l'abonné. **Même si l'abonné s'abonne beaucoup plus tard que la valeur a été stockée.**
- Vous pouvez **créer un BehaviourSubject avec une valeur initiale** qui sera donc émise directement pour la première inscription.

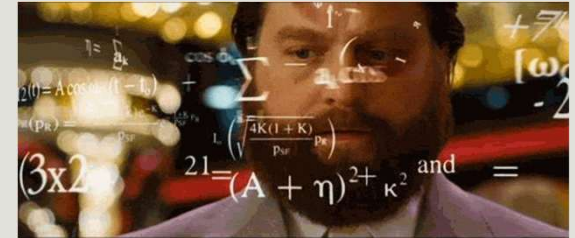
Les ReplaySubject

- Le **ReplaySubject** est comparable au BehaviorSubject dans la mesure où il peut envoyer les "anciennes" valeurs aux nouveaux abonnés.
- Il a cependant la particularité supplémentaire de pouvoir **enregistrer une partie de l'exécution de l'observable** et donc de **stocker plusieurs anciennes valeurs** et de les "rejouer" aux nouveaux abonnés.
- Lors de la création du ReplaySubject, vous pouvez spécifier **la quantité de valeurs que vous souhaitez stocker** et **pendant combien de temps** vous souhaitez les stocker.

Les AsyncSubject

- Alors que BehaviorSubject et ReplaySubject stockent tous deux des valeurs, **AsyncSubject** fonctionne un peu différemment. L'AsyncSubject est une variante de l'objet où **seule la dernière valeur** de l'exécution Observable est **envoyée à ses abonnés**, et **uniquement lorsque l'exécution est terminée**.

Exercice



- Nous voulons reproduire l'interface suivante qui permet de récupérer au fur et à mesure les produits en utilisant l'API suivante :

<https://dummyjson.com/products>
(<https://dummyjson.com/docs/products>)

- Au départ afficher un nombre de produit (12)
- En cliquant sur le bouton plus de produits, récupérer au fur et à mesure d'autre produits.
- S'il ne reste aucun produit, arrêter d'appeler l'API
- Faire en sorte que le code soit déclarative.

RxJS Home Cv Add Cv Todo Products Logout

TUESDAY, SEPTEMBER 26, 2023 AT 3:32:01 PM GMT+01:00

	Nidhal Jelassi
	Aymen Sellaouti
	Skander Sellaouti

Footer

Opérateur de Multicasting

share

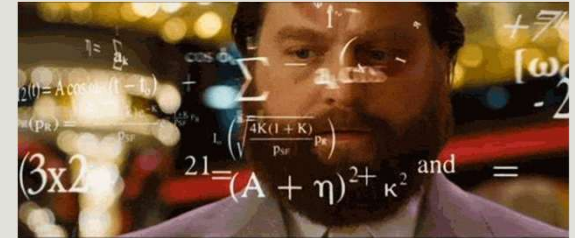
- L'opérateur **share** va **multicaster** les valeurs émises par une source Observable pour les abonnés.
- Les multicaster signifie que **les données sont envoyées à plusieurs destinations**.
- En tant que tel, le partage vous permet **d'éviter plusieurs exécutions de la source Observable** lorsqu'il existe plusieurs abonnements.
- **share** est particulièrement utile si vous avez besoin d'**empêcher des appels d'API répétés** ou des **opérations coûteuses exécutées par Observables**.

Opérateur de Multicasting

share

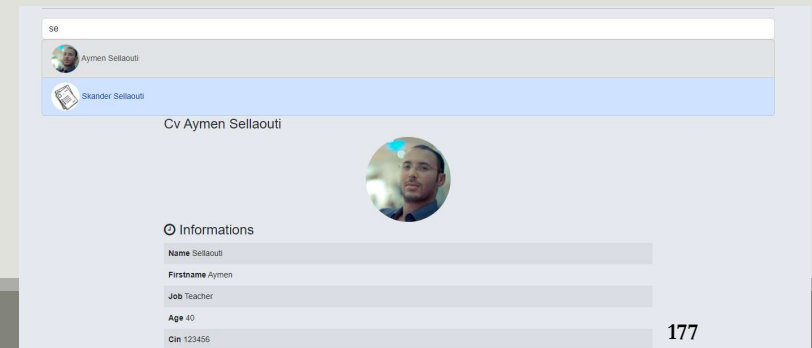
```
const sharedSource$ = interval(1000).pipe(  
  tap((ind) => console.log('Processing: ', ind)),  
  map(() => Math.round(Math.random() * 100)),  
  take(2),  
  share()  
);  
sharedSource$.subscribe((x) => console.log('subscription 1: ', x));  
sharedSource$.subscribe((x) => console.log('subscription 2: ', x));  
setTimeout(  
  () => sharedSource$.subscribe((x) => console.log('subscription 3: ', x)),  
  4500  
);
```


Exercice



- Sachant que pour sélectionner une personne dont le nom contient une chaîne donnée, loopback utilise la syntaxe suivante :

```
{"where": {"name": {"like": "%${name}%"}}}
```
- Ceci doit être fourni dans les paramètres de votre requête avec la clé `filter`. Tester le sur votre swagger.
- Créer un composant autocomplete, Ajouter y un champ input. En saisissant des caractères, la liste des choix doit automatiquement changer et n'afficher que les cvs qui contiennent la chaîne saisie.
- En sélectionnant un des choix, afficher le Cv correspondant.
- Pensez à optimisez votre code en minimisant les appels http afin de ne pas spammer votre serveur
- Faites en sorte d'avoir le code le plus réactive possible.



Se désinscrire

- La méthode **subscribe** permet de s'inscrire à un **observable**.
- **Problème** : Cette souscription reste valide même après la disparition de la variable ce qui sature la mémoire pour rien.

Se désinscrire complete

- Lorsqu'un observable se termine (méthode complete), tous les abonnements sont automatiquement désabonnés.
- Si vous savez qu'un observable se terminera, vous n'avez pas à vous soucier de nettoyer les abonnements.
- Cela est vrai pour tout observable créé à partir de plusieurs des sources observables intégrées dans Angular telles que les méthodes du **HttpClient** ou le service **ActivatedRoute**.
- Il n'y a pas de convention standard qui indique ce que les observables peuvent terminer, ou quand, il appartient donc au développeur de faire les recherches nécessaires.

Se désinscrire async pipe

- La première et la plus courante solution pour **gérer les abonnements** est **l'async pipe**.
- Plutôt que de vous abonner à des observables dans vos composants et de définir des propriétés sur votre classe pour les données de l'observable, préférez l'utilisation de **l'async pipe**.
- L'async **pipe nettoie ses propres abonnements** quand et selon les besoins, vous déchargeant de cette responsabilité. Maintenant, cela devrait être **votre premier choix et fonctionnera pour la plupart des scénarios**.

Se désinscrire async pipe

```
export class TestUnsubscribeComponent {  
  todos$: Observable<Todo[]>;  
  firstTodo$: Observable<Todo>;  
  constructor(private todoService: TodoService) {  
    this.todos$ = this.todoService.getTodos();  
    this.firstTodo$ = this.todoService.getTodo(1);  
  }  
}
```

```
<div *ngIf="firstTodo$ | async as firstTodo">  
  The first todo is :  
  {{ firstTodo.id }} | {{firstTodo.title}}  
</div>  
<ol>  
  <li *ngFor="let todo of todos$ | async">  
    <pre>{{todo | json}}</pre>  
  </li>  
</ol>
```

Se désinscrire

Et si on peut pas utiliser l'async pipe ?

- Que faire lorsque vous ne pouvez pas utiliser l'async pipe ? Il y a plusieurs options.
- La première et la plus simple consiste à utiliser un **Subscription** pour **collecter tous les abonnements** que vous créez, vous permettant de les nettoyer plus tard au moment opportun.
- L'objet **Subscription** dans RxJs n'est pas seulement représentatif d'un seul **subscription** à un flux. Il est également représentatif d'un **agrégat de subscription** qui contient d'autres abonnements. Cela lui permet d'être utilisé comme point de contrôle unique pour n'importe quel nombre d'abonnements.
- Pour se faire, utilisez sa méthode **add** qui prend en paramètre une **subscription**.

Se désinscrire

Et si on peut pas utiliser l'async pipe ?

```
export class TestUnsubscribeComponent implements OnDestroy{
  todos$: Observable<Todo[]>;
  firstTodo$: Observable<Todo>;
  subscription: Subscription;
  nb = 0;
  constructor(private todoService: TodoService, private testService: TestService ) {
    this.todos$ = this.todoService.getTodos();
    this.firstTodo$ = this.todoService.getTodo(1);
    this.subscription = this.testService.click$.subscribe(() => this.nb ++);
  }

  ngOnDestroy(): void {
    this.subscription.unsubscribe();
  }
}
```

Se désinscrire

Et si on peut pas utiliser l'async pipe ?

```
export class TestUnsubscribeComponent implements OnDestroy{
  todos$: Observable<Todo[]>;
  firstTodo$: Observable<Todo>;
  subscription: Subscription = new Subscription();
  nbClick = 0;
  nbUpdates = 0;
  constructor(private todoService: TodoService, private testService: TestService ) {
    this.todos$ = this.todoService.getTodos();
    this.firstTodo$ = this.todoService.getTodo(1);
    this.subscription.add(this.testService.click$.subscribe(() => this.nbClick ++));
    this.subscription.add(this.testService.update$.subscribe(() => this.nbUpdates ++));
  }
  ngOnDestroy(): void {
    this.subscription.unsubscribe();
  }
}
```


Se désinscrire Utiliser un signal

- Une autre façon de se désabonner, et peut-être une manière plus élégante, consiste à utiliser des **signaux**.
- Les signaux sont un moyen d'utiliser d'autres flux pour contrôler les flux auxquels vous êtes peut-être abonné.
- Les signaux sont obtenus avec l'opérateur **takeUntil** dans le pipe et tout autre observable. L'autre observable peut être un Subject ou un autre observable ; tout ce qui émet un next.

Se désinscrire

Et si on peut pas utiliser l'async pipe ?

```
export class TestUnsubscribeComponent implements OnDestroy{
  todos$: Observable<Todo[]>;
  signal$ = new Subject();
  nbClick = 0;
  constructor(private todoService: TodoService, private testService: TestService ) {
    this.todos$ = this.todoService.getTodos();
    this.testService.click$
      .pipe(takeUntil(this.signal$))
      .subscribe(() => this.nbClick ++);
  }
  ngOnDestroy(): void {
    this.signal$.next('i am emitting stop emitting on your side :S');
    this.signal$.complete();
  }
}
```

Angular Reactive Form

AYMEN SELLAOUTI

Reactive form

- Les Reactive Form sont une deuxième méthode de gérer vos formulaires avec Angular.
- Au contraire des Template Driven Form, ses formulaires sont **générés programmatiquement** dans la partie TS.
- Ceci permet **d'alléger le template** des validateurs.
- D'autre part ca permet d'avoir un **formulaire testable**
- Finalement ceci permet de plus facilement **générer des Validateurs personnalisés.**
- Un formulaire réactif peut être crée avec **une instanciation de l'objet FormGroup** ou via **le service FormBuilder.**

Reactive form

Les Validateurs

- Un **validateur** est une **fonction** qui retourne **false** si un champ est **valide** selon une certaine condition **sinon elle retourne une information sur l'erreur**.
- Afin de valider votre formulaire, passer en deuxième paramètre de **FormControl** une **référence à une méthode de la classe Validators** ou un **tableau de ces méthodes**.

```
FormControl( formState: null, validatorOrOpts: [Validators.]),
  • email(control: AbstractControl)
  • required(control: AbstractControl)
  • compose(validators: null)
  • nullValidator(control: AbstractControl)
  • max(max: number)
  • composeAsync(validators: (AsyncValidatorFn | null)[])
  • maxLength(maxLength: number)
  • min(min: number)
  • minLength(minLength: number)
  • pattern(pattern: string | RegExp)
```

Reactive form

Les Validateurs

- Afin de valider votre formulaire, passer en deuxième paramètre de **FormControl** une **référence à une méthode de la classe Validators** ou un **tableau de ces méthodes**.

```
FormControl( formState: null, validatorOrOpts: [Validators.]),
```

- email(control: AbstractControl)
- required(control: AbstractControl)
- compose(validators: null)
- nullValidator(control: AbstractControl)
- max(max: number)
- composeAsync(validators: (AsyncValidatorFn | null)[])
- maxLength(maxLength: number)
- min(min: number)
- minLength(minLength: number)
- pattern(pattern: string | RegExp)

Reactive form

Les Validateurs

- **Validateurs synchrones** : Ce sont des fonctions qui contrôlent votre élément et retournent un **ensemble d'erreurs de validation ou null**. C'est ce qu'on passe en deuxième paramètre lors de l'instanciation d'un FormControl.
- **Validateurs asynchrones** : Ce sont des fonctions asynchrones qui contrôlent votre élément et retournent une **Promise ou un Observable** qui émettent un **ensemble d'erreurs de validation ou null**. C'est ce qu'on passe en troisième paramètre lors de l'instanciation d'un FormControl.
- Pour des raisons de **performance**, Angular commence par les validateurs synchrones, s'ils passent il déclenche les validateurs asynchrones.

Reactive form

Les Validateurs offerts par Angular

- Vous pouvez utiliser des validateurs offerts par angular ou créer vos propres validateurs.
- Les validateurs de base sont les mêmes que ceux de l'approche basée Template.

```
new FormGroup({  
  email: new FormControl(null, [Validators.required, Validators.email]),  
  username: new FormControl(null, [Validators.minLength(3)]),  
  age: new FormControl(null, [  
    Validators.required, Validators.pattern('[0-1]?\d{1,2}'))  
  ]),  
});
```

Méthode 1

Reactive form

Les Validateurs offerts par Angular

```
class Validators {  
  static min(min: number): ValidatorFn  
  static max(max: number): ValidatorFn  
  static required(control: AbstractControl<any, any>): ValidationErrors | null  
  static requiredTrue(control: AbstractControl<any, any>): ValidationErrors | null  
  static email(control: AbstractControl<any, any>): ValidationErrors | null  
  static minLength(minLength: number): ValidatorFn  
  static maxLength(maxLength: number): ValidatorFn  
  static pattern(pattern: string | RegExp): ValidatorFn  
  static nullValidator(control: AbstractControl<any, any>): ValidationErrors | null  
  static compose(validators: ValidatorFn[]): ValidatorFn | null  
  static composeAsync(validators: AsyncValidatorFn[]): AsyncValidatorFn | null  
}
```

Reactive form

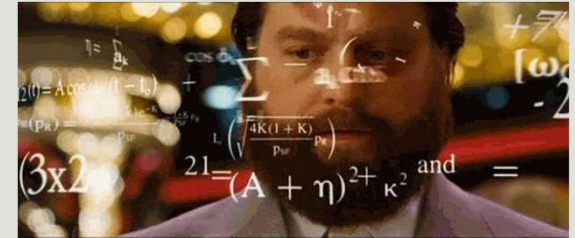
Les Validateurs offerts par Angular

- Vous pouvez utiliser des validateurs offerts par angular ou créer vos propres validateurs.
- Les validateurs de base sont les mêmes que ceux de l'approche basée Template.

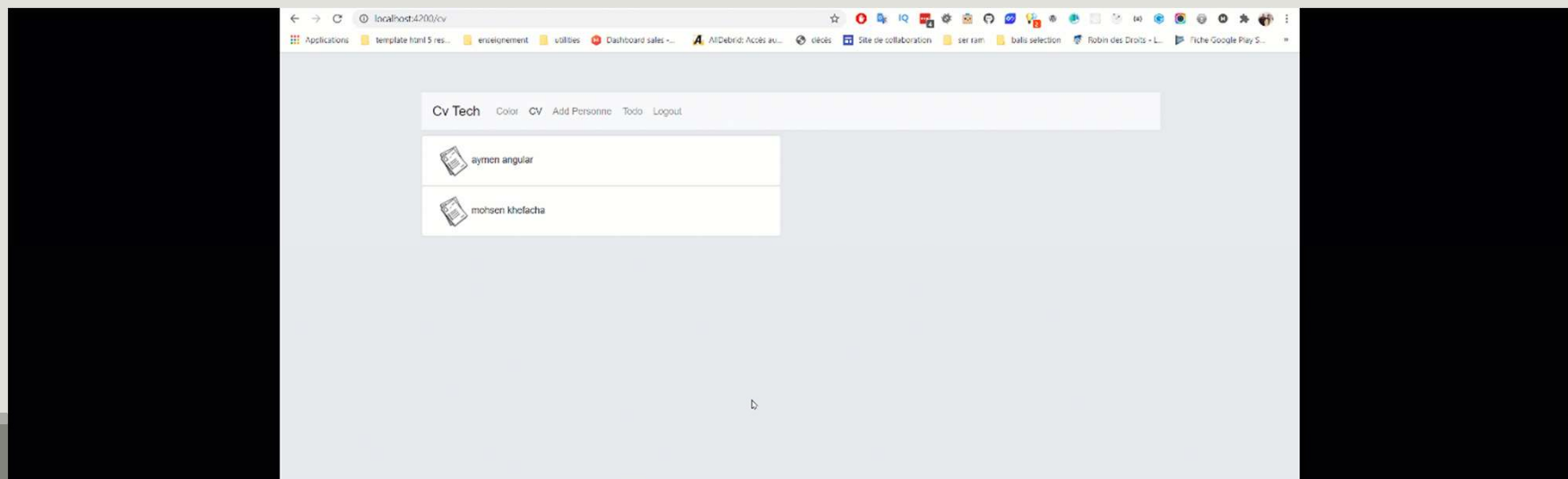
```
formGroup: FormGroup = new FormGroup({  
  name: new FormControl(null, {  
    validators: [Validators.required, Validators.minLength(3)],  
    asyncValidators: [],  
    updateOn: "change"  
  }),  
  age: new FormControl(null),  
});
```

Méthode 2

Exercice



- Ajouter dans votre cvTech un composant contenant un formulaire. Ce formulaire devra vous permettre d'ajouter un utilisateur.
- Ajouter les validateurs nécessaires.
- Après l'ajout forwarder le user vers la liste des cvs.



Reactive form

Manipulez les valeurs de votre form

- Afin de **mettre à jour la valeur** d'un **FormControl** ou d'un **FormGroup**, vous pouvez utiliser la méthode **setValue()** qui met à jour la valeur du contrôle de formulaire et valide la structure de la valeur fournie par rapport à la structure du contrôle.
- Vous pouvez utiliser la méthode **patchValue** si vous modifiez uniquement une partie.

```
this.form.setValue({ name: "Sellaouti", firstname: "Aymen" });
```

```
this.form.patchValue({firstname: "Aymen" });
```

Reactive form

Manipulez les valeurs de votre form

- En deuxième paramètre de ces deux fonctions, vous pouvez passer un objet d'options avec deux propriétés:
 - **onlySelf**: Angular vérifie l'état de validation du formulaire, chaque fois qu'il y a un changement de valeur. La **validation commence à partir du contrôle** dont la valeur a été modifiée et **se propage au FormGroup** de niveau supérieur. Il s'agit du comportement par défaut. Si vous ne souhaitez pas **qu'Angular vérifie la validité de l'ensemble du formulaire, chaque fois** que vous modifiez la valeur à l'aide de `setValue` ou `patchValue`, définissez **onlySelf à true**.

Reactive form

Manipulez les valeurs de votre form

- En deuxième paramètre de ces deux fonctions, vous pouvez passer un objet d'options avec deux propriétés:
 - **emitEvent**: Les forms Angular **émettent deux événements**. L'un est **ValueChanges** et l'autre est **StatusChanges**. L'événement ValueChanges est émis chaque fois que la valeur du formulaire est modifiée. L'événement StatusChanges est émis chaque fois qu'angular calcule l'état de validation du formulaire. C'est le comportement par défaut Nous pouvons **empêcher que cela se produise**, en **définissant l'emitEvent à false**

Reactive form

Manipulez les valeurs de votre form

```
this.form.patchValue(  
  { firstname: "Aymen" },  
  {  
    emitEvent: false,  
    onlySelf: true,  
  }  
);
```

Reactive form

Suivre les modifications de vos FormControl et de vos FormGroup

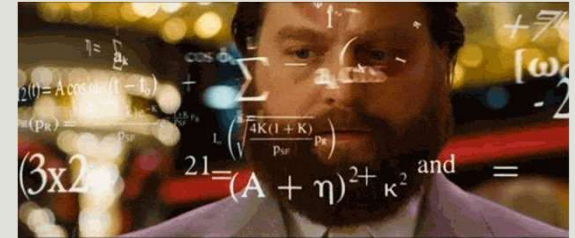
- **FormControl et FormGroup**, vous fournissent deux Observables permettant le suivi des changements de valeur et de status.
- **ValueChanges** est un événement déclenché par les formulaires Angular chaque fois que la valeur de FormControl, FormGroup ou FormArray change. L'observable obtient la dernière valeur du contrôle. Il nous permet de suivre les modifications apportées à la valeur en temps réel et d'y répondre. Par exemple, nous pouvons l'utiliser pour valider la valeur, calculer les champs calculés, ...

Reactive form

Suivre les modifications de vos FormControl et de vos FormGroup

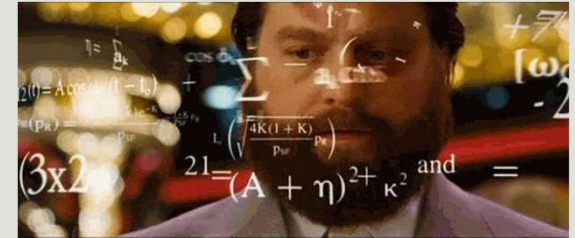
- **statusChanges** est un événement déclenché par les formulaires Angular **chaque fois que Angular calcule le statut de validation** de FormControl, FormGroup ou FormArray. Il renvoie un observable afin que vous puissiez vous y abonner. L'observable obtient le dernier état du contrôle.

Exercice



- Afin de protéger les informations personnelles des mineurs, faites en sortes que lorsque l'age de la personne possédant le Cv est inférieur à 18 ans, il ne puisse pas renseigner le path de l'image.

Exercice



- Etant donné que notre formulaire est assez volumineux et pour permettre une meilleure expérience utilisateur, nous voulons faire en sorte que si l'utilisateur saisisse un formulaire valide et qu'il sorte de la page sans l'envoyer, il puisse le retrouver rempli lorsqu'il retourne à la page d'ajout d'un cv.

Reactive form

Customiser vos validateurs

- Un custom validator est une fonction de type **ValidatorFn** qui prend en paramètre un control de type **AbstractControl** (qui contient une propriété **value** représentant la **valeur du champ à valider**) et **retourne null si la valeur est valide** ou un objet de type **ValidationErrors** s'il y a des **erreurs de validation**.

```
export declare interface ValidatorFn {  
  (control: AbstractControl): ValidationErrors | null;  
}
```

```
export declare type ValidationErrors = {  
  [key: string]: any;  
};
```

Reactive form

Customiser vos validateurs

- Le Validator, renvoyée par la fonction de création de validateur, doit suivre ces règles :
- **Un seul argument** d'entrée est attendu, qui est de type **AbstractControl**. La fonction validateur peut obtenir la **valeur à valider** via la propriété **control.value**
- La fonction de validation **doit renvoyer null** si **aucune erreur** n'a été trouvée dans la valeur du champ, ce qui signifie que la **valeur est valide**
- Si des **erreurs de validation sont trouvées**, la fonction doit renvoyer un objet de type **ValidationErrors**.

Reactive form

Customiser vos validateurs

- L'objet **ValidationErrors** peut avoir comme propriétés **les multiples erreurs trouvées** (généralement une seule) et comme valeurs les détails de chaque erreur.
- Si nous voulons simplement indiquer qu'une erreur s'est produite, **sans fournir plus de détails**, nous pouvons simplement **renvoyer true** comme valeur d'une propriété error dans l'objet **ValidationErrors**.

```
return !passwordValid ? {passwordStrength: true} : null;
```

Reactive form

Customiser vos validateurs

```
export function createPasswordStrengthValidator(): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const value = control.value;
    if (!value) {
      return null;
    }
    const hasUpperCase = /[A-Z]+/.test(value);
    const hasLowerCase = /[a-z]+/.test(value);
    const hasNumeric = /[0-9]+/.test(value);
    const passwordValid = hasUpperCase && hasLowerCase && hasNumeric;
    return !passwordValid ? {passwordStrength: true} : null;
  };
}
```

Reactive form

Customiser vos validateurs

Validateurs Asynchrones

- Dans certains cas d'utilisation, la validation de votre champ ne se fait pas d'une façon **asynchrone**.
- Le cas le plus répandu est la **validation par votre backend**.
- Imaginez que vous avez un champ email et qu'il ne doit pas être redondant. La solution est d'avoir une API qui vérifie ça et qui vous retourne la réponse.
- Ce traitement étant Asynchrone, le Validateur synchrone ne fait plus affaire.
- Il faut donc créer un validateur **ASYNCHRONE**.

Reactive form

Customiser vos validateurs

Validateurs Asynchrones

- Le Validator, renvoyée par la fonction de création de validateur, doit suivre ces règles :
- **Un seul argument** d'entrée est attendu, qui est de type **AbstractControl**. La fonction validateur peut obtenir la **valeur à valider** via la propriété **control.value**
- La fonction de validation **doit renvoyer une Promise<null>, un Observable<null>** si **aucune erreur** n'a été trouvée dans la valeur du champ, ce qui signifie que la **valeur est valide**
- Si des **erreurs de validation sont trouvées**, la fonction doit renvoyer **une Promise ou un Observable** d'un objet de type **ValidationErrors**.

Reactive form

Customiser vos validateurs

Validateurs Asynchrones

```
export function userExistsValidator(authService: AuthService): AsyncValidatorFn {  
  return (control: AbstractControl) => {  
    return authService.findUserByEmail(control.value);  
  };  
}
```

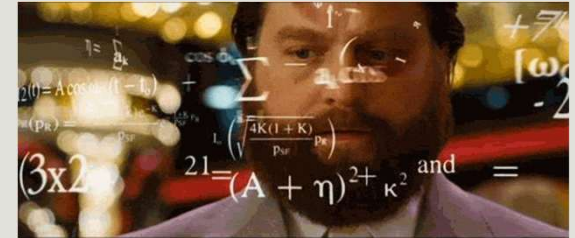
Reactive form

Customiser vos validateurs

Validateurs Asynchrones

```
findUserByEmail(value: any): Observable<ValidationErrors | null> {  
  return new Observable<ValidationErrors | null>(  
    (observer) => {  
      setTimeout(  
        () => {  
          const date = new Date();  
          if (date.getTime() % 2) {  
            observer.next(null);  
          } else {  
            observer.next({userExists: true});  
          }  
        }, 1500);  
      }  
    );  
  }  
}
```

Exercice



- Le champ Cin étant unique, créer un validateur asynchrone permettant de gérer cette contrainte et tester le.

Reactive form

Customiser vos validateurs

Valider un form

- Afin de valider un form, vous devez faire la même chose que pour le validateur d'un FormControl.

```
export function createDateRangeValidator(): ValidatorFn {
  return (form: AbstractControl): ValidationErrors | null => {
    const start: Date = form.get('startAt').value;
    const end: Date = form.get('endAt').value;
    if (start && end) {
      const isRangeValid = (end.getTime() - start.getTime() > 0);
      return isRangeValid ? null : {dateRange: true};
    }
    return null;
  };
}
```

Reactive form

Customiser vos validateurs

Valider un form

- Maintenant et pour l'appliquer à votre FormGroup ajouter à la création de votre FormGroup un second paramètre qui est un objet options et utiliser la propriété validators

```
this.form = new FormGroup(  
  {},  
  {  
    validators: VotreValideur  
  }  
);  
this.form = this.formBuilder.group(  
  {},  
  {  
    validators: VotreValideur  
  }  
);
```

- 215

Angular

Les modules

AYMEN SELLAOUTI

Qu'est ce qu'un module

- C'est une **classe** avec une décoration **NgModule**
- Un module est un **conteneur** qui **englobe** un **ensemble de fonctionnalités** liées.
- Les applications Angular sont **modulaire**.
- Une application Angular contient **au moins un Module** : AppModule.
- Un Module Angular peut contenir des composants, des providers de service...
- Une application simple est généralement composée d'un seul module. Par contre dès que votre application grandit penser à la séparer en Modules.
- Chaque module **vie séparée** des autres modules. Par défaut **il n'expose rien**, tout ce qui est à l'intérieur du module **reste uniquement dans le module tant qu'on ne l'exporte pas**.
- Lorsqu'on importe un module, on **importe réellement tout ce qu'il exporte**.

Rôle d'un module

- Organise votre application en des briques fonctionnelles
- Etend votre application avec des librairies externes
- Permet d'agréger et d'exporter des briques fonctionnels
- Faciliter le développement et la maintenance de votre application

Définition d'un module

- L'annotation (decorator) `@NgModule` identifie un module Angular.
- L'annotation prend en paramètre un objet spécifiant à Angular comment compiler et lancer l'application.
 - `imports` : tableau contenant les modules utilisés.
 - `declarations` : tableau contenant les classes de vue appartenant à ce module, i.e. composants, directives et pipes de l'application.
 - `exports` : **tableau des classes de vues à exporter.**
 - `providers` : déclaration des services
 - `bootstrap` : indique le composant exécuter au lancement de l'application et elle ne concerne que le module racine.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';

@NgModule({
  imports: [ BrowserModule ],
  declarations: [ AppComponent ],
  bootstrap: [ AppComponent ]
})
export class AppModule { }
```

declarations

- Dans la partie **declarations**, faite en sorte que chaque composant, directive ou pipe soit associé à **un et un seul module**. **Ne déclarer pas un même composant dans deux modules différents.**
- Ne déclarer que les composants, directives et les pipes dans cette partie.
- Tous les composants, directives et les pipes déclarés sont **privés par défaut**. Ils ne sont accessible que pour les composants, directives et les pipes déclarés dans le même module.
- Pour utiliser un de ces éléments à l'extérieur du module, il faudra penser à les exporter.

Routing

- Vous pouvez créer les routes de vos modules de la même manière que pour le routing de l'AppModule.
- Cependant, au lieu d'utiliser `RouterModule.forRoot` vous devez utiliser la méthode **forChild** du `RouterModule`.

Exemple pour le TodoModule

```
import { CommonModule } from "@angular/common";
import { NgModule } from "@angular/core";
import { FormsModule } from "@angular/forms";

import { TodoComponent } from "../todo.component";
import { TodoRoutingModule } from "../todo.routing";

@NgModule({
  declarations: [TodoComponent],
  imports: [CommonModule, FormsModule, TodoRoutingModule],
  // Si vous n'avez pas besoin de TodoComponent à l'extérieur,
  // ne l'exporter pas
  exports: [TodoComponent],
})
export class TodoModule {}
```

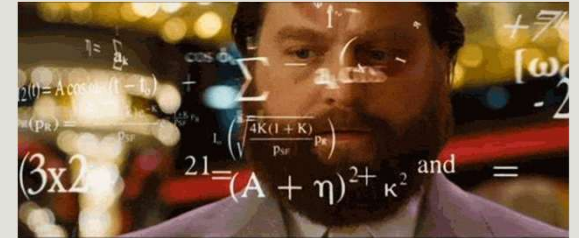
```
import { NgModule } from "@angular/core";
import { Route, RouterModule } from "@angular/router";
import { NF404Component } from "../nf404/nf404.component";
import { TodoComponent } from "../todo.component";

const routes: Route[] = [
  { path: "todo", component: TodoComponent },
  { path: "**", component: NF404Component },
];

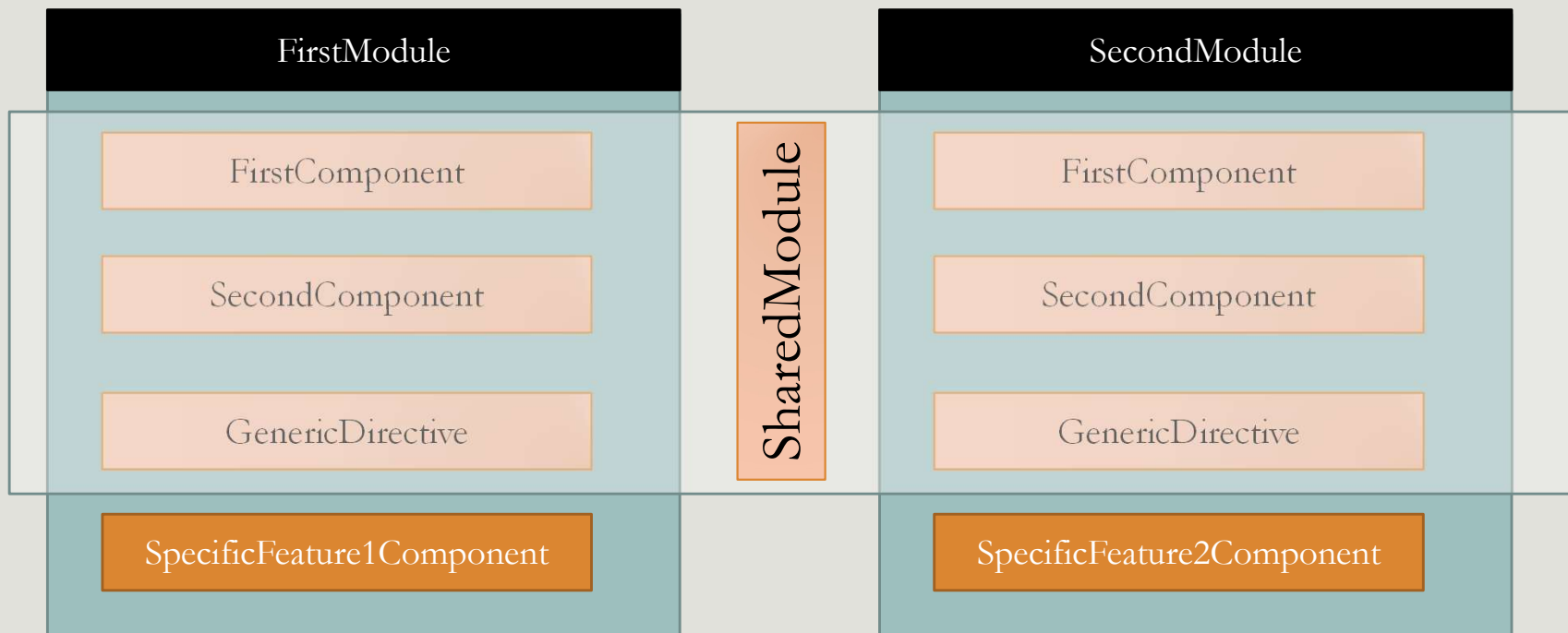
@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class TodoRoutingModule {}
```

Exercice

- Implémentez le CvModule.



Shared Module



Shared Module

```
@NgModule({
  declarations: [
    FirstComponent,
    SecondComponent,
    GenericDirective,
  ],
  imports: [
    CommonModule
  ]
  exports: [
    FirstComponent,
    SecondComponent,
    GenericDirective,
  ]
})
export class SharedModule {
}
```

Lazy Loading

- Par défaut, tous les modules que vous déclarez au niveau du AppModule sont chargés au lancement de l'application.
- Ceci pose un problème au niveau du Bundle généré de votre application.
- Une grande application aura une taille assez conséquente ce qui peut provoquer un problème au chargement de l'application et donc un problème d'expérience utilisateur.
- L'idée du lazyLoading et de **charge au départ le module principale** et puis de **ne charger un module que si on appelle l'une de ses routes**.
- Ceci va nous faire gagner en performance.

The screenshot shows a web browser at localhost:4200. The page has a header 'Cv Tech' and a file upload section with buttons 'Choisir un fichier', 'Aucun fichier choisi', and 'Upload'. Below this, the text 'front works!' is displayed. The Chrome DevTools Network tab is open, showing a list of requests. The 'personnes' request is highlighted in red, indicating a failure.

Name	Status	Type	Initiator	Size	Time	Waterfall
localhost	304	docu...	Other	210 B	298 ...	
runtime.js	200	script	(index)	211 B	13 ms	
polyfills.js	200	script	(index)	212 B	13 ms	
styles.js	200	script	(index)	212 B	180 ...	
vendor.js	200	script	(index)	213 B	302 ...	
main.js	200	script	(index)	212 B	26 ms	
personnes	(failed)	xhr	VM3716:1	0 B	2.02 s	

Lazy Loading

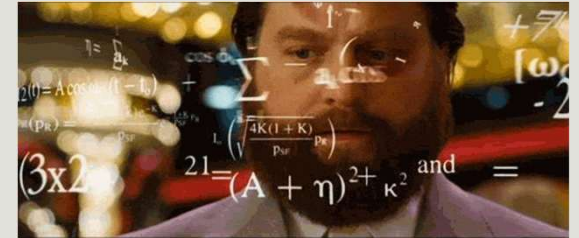
➤ Afin d'implémenter le *Lazy Loading*, on doit suivre les étapes suivantes :

1. Le **module** à charger doit **lui-même gérer sa partie routing**
2. Au niveau du routing principal (AppRoutingModule), créer une **route**, ajouter lui un path 'ca sera le **préfixe** de toutes les routes du module', et ajouter une nouvelle clé qui est **loadChildren**. Cette clé va informer angular et lui demander de ne charger le module associé que lorsque on appelle le path défini.
3. Ce **paramètre** prend ou une **chaîne de caractère** qui spécifie le module à charger ou une callback function.
4. Finalement **enlever les imports des modules lazy loaded** au niveau du AppModule

```
{  
  path: "cv",  
  loadChildren: "./cv/cv.module#CvModule",  
},
```

```
{  
  path: "cv",  
  loadChildren: () => import('./cv/cv.module').then(  
    m => m.CvModule  
  ),  
},
```

Exercice



- ## ➤ Testez le lazy loading avec le CvModule

Preloading Lazy Loading

- Le **problème** qu'on peut identifier avec le **lazy loading** est le fait qu'en passant d'un module à un autre on aura **toujours un chargement des nouveaux modules**.
- Si vos **modules** sont **très volumineux** ou que la connexion du client est mauvaise, il y aura **plusieurs latences**. Ceci va provoquer un problème avec l'utilisateur.
- La question qui se pose est : **Y a-t-il un moyen de personnaliser les stratégies de chargement** ???

Preloading Lazy Loading

- La méthode **forRoot** de votre RouterModule prend en **second paramètre un objet** vous permettant de **configurer** la **stratégie de chargement** avec la propriété **preloadingStrategy**.
- Cette propriété prend en paramètre, par défaut **NoPreloading**.
- La deuxième valeur qu'elle peut prendre est **PreloadAllModules**. Elle demande à Angular de **précharger** tous les **lazyLoaded Modules** une fois le **Module principal chargé**.

```
import { PreloadAllModules } from "@angular/router";
@NgModule({
  imports: [RouterModule.forRoot(routes, {
    preloadingStrategy: PreloadAllModules
  })],
  exports: [RouterModule],
})
```

vendor.js	200	script	(index)	213 B	270 ms	
main.js	200	script	(index)	212 B	25 ms	
personnes	(failed)	xhr	VM14822:1	0 B	2.01 s	
personnes	(failed)		Other	0 B	2.00 s	
common.js	200	script	bootstrap:149	210 B	9 ms	
cv-cv-module.js	200	script	bootstrap:149	211 B	9 ms	
todo-todo-module.js	200	script	bootstrap:149	211 B	8 ms	

Preloading Lazy Loading

Créer votre propre stratégie

- Avec **PreloadAllModules**, tous les modules sont **préchargés**, ce qui peut en fait créer un **goulot d'étranglement** si l'application a un **grand nombre de modules à charger**.
- Une meilleure stratégie serait de **charger sélectivement les modules requis au démarrage**. Par exemple, module d'authentification, module principal, module partagé, etc.
- Pour **précharger sélectivement** un module, nous devons utiliser une **stratégie de préchargement personnalisée**.
- Créez d'abord une **classe** qui implémente l'interface **PreloadingStrategy**. La classe doit implémenter la méthode **preload()**.
- C'est cette méthode qui détermine s'il **faut précharger le module ou non**.

Preloading Lazy Loading

Créer votre propre stratégie

- La signature de la fonction **preload** prend en paramètre un **objet Route** représentant la route ciblée et en deuxième paramètre une fonction **load** qui retourne un Observable.
- Si vous retournez la méthode load, le module sera **préchargé**.
- Si vous **ne voulez pas le précharger** retourner un **Observable de null**.

```
@Injectable({providedIn: 'root'})
export class CustomPreloadingStrategy implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<any>): Observable<any> {
    if (route.data["preload"]) {
      return load();
    }
    else {
      return of(null);
    }
  }
}
```


Angular Zone et Change Détection

AYMEN SELLAOUTI

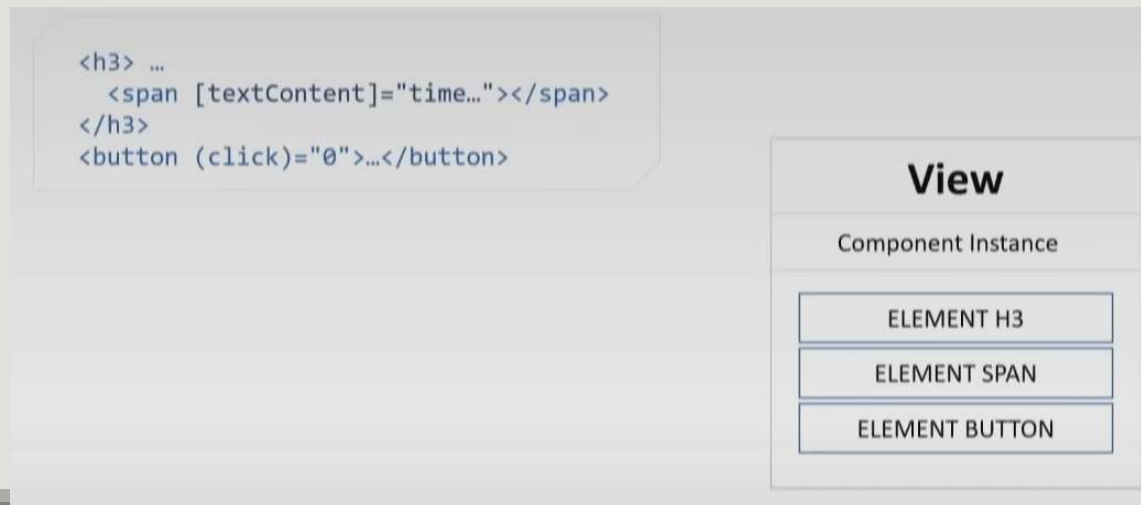
Change Detection

C'est quoi ?

- **Change Detection** est l'un des mécanismes les plus importants dans Angular.
- Il permet de **suivre les changements d'état** de votre application et **d'afficher les modifications** dans votre vue.
- Il **garantit que l'interface utilisateur suit toujours** d'une façon synchrone **l'état interne de votre application**.

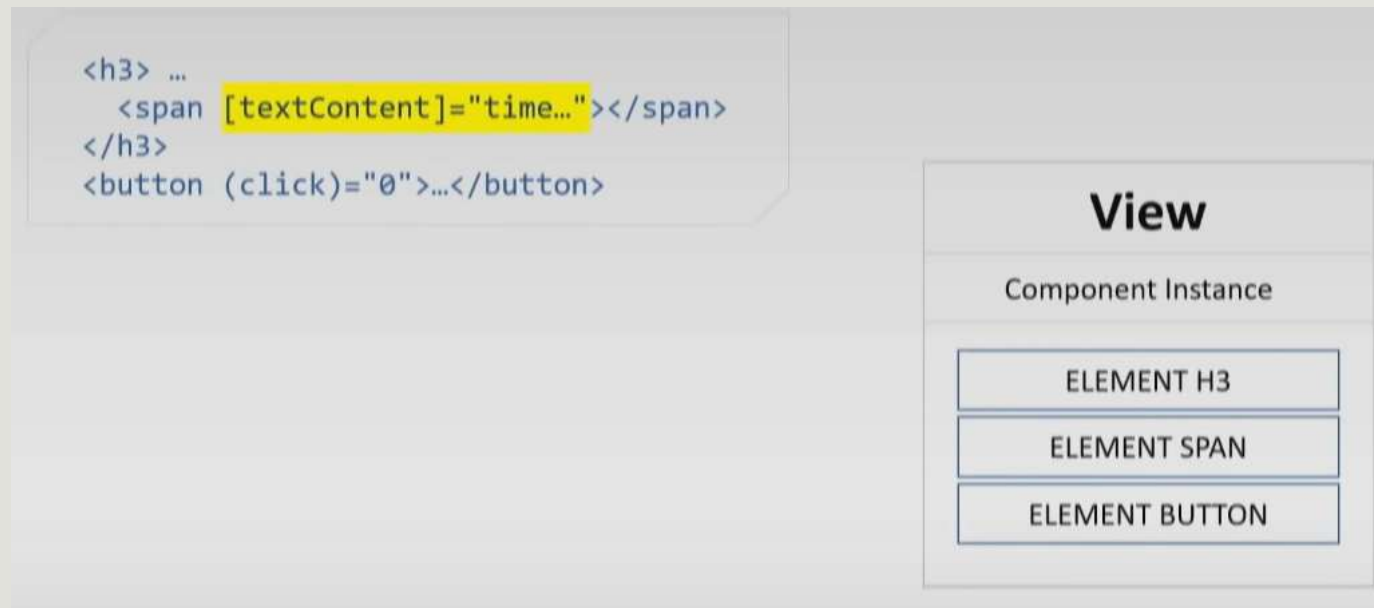
Change Detection

- **Chaque composant** dans Angular est représenté par une structure appelé **View**. Elle contient entre autre **l'instance** de la classe Composant appelée **componentInstance** ainsi que la **liste** des **éléments du DOM** représentant le Template.



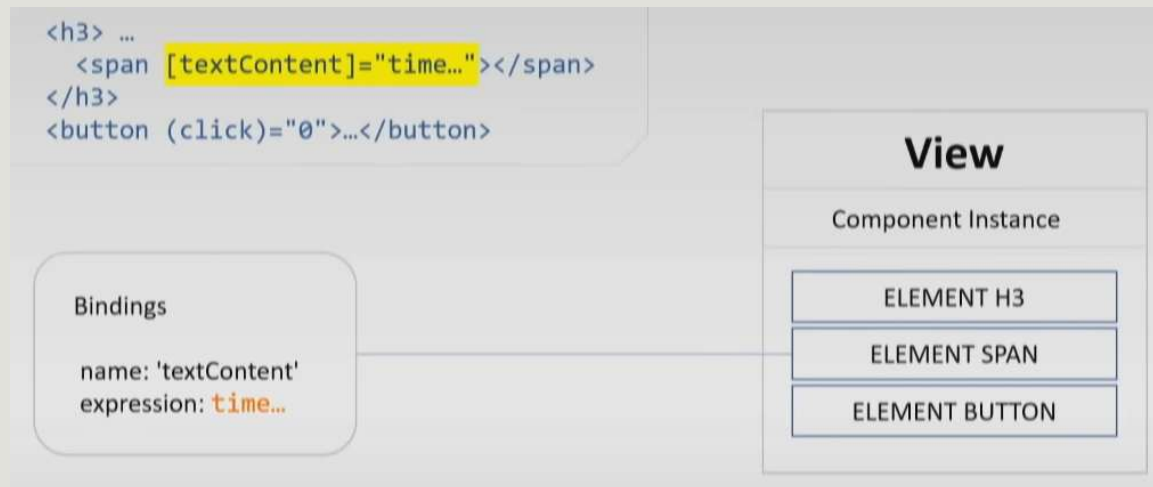
Change Detection

- Ensuite, quand le compilateur traite le composant, il **identifie les élément qui nécessite un changement** lors du changement d'état.



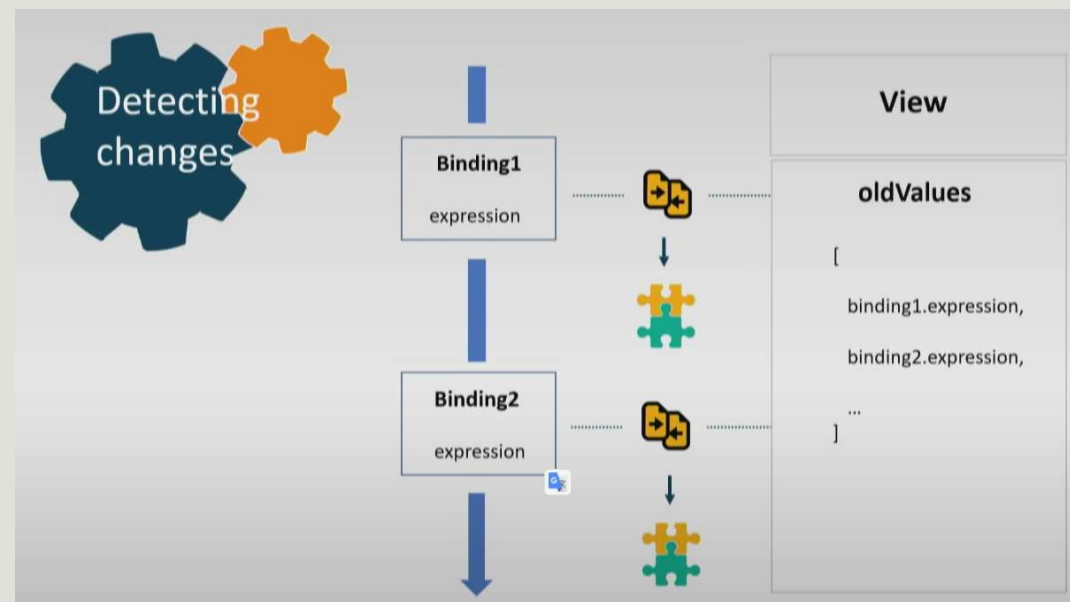
Change Detection

- Pour chacun de ces éléments, il crée des objets Bindings. C'est une structure de données qui informe sur deux choses :
 - Que voulons nous mettre à jour dans le Dom
 - Ou récupérer la nouvelle valeur

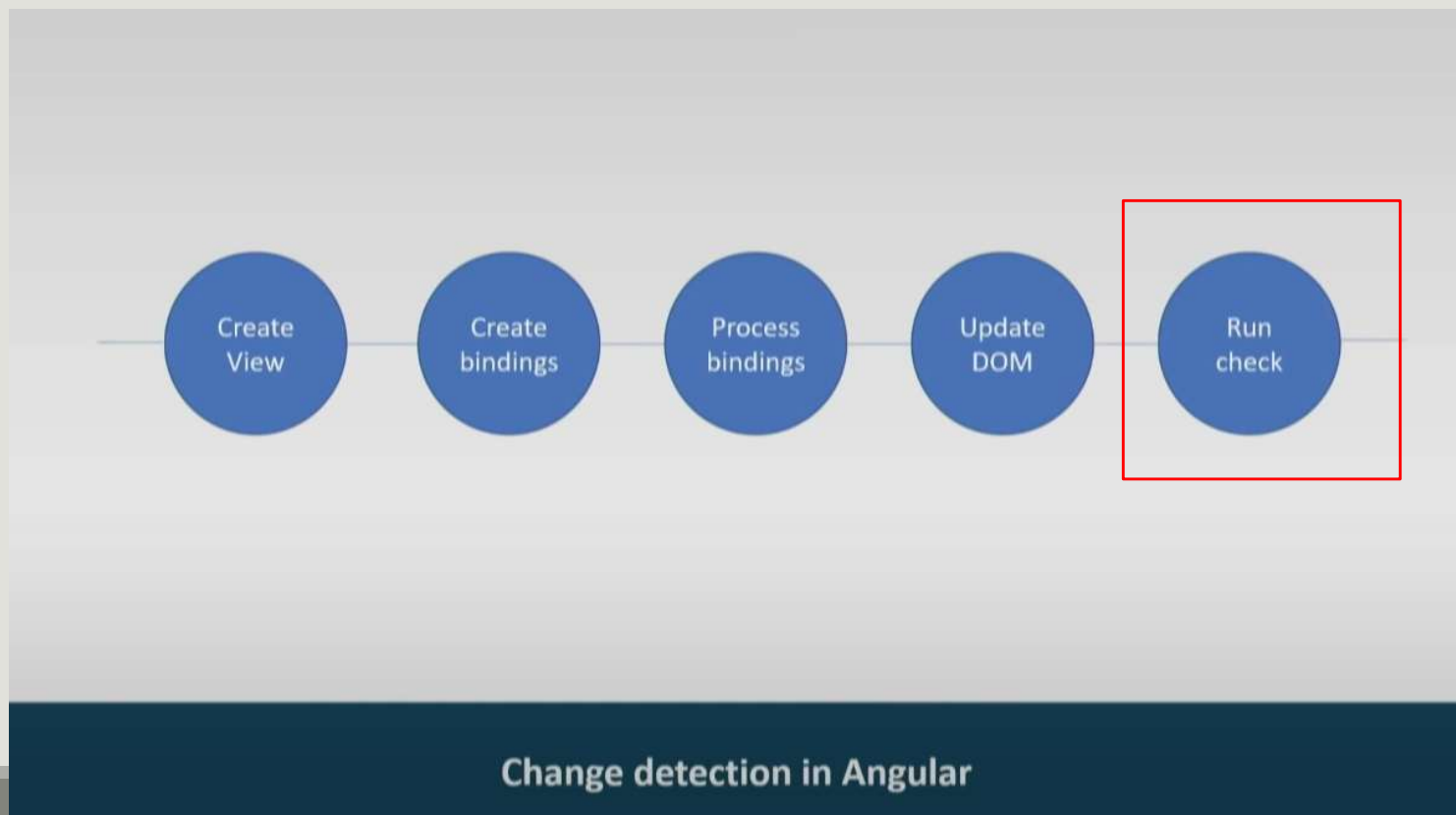


Change Detection

- Ensuite, dès qu'un **change Detectin** est déclenché, Angular va parcourir l'ensemble des Views (Component) et évaluer la nouvelle expression du Binding et la **comparer à la précédente**.
- Si la valeur est modifiée, elle met à jour le DOM.



Change Detection



Change Detection

Quand déclencher un Change Detection

- Un Change Detection est déclenché dans ces cas d'utilisation
 1. **Initialisation des composants.** Par exemple, lors du lancement d'une application angular, Angular charge le composant principal et déclenche **`ApplicationRef.tick()`** pour appeler la détection de changement et le rendu de la vue.
 2. Les **event listener** du DOM peuvent mettre à jour les données dans un composant Angular et déclencher le Change Detection.
 3. **Les requêtes HTTP.**

Change Detection

Quand déclencher un Change Detection

4. Les **MacroTasks**, tels que **setTimeout()** ou **setInterval()**. En effet vous pouvez mettre à jour les données dans la callback function d'une macroTask comme **setTimeout()**.
5. Les **MicroTasks**, comme **Promise.then()** dont les callback peuvent mettre à jour les données.
6. **D'autres opérations asynchrones** qui peuvent mettre à jour vos données telles que **WebSocket.onmessage()** et **Canvas.toBlob()**.

Change Detection

Quand déclencher un Change Detection

- La liste précédente contient les scénarios **les plus courants** dans lesquels **l'application peut modifier les données**.
- Angular **exécute le Change Detection** à chaque fois qu'il **détecte la possibilité d'un changement de données**.
- Le résultat de la détection des changements est que le **DOM est mis à jour avec de nouvelles données**.
- Angular détecte les changements de différentes manières. Pour **l'initialisation des composants**, **Angular appelle explicitement la détection des changements**.
- Pour les **opérations asynchrones**, Angular utilise une **zone** pour détecter les changements aux endroits où les données auraient pu muter et **exécute automatiquement la détection des changements**.

Change Detection

Zones et contexte d'exécution

- Afin de **détecter les éléments susceptible de déclencher un Change Détection**, Angular utilise **zone.js**.
- zone.js peut **suivre et intercepter** les tâches asynchrones.
- Une zone a généralement 3 phases :
 - Elle commence dans un état stable
 - Elle devient instable lorsque une tâche est déclenchée dans la zone
 - Elle redevient stable lorsque les tâches sont finalisées.
- Angular suit plusieurs API de navigateur de bas niveau au démarrage pour pouvoir détecter les changements dans l'application

Change Detection Zone.js c'est quoi ?

```
//Comment savoir quand est ce que le setTimeout commence et quand ca se // termine sans toucher le  
setTimeout  
  
const oldSetTimeout = setTimeout;  
setTimeout = (handler, timer) => {  
  console.log('START');  
  oldSetTimeout(_ => {  
    handler();  
    console.log('FINISH');  
  }, timer);  
}  
  
//-----  
setTimeout(_ => {  
  console.log('some action');  
}, 3000);
```

Change Detection

Zones et contexte d'exécution

- Ceci est donc **délégué à zone.js** qui suit les APIs comme EventEmitter, DOM event listeners, XMLHttpRequest, l'API fs dans Node.js et plus encore.
- Donc **si zoneJs détecte un des déclencheur** du Change Detection, elle **notifie Angular** qui lui va déclencher le processus de Change Detection.
- Angular utilise sa **propre zone** appelée **NgZone**.
- C'est une seule zone et le Change Detection est uniquement déclenché si une **opération Asynchrone est déclenchée dans cette zone**.

Change Detection

Zones et contexte d'exécution

```
// https://github.com/angular/angular/blob/master/packages/core/src/zone/ng_zone.ts#L337
// ngzone simplified
function onEnter() {
  _nesting++;
}
function onLeave() {
  _nesting--;
  checkStable();
}
function checkStable() {
  if (zone._nesting == 0 && !zone.hasPendingMicrotasks) {
    onMicrotaskEmpty.emit(null);
  }
}
//https://github.com/angular/angular/blob/7954c8dfa3c85d12780949c75f1448c8d783a8cf/packages/core/src/application_ref.ts#L628
```

```
this._onMicrotaskEmptySubscription = this._zone.onMicrotaskEmpty.subscribe({
  next: () => {
    this._zone.run(() => {
      this.tick();
    });
  }
})
```

Change Detection

- Il existe **deux stratégies** de Change Detection :
 - La stratégie **par défaut**
 - La stratégie **OnPush**

Change Detection

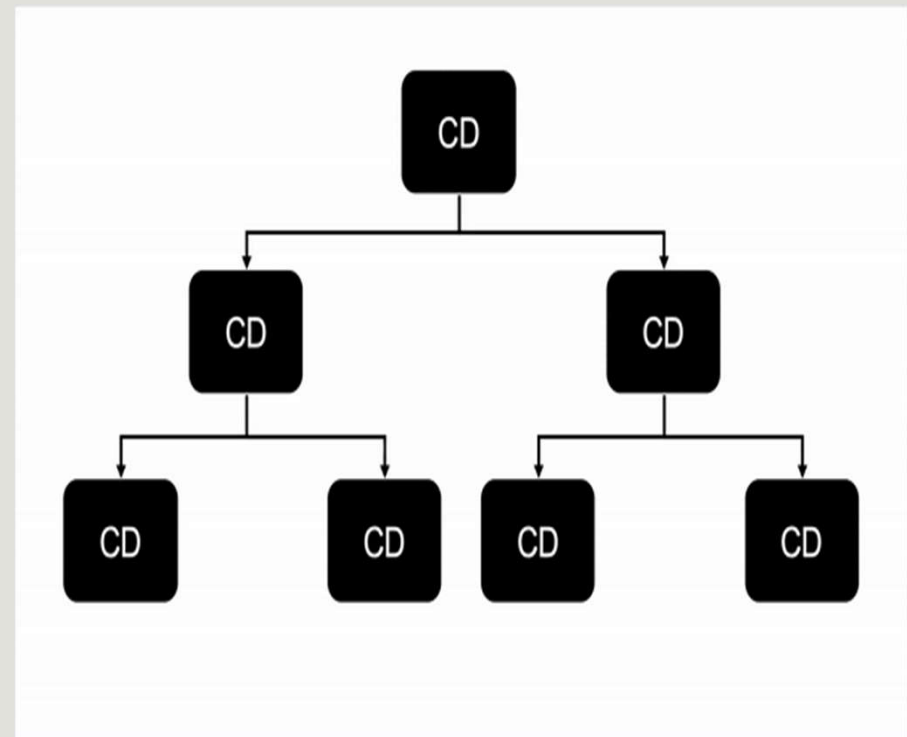
La stratégie par défaut

- Dans la stratégie par défaut, le mécanisme est le suivant :
- 1. NgZone détecte une possibilité de modification.
- 2. Angular est notifié afin de déclencher **le change detection**.
- 3. La détection de changement **vérifie chaque composant dans l'arborescence des composants** de haut en bas **pour voir si le modèle correspondant a changé**. Ceci est appelé **dirty checking**.
- 4. S'il y a une nouvelle valeur, **il mettra à jour la partie correspondante (DOM)**

Change Detection

La stratégie par défaut

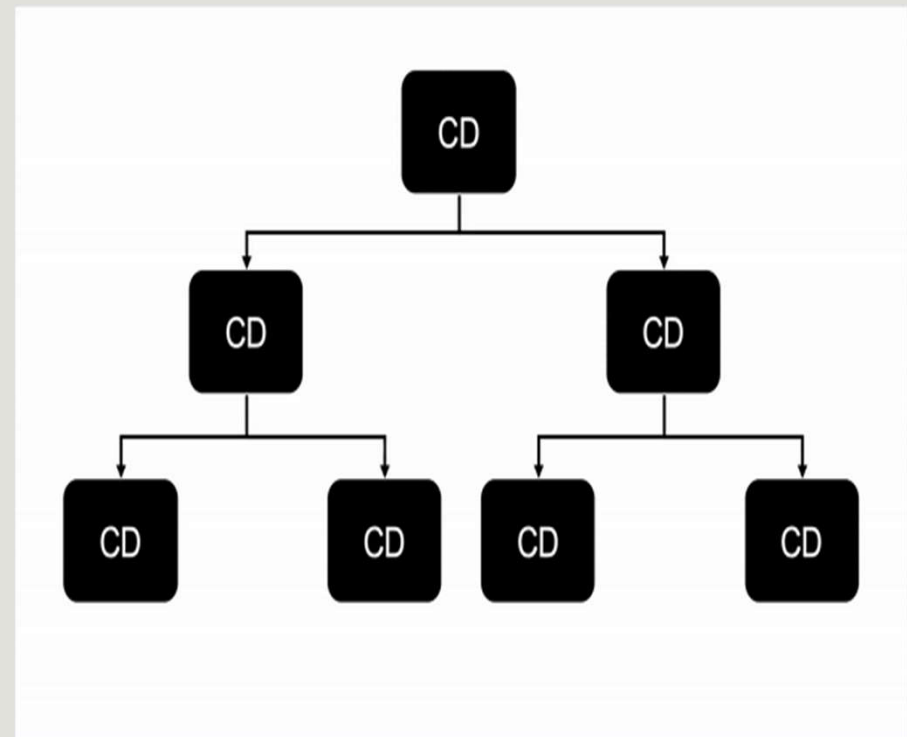
- Ici, dans **tous les composants** de l'arbre de composant, le **change detector alloué à chaque composant**, compare la valeur courante et la valeur précédente des propriétés.
- Si la **valeur change**, il va marquer une propriété **isChanged à true**.



Change Detection

La stratégie par défaut

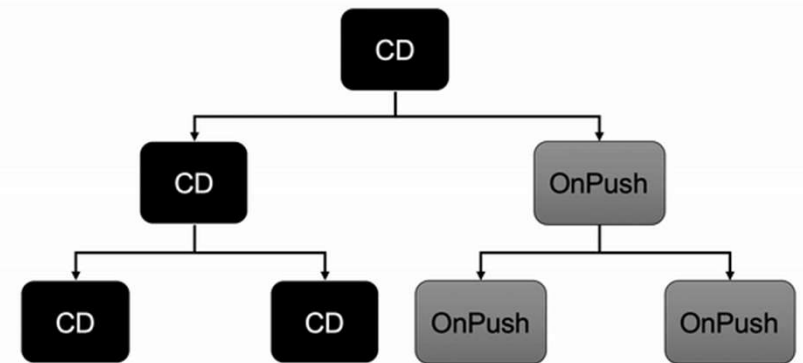
- Cette première stratégie est assez **performante pour les application petite et moyenne**.
- Ceci est du au fait qu'Angular est **très rapide pour le Change détection** pour chaque composant en utilisant la technique du **inline-caching**.
- Cependant, ceci peut ne plus suffire pour les application assez volumineuse.



Change Detection

La stratégie OnPush

- Le but de cette stratégie est d'**éviter** les **tests non nécessaires** pour un **composant et sa descendance**.
- En appliquant cette stratégie, on définit une nouvelle façon pour le déclenchement du Change Detection pour ce composant

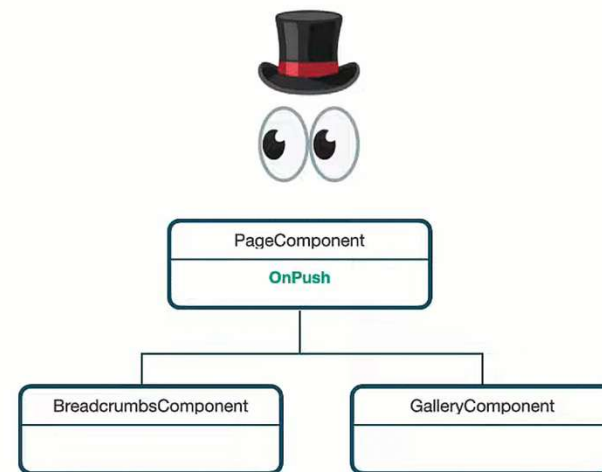


Change Detection

La stratégie OnPush

- Afin d'activer la stratégie **OnPush**, il suffit d'ajouter la propriété **changeDetection** du `@Component` object et de lui affecter la valeur **OnPush** de l'objet **ChangeDetectionStrategy**.

```
@Component({  
  selector: 'app-list',  
  templateUrl: './list.component.html',  
  styleUrls: ['./list.component.css'],  
  changeDetection: ChangeDetectionStrategy.OnPush  
})
```

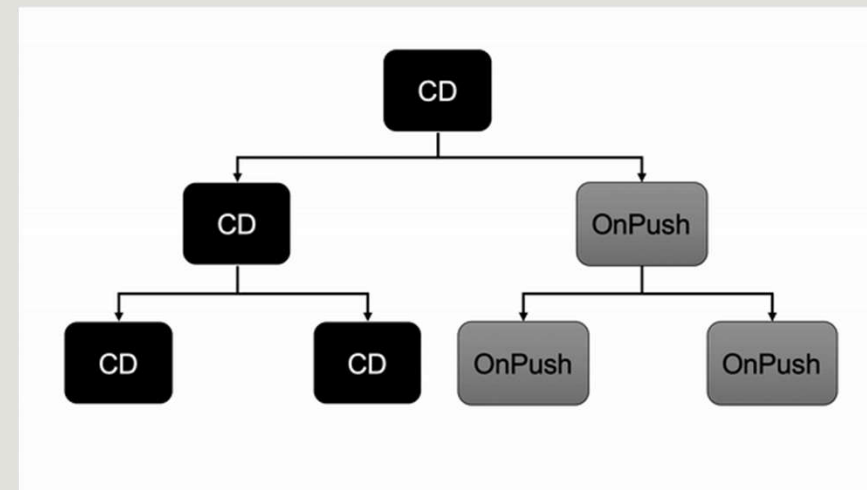


https://www.youtube.com/watch?v=WAu7omIoerM&ab_channel=DecodedFrontend

Change Detection

La stratégie OnPush

- En utilisant cette stratégie, Angular sait que le **composant n'a besoin d'être mis à jour que si** :
 - La **référence** d'entrée d'un **Input** du composant est **modifiée**
 - Le **composant ou l'un de ses enfants** déclenche un **événement**.
 - La **Change Detection** est déclenchée **manuellement**
 - Un **observable** lié au Template via le pipe **async** émet une **nouvelle valeur**.



Change Detection

La stratégie OnPush

- Les actions suivantes **ne déclenchent pas le Change Detection** dans ce contexte :
- `setTimeout`
 - `setInterval`
 - `Promise.resolve().then()`, `Promise.reject().then()`
 - `this.http.get('...').subscribe()` (En générale, n'importe quelle inscription à un Observable RxJs)

Change Detection

La stratégie OnPush

Le changement de la référence Input

- Dans la stratégie de détection de changement par défaut, **Angular** exécutera le détecteur de changement **chaque fois que les données @Input()** sont changées ou modifiées.
- En utilisant la stratégie OnPush, le détecteur de changement **n'est déclenché que si une nouvelle référence est transmise en tant que valeur @Input().**
- Les types primitifs tels que les nombres, les chaînes, les booléens, null et indéfini sont passés **par valeur**.

Change Detection

La stratégie OnPush

Le changement de la référence Input

- L'objet et les tableaux sont également **passés par valeur**, mais la **modification des propriétés** d'objet ou des entrées de tableau **ne crée pas de nouvelle référence et ne déclenche donc pas la détection de changement** sur un composant OnPush.
- Pour **déclencher le détecteur de changement**, vous devez passer une **nouvelle référence** d'objet ou de tableau à la place.
- Immutable.js facilite l'utilisation de l'Immutabilité.
- Il fournit des structures de données immuables persistantes pour les objets (Map) et les listes (List).

Change Detection

Le changement déclenché manuellement

Zone Pollution Pattern

- Ce problème est identifié comme « **Zone Pollution Pattern** »
- Il se produit lorsque Angular Zone encapsule un callback qui déclenche des détections de changement redondants.
- **La solution est donc de déplacer la logique d'initialisation à l'extérieur de Angular Zone**
- **Injecter NgZone**
- Appeler la méthode **runOutsideAngular**
- Passez y une **callback** qui **effectue le traitement que vous voulez isoler.**

Change Detection

Le changement déclenché manuellement

Zone Pollution Pattern

```
constructor (ngZone: NgZone) {  
  ngZone.runOutsideAngular(() => {  
    // runs outside Angular zone, for performance-critical code  
  
    ngZone.run(() => {  
      // runs inside Angular zone, for updating view afterwards  
    });  
  });  
}
```



View and model can
get out of sync!

Change Detection

Le changement déclenché manuellement

Zone Pollution Pattern

- Vous pouvez aussi désactiver complètement la prise en main de zone.js et tout faire vous-même.

```
platformBrowserDynamic().bootstrapModule(AppModule , [  
    {ngZone: 'noop'}  
])
```

```
constructor(private applicationRef: ApplicationRef) {  
    applicationRef.tick();  
}
```

Optimiser une application

Change Detection - La stratégie OnPush

Out of Bound Change Detection

- Ce problème se produit lorsqu'une **action qui modifie uniquement l'état local d'un composant déclenche des changes detection dans des parties qui n'ont aucun rapport avec cette action.**
- Solution : Utiliser OnPush et considérer du refactoring de votre composant.

Problem

Local state change triggers out of bounds change detection

Identification

Change detection performed in components outside the scope of the change

Resolution

Use OnPush and consider refactoring

Change Detection

La stratégie OnPush

Le changement de la référence Input

Optimisation

- Pensez à externaliser les parties où vous avez des événements et le composant recevant le **@Input**.
- Ceci va faire en sorte que le composant avec le **@Input** **ne sera réaffiché que lorsqu'il aura une nouvelle référence**.

Optimiser une application

Recalculation of referentially transparent expressions

- Cette problématique est soulevée lorsque vous avez une **expression** dans votre Template qui doit être **recalculé même lorsque ses paramètres ne changent pas**.

Problem

Redundant calculations

Identification

Detection for changes takes longer than expected given the state changes

Resolution

Use pure pipes or memoization

Change Detection

Recalculation of referentially transparent expressions

Optimisation

- Pensez à utiliser des **pipes** pour vos calculs.
- Il faut avoir deux conditions :
 - **Pas d'effets de bords** (side effect), vous n'appellez pas d'api, pas de console, ...
 - Se sont des **opérations pures**, si l'entrée ne change pas, le résultat ne change pas il reste le même

Pourquoi recalculer alors => utiliser les pipes, si l'entrée ne change pas, il ne recalculera pas l'opération.

Change Detection

Recalculation of referentially transparent expressions

Optimisation

- Vous pouvez alors aller encore plus dans l'optimisation, puisque ce sont des **fonctions pures**. Si une fonction a été **déjà calculée**, **pourquoi le refaire** même si ce n'est pas la même entrée.
- Pensez à utiliser le concept de cache avec le **memo-decorator**.
- importer le décorateur memo de la bibliothèque **memo-decorator** et appliquer le à votre fonction transform.

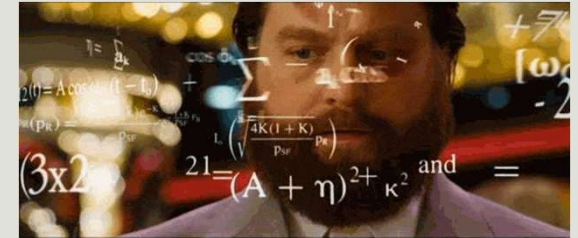
npm i memo-decorator

Pipe pure et impure

- Par défaut un pipe `t` considéré comme une fonction pure
- Une fonction est dite pure si elle :
 - Ne provoque pas de side effect.
 - Renvoie la même valeur pour les mêmes paramètres.
- Lorsque le pipe est pur, la méthode `transform()` est invoquée uniquement lorsque ses arguments d'entrée changent.

```
@Pipe({  
  name: 'myPipe',  
  pure: true  
})
```

Exercice



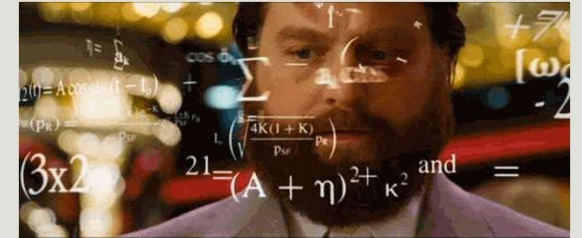
- Créer un composant contenant un input de type texte et une ol
- A chaque fois que vous écrivez dans l'input, afficher le contenu dans un paragraphe (**utilisez ngModel**).
- Dans le composant initialiser un tableau avec **100 valeurs entre 20 et 30**.
- Dans le ol afficher la valeur de chacun des éléments du tableau et en face la valeur de cet élément lorsque on lui applique la fonction suivante:
 - $f(x) = 2f(x-1) + 3f(x-2)$ si $n > 1$ et $f(n) = 1$ si $n == 0$ ou 1
- Tester l'input. Que remarquez vous lorsque vous écrivez dans l'input ?

Memo

- Il existe une bibliothèque **memo-decorator** qui permet de mémoriser des fonctions pures.
- Elle permet en l'utilisant de cacher les résultats des fonctions pures et de les réutiliser.
- Pour l'installer utiliser la commande **npm i memo-decorator**

```
import memo from 'memo-decorator';  
@memo()  
pureFonction(n: number): number { //... }
```

Exercice

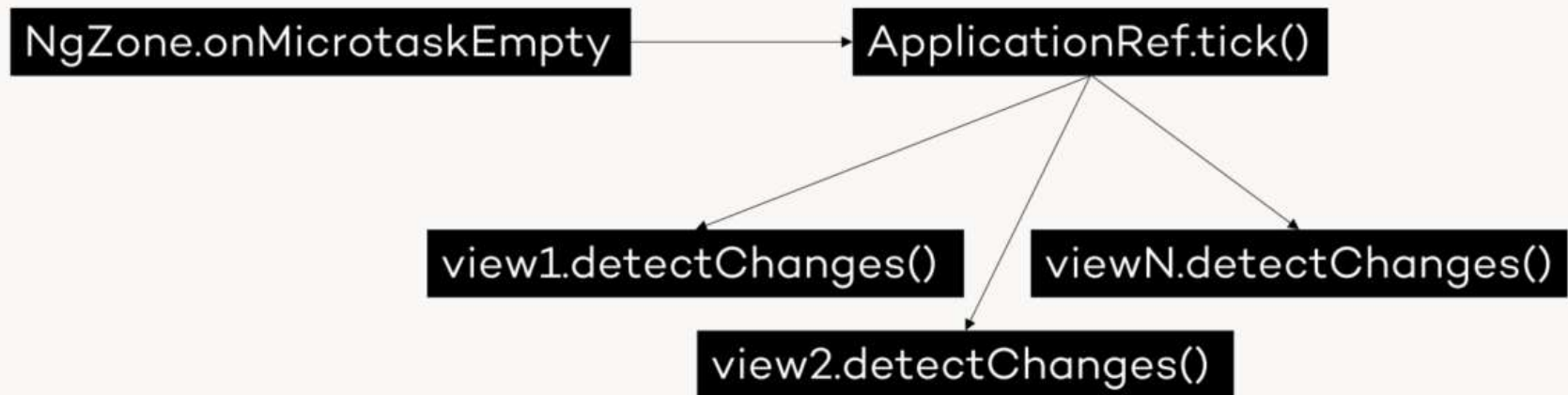


- Utilisez les pipes et utiliser la bibliothèque memo pour optimiser votre pipe

Change Detection

Le changement déclenché manuellement

- Par défaut c'est zone.js à travers NgZone qui gère la partie déclenchement du change detection.



Change Detection

Le changement déclenché manuellement

```
@Injectable()
export class ApplicationRef {
  // ...
  constructor (/* ... */) {
    this._zone.onMicrotaskEmpty
      .subscribe( {next: () => { this._zone.run(() => { this.tick(); }); }});
  }
  // ...
  tick(): void {
    // ...
    for (let view of this._views) {
      view.detectChanges();
    }
    // ...
  }
  // ...
}
```

Change Detection

Le changement déclenché manuellement

- Dans **certains cas d'utilisation**, ce mode de fonctionnement peut causer des problèmes et **ralentir l'application**.
- Prenons le cas des **événements fréquents** qu'on combinera avec des Change Detection ayant beaucoup de calcul :
 - mousemove,
 - des scroll,
 - un setInterval avec des interval courts.

Change Detection

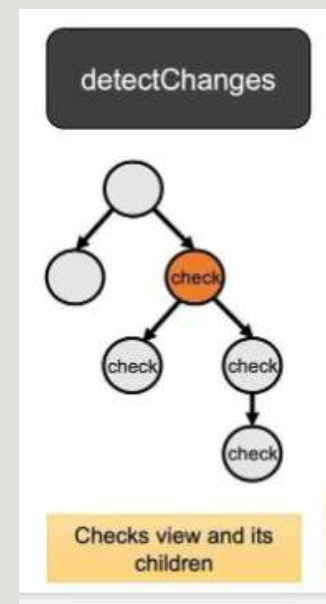
Le changement déclenché manuellement

- Il existe trois méthodes pour déclencher manuellement les détections de changement :
- **La méthode `tick()` de `ApplicationRef`** qui déclenche la détection de changement pour **l'ensemble de l'application** en **respectant la stratégie de détection de changement d'un composant**

Change Detection

Le changement déclenché manuellement

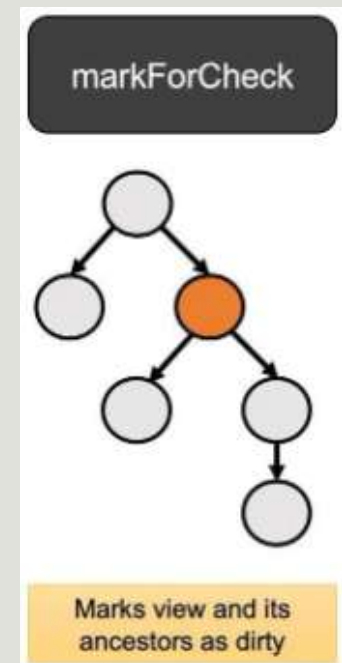
- **detectChanges()** du **ChangeDetectorRef** qui exécute la **détection de changement** sur **ce composant et ses enfants** en gardant à l'esprit la stratégie de détection de changement.
- Il peut être utilisé en combinaison avec **detach()** pour implémenter des contrôles de détection de changement locaux.



Change Detection

Le changement déclenché manuellement

➤ **markForCheck()** de **ChangeDetectorRef** qui **ne déclenche pas la détection de changement** mais **marque tous les ancêtres OnPush** comme devant être vérifiés une fois, soit dans le cadre du cycle actuel ou du cycle de détection de changement suivant. Il exécutera la Change Detection sur les composants **marqués même s'ils utilisent la stratégie OnPush**.

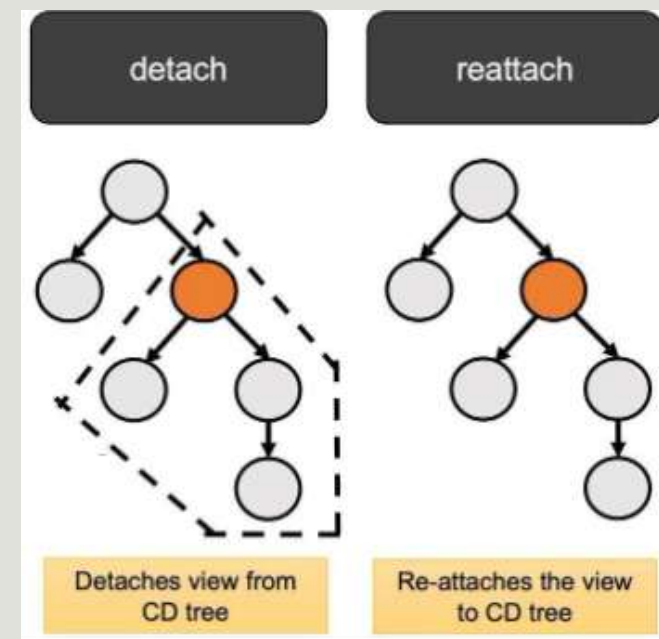


Change Detection

Détacher un composant du change detection

- Vous pouvez **détacher un composant complètement du Change Detection**.
- Ceci peut être fait pour les **composants qui n'ont pas d'état** et donc pas besoin que le ChangeDetector agisse.
- Vous avez **beaucoup de calculs lourds** et vous voulez **gérer seul le déclenchement du Change Detection** localement.
- Vous pouvez rattacher le composant quand c'est nécessaire.

```
constructor(cdRef: ChangeDetectorRef) {  
  cdRef.detach(); // detaches this view from the CD tree  
  // cdRef.detectChanges(); // detect this view & children  
  // cdRef.reattach();  
}
```



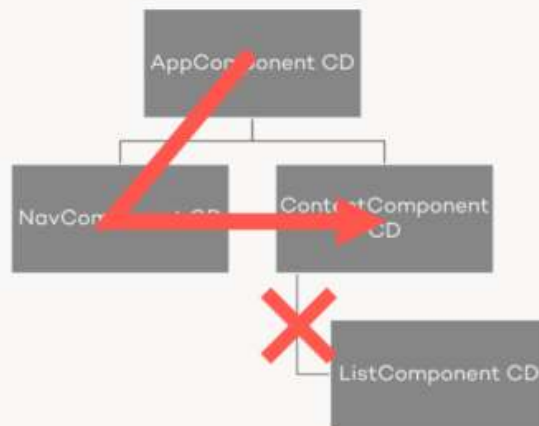
Change Detection

Détacher un composant du change detection

Change Detector

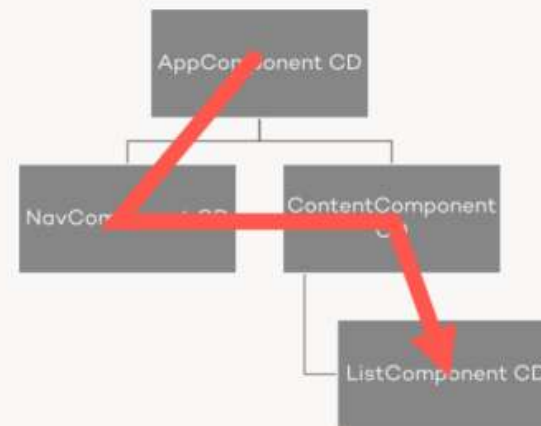
Detaching Components

`changeDetector.detach();`

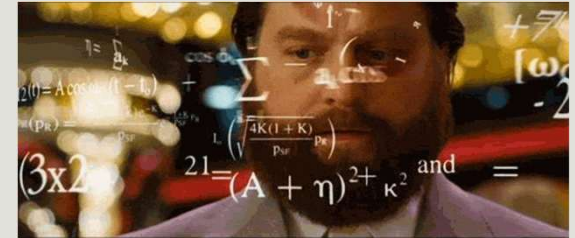


View and model can get out of sync!

`changeDetector.reattach();`

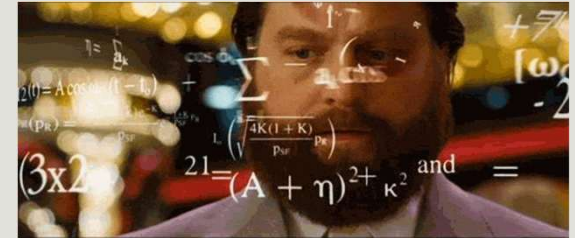


Exercice



- Clone ce projet :
<https://github.com/aymensellaouti/ExempleNgOptimisation>
- Lancez votre projet, c'est le RhComponent qui est exécuté.
- Analysez ses problèmes et essayez de les résoudre avec les techniques étudiées.

Migrer vers Angular 19



- Afin de migrer vers la version 19 à partir de la version 16, j'ai effectué deux migrations :
16>17 puis 17>18 puis 18>19 en utilisant la commande :
- `ng update @angular/core@17 @angular/cli@17` puis
- `ng update @angular/core@18 @angular/cli@18` puis
- `ng update @angular/core@19 @angular/cli@19` puis
- `ng update @angular/core@20 @angular/cli@20`

Angular

Les standalones

composants

AYMEN SELLAOUTI

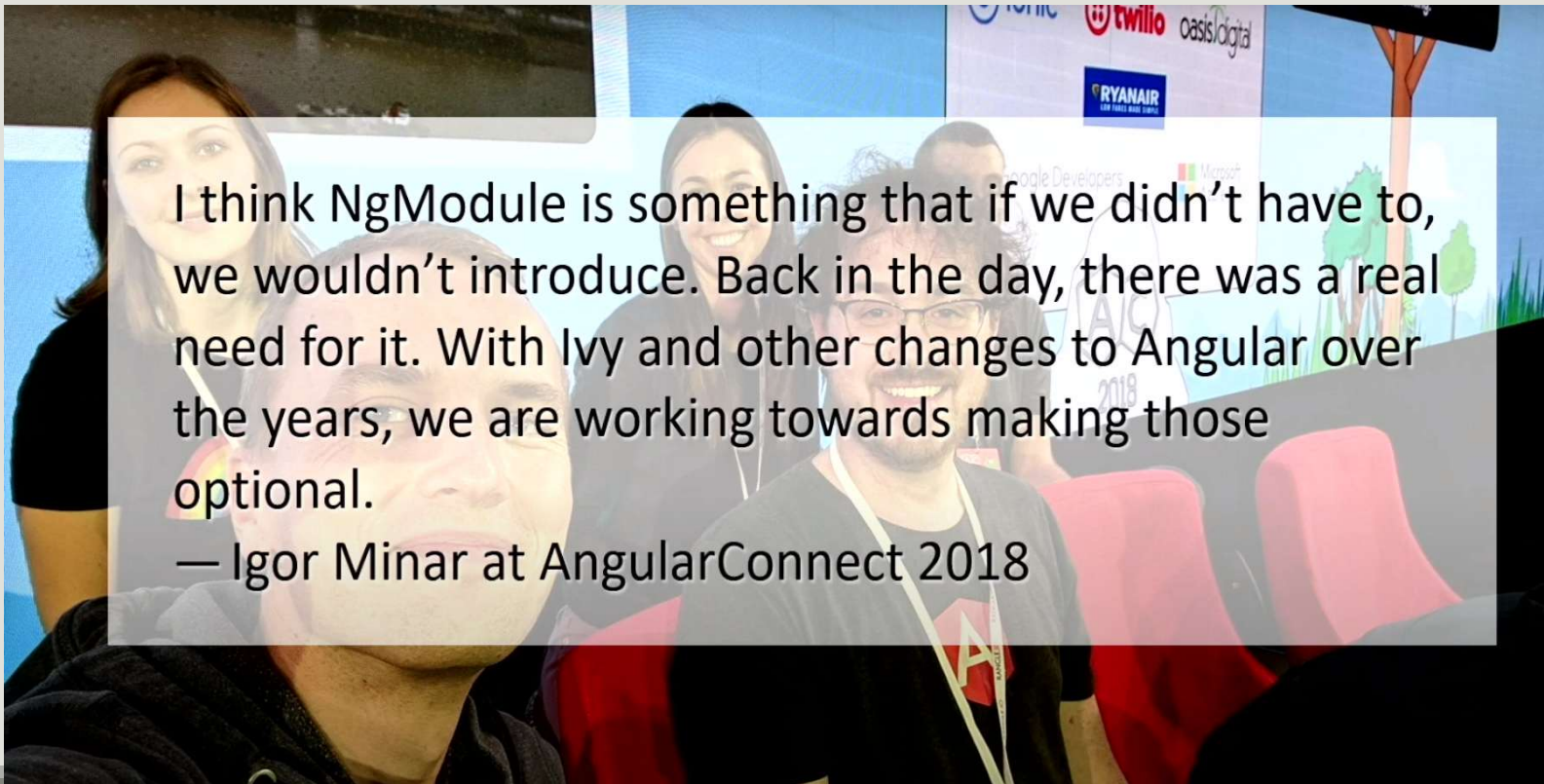
Pourquoi les NgModules

- La principale raison de l'introduction initiale de NgModules était pragmatique : **regrouper des blocs de construction qui sont utilisés ensemble**.
- Ceci permet non seulement d'augmenter la commodité pour les développeurs, mais également pour **le compilateur Angular**.
- En effet le NgModule permet de **définir le contexte de compilation**. A partir de ce contexte, le compilateur apprend où le programme est autorisé à appeler quels composants.
- Les deux parties **imports et declarations** permettent de définir ce contexte de compilation.

Pourquoi les standalone Components ?

- Le premier problème qui s'est posé est **l'ambiguïté** que pose ce **deuxième système de module en parallèle à celui de EcmaScript**.
- Ceci a **compliqué l'apprentissage** pour les nouveaux apprenants.
- Ceci a poussé la Team Angular à faire en sorte qu'avec Ivy, le nouveau compilateur d'Angular, **on ait un contexte de compilation par composant**.
- C'est ce qui a permis la naissance des standalone component.

Pourquoi les standalone Components ?



I think NgModule is something that if we didn't have to, we wouldn't introduce. Back in the day, there was a real need for it. With Ivy and other changes to Angular over the years, we are working towards making those optional.

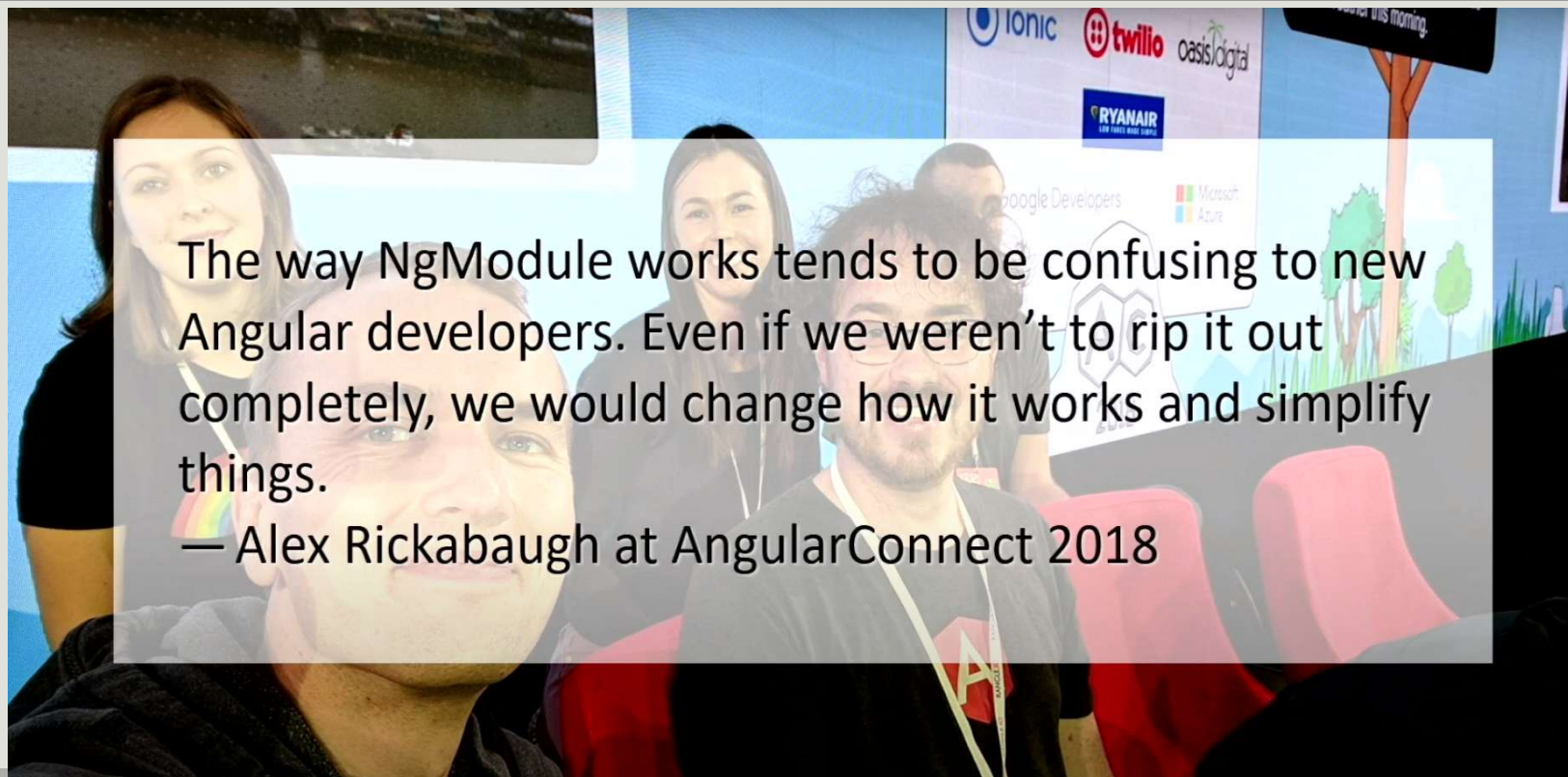
— Igor Minar at AngularConnect 2018

https://www.youtube.com/watch?v=DA3efofhpq4&ab_channel=ngVikings

Pourquoi les standalone Components ?

Single Component Angular Module

SCAM



The way NgModule works tends to be confusing to new Angular developers. Even if we weren't to rip it out completely, we would change how it works and simplify things.

— Alex Rickabaugh at AngularConnect 2018

Pourquoi les standalone Components ?

Single Component Angular Module

SCAM

- La deuxième problématique est la **granularité de lazy loading**.
- La question que beaucoup d'équipes se sont posée est : Pourquoi devons nous faire du **lazy loading pour tout un module quand nous avons besoin de juste un composant ?**
- C'est là où le pattern **SCAM (Single Component Angular Module)** a été introduit, c'est l'ancêtre du Standalone Component

Standalone Component

Les composants autonomes

- Un standalone component est un **composant indépendant d'un module**.
- Les composants standalone offrent un **moyen simplifié** de créer des applications Angular.
- Les applications existantes peuvent éventuellement et progressivement adopter le nouveau style autonome sans aucune modification majeure.
- Ceci est aussi **applicable** aux **directives** et aux **pipes**.
- L'utilisation d'une telle vision **simplifie l'apprentissage d'Angular**
- Elle **facilite aussi plusieurs aspects** comme le **lazy loading**

Standalone Component

Les composants autonomes

- Une nouvelle propriété a été introduite au niveau de l'objet de configuration, c'est la propriété **standalone**.
- Si elle est à **true**, l'élément est **considéré standalone**.
- Ces éléments **ne doivent plus être associés** à un **tableau déclaration** d'un NgModule.
- Une nouvelle clé **import** permet aussi **d'importer les dépendances nécessaires**, que ce soit d'autres standalone components ou d'autres NgModule.
- Pour faire simple un composant Standalone est un **composant autonome**, avant c'est sa **famille** (Le NgModule auquel il appartient) qui été **en charge de ses dépendances**(imports et declarations). **Maintenant, c'est à lui de le faire**

Standalone Component

Les composants autonomes

- Afin de créer un *standalone* component, procéder comme vous le faite d'habitude avec un composant et ajouter l'option **–standalone** (à partir d'Angular 18 c'est de venu le comportement par défaut).

```
@Component({  
  selector: 'app-standalone',  
  standalone: true,  
  imports: [CommonModule],  
  templateUrl: './standalone.component.html',  
  styleUrls: ['./standalone.component.css']  
})  
export class StandaloneComponent {}
```


Standalone Component

Les composants autonomes

- Les **imports définissent le contexte de compilation du composant** : tous les autres blocs de construction que les composants autonomes sont autorisés à utiliser.
- Elles permettent donc **d'importer d'autres composants autonomes, mais aussi des NgModules existants**.
- Ceci rend le **composant autonome et augmente ainsi sa réutilisabilité**.
- Elle nous oblige également à réfléchir aux dépendances du composant.
- **Cette tâche s'avère extrêmement monotone et chronophage**. Ce qu'on faisant une fois dans le module on le fera pour chaque composant.
- Les IDEs essayent de remédier à ça via les autocomplètes

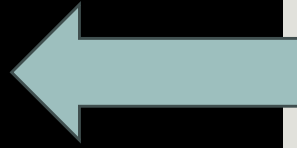
```
@Component({  
  selector: 'app-standalone',  
  standalone: true,  
  imports: [  
    CommonModule,  
    AnotherStandaloneComponent  
  ],  
  templateUrl: './standalone.component.html',  
  styleUrls: ['./standalone.component.css']  
})  
export class StandaloneComponent {}
```

Standalone Component

Le modèle mental

- Même si ce n'est pas géré de la même manière par le compilateur d'Angular, vous pouvez voir un standalone component comme un NgModule avec un seul composant

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  imports: [
    RouterOutlet,
    HighlightDirective,
    CardComponent
  ],
  standalone: true,
})
export class AppComponent {
```



```
@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule,
  ],
})
export class AppModule {}
```

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
})
export class AppComponent {}
```

Standalone Component

Les composants autonomes

Utilisation dans les NgModule

- Si vous voulez utiliser un **Standalone Component** **dans un NgModule**, **importer** le **comme** vous importer **un module**.
- Avec cette manière de faire, un **composant standalone peut être importé dans plusieurs modules**, alors qu'avant **un composant ne pouvait être déclaré que dans un seul module**.

```
@NgModule({  
  declarations: [],  
  imports: [  
    BrowserModule,  
    AppRoutingModule,  
    FormsModule,  
    HttpClientModule,  
    StandaloneComponent  
  ],  
  providers: [TestService],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

Standalone Application

Pipes et directives

- Tout comme les composants autonomes, il existe également des **pipes et des directives autonomes**.
- À cet effet, les **décorateurs des pipes et des directives** bénéficient également d'une **propriété standalone**.

```
@Directive({
  selector: '[appHighlight]',
  standalone: true,
  host: {
    '[style.backgroundColor]': 'this.color()',
    '(mouseenter)': 'this.onMouseEnter()',
    '(mouseleave)': 'this.onMouseLeave()',
  }
})
export class HighlightDirective {
```

```
@Pipe({
  name: 'btc2Usd',
  pure: true,
  standalone: true
})
export class Btc2UsdPipe implements PipeTransform {

  transform(value: unknown, ...args[]){}

}
```

Standalone Component

Bootstraper une application avec un Standalone Component

- Afin de **bootstraper** votre application avec un **standalone component**, utilisez la fonction **bootstrapApplication** dans **main.ts**, à la place de **platformBrowserDynamic().bootstrapModule**, en lui passant le **Standalone Component** que vous voulez déclencher au **lancement de l'application**.
- Dans **index.html** appelez **ce Standalone Component**.

```
bootstrapApplication(StandaloneComponent);
```

main.ts

```
<body>  
<div class="container">  
  <app-standalone></app-standalone>  
</div>  
</body>  
</html>
```

index.html

Standalone Component

Les composants autonomes

Configurez l'injection de dépendance

- Le deuxième paramètre de `bootstrapApplication` est un objet de type **ApplicationConfig**.
- Cet objet contient une **clé providers** qui prend en paramètre un **tableau de providers**.

```
interface ApplicationConfig {  
    /**  
     * List of providers that should be available to the  
     * root component and all its children.  
     */  
    providers: Array<Provider | EnvironmentProviders>;  
}
```

Standalone Component

Les composants autonomes

Configurez l'injection de dépendance

```
interface AppConfig {  
    /**  
     * List of providers that should be available to the  
     * root component and all its children.  
     */  
    providers: Array<Provider | EnvironmentProviders>;  
}
```

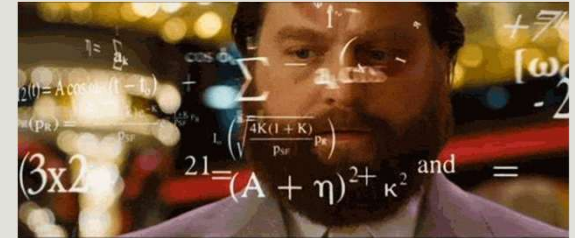
- Vous pouvez **recupérer des providers** offerts par un **module** avec la fonction **importProvidersFrom(moduleCible)**.
- A partir de la **version 15**, vous avez **certaines fonctions spécifiques** pour les **modules** les **plus utilisés** comme le **http** avec sa fonction **provideHttpClient()**, ou **provideRouter(APP_ROUTES)**.

Standalone Component

Configurez l'injection de dépendance

```
bootstrapApplication(AppComponent, {  
  providers: [  
    importProvidersFrom(BrowserModule, ToastrModule.forRoot()),  
    AuthInterceptorProvider,  
    {  
      provide: LoggerProviderToken,  
      useClass: LoggerService,  
      multi: true,  
    },  
    provideRouter(routes),  
    provideHttpClient(withInterceptorsFromDi()),  
    provideAnimations(),  
  ],  
})
```


Exercice



Angular offre un moyen **de migrer automatiquement une partie** de votre application Angular modulaire (15.2.0) et plus vers une application standalone.

- Vous pouvez suivre les différentes étapes du Guide qui présente des parties automatiques et d'autres manuelles.
- Exécutez la migration **dans l'ordre indiqué** ci-dessous, **en vérifiant que votre code est généré et exécuté entre chaque étape** :
 1. Exécutez `ng g @angular/core:standalone` et sélectionnez "Convert all components, directives and pipes to standalone"
 2. Exécutez `ng g @angular/core:standalone` et sélectionnez "Remove unnecessary NgModule classes", **cette étape risque de garder plusieurs modules.**
 3. Exécutez `ng g @angular/core:standalone` et sélectionnez "Bootstrap the project using standalone APIs".

Angular 17

Le nouveau flux de control

- Afin de continuer sur la logique de simplification de l'apprentissage d'Angular, l'équipe Angular a proposé de **nouveaux flux de contrôle**.
- Les flux suivants ont été proposés :
 - @If
 - @for
 - @switch

Control flow

@if

- ▶ **@if** est associé à une expression booléenne. Si cette expression est fausse (false) alors l'élément et son contenu sont retirés du DOM, ou jamais ajoutés).
- ▶ **@if(condition)**
 - ▶ Si le booléen est true alors l'élément host est visible.
 - ▶ Si le booléen est false alors l'élément host est caché.

```
@if (isAuthenticated()) {  
<button  
  (click)="deleteCv(cv)"  
  class="btn btn-danger">  
  Delete  
</button>  
}
```

Control flow

@if, @else if et @else

- ▶ **@if** peut également être utilisé avec **@else if** et/ou **@else** selon le besoin.
- ▶ La **syntaxe est très intuitive**, demandé vous comment vous aurez fait en Javascript et **ajoutez un @ :D**.

```
@if (connectedUser) {  
    Hello {{ connectedUser.name }}  
} @else {  
<div>Merci de vous connectez</div>  
}
```

Control flow

@if, @else if et @else

```
@if (!connectedUser) {  
  <div class="alert alert-danger">Merci de vous connectez</div>  
} @else if (!connectedUser.activated) {  
  <div class="alert alert-warning">  
    Hello {{ connectedUser.name }}, merci d'activer votre compte  
  </div>  
} @else {  
  <div class="alert alert-success">Hello {{ connectedUser.name }}</div>  
}
```

Control flow

@For

- ▶ La directive structurelle **@for** permet de boucler sur un itérable et d'injecter les éléments dans le DOM.

```
<ul>
  <li *ngFor="let episode of episodes">{{ episode.title }}</li>
</ul>
<ul>
  @for (episode of episodes; track episode.id) {
    <li>{{ episode.title }}</li>
  }
</ul>
```

Control flow

@For

- ▶ **@for** fournit certaines informations sur la boucle en cours :
 - ▶ *\$index*: position de l'élément.
 - ▶ *\$odd*: true si l'élément est à une position impaire.
 - ▶ *\$even*: true si l'élément est à une position paire.
 - ▶ *\$first*: true si l'élément est à la première position.
 - ▶ *\$last*: true si l'élément est à la dernière position.

```
<ul>
  <li
    *ngFor="
      let episode of episodes;
      let i = index;
      let isOdd = odd;
      let isFirst = first
    "
    [ngClass]="{ odd: isOdd, bgfonce: isFirst }"
  >
    Episode {{ i + 1 }}{{ episode.title }}
  </li>
</ul>

<ul>
  @for (episode of episodes; track episode.id) {
    <li
      [ngClass]="{ odd: $odd, bgfonce: $first }"
    >
      Episode {{ $index + 1 }} : {{ episode.title }}
    </li>
  }
</ul>
```

Control flow

@For

track

3
0
4

- ▶ Avec **@For** la **fonction de tracking** est devenue obligatoire
- ▶ La fonction de suivi créée via l'instruction **track** est **utilisée pour permettre au mécanisme de détection des changements d'Angular de savoir exactement quels éléments mettre à jour dans le DOM** après les modifications de l'itérable d'entrée.
- ▶ La fonction de suivi **indique à Angular comment identifier de manière unique un élément de la liste.**

```
@for (episode of episodes; track episode.id) {  
  <li>{{ episode.title }}</li>  
}
```


Control flow

@For

track

3
0
5

- ▶ En principe, il devrait toujours y avoir quelque chose d'unique dans les éléments sur lesquels vous itérer.
- ▶ Dans le pire des cas, s'il n'y a rien d'unique dans les éléments du tableau, vous pouvez utiliser `$index` de l'élément, c'est-à-dire la position de l'élément dans le tableau.

```
@for (episode of episodes; track $index) {  
  <li>{{ episode.title }}</li>  
}  
</ul>
```

Control flow

@For

track

3
0
6

- track peut prendre en paramètre une fonction à laquelle on passe l'index et l'élément actuel de l'itération

```
@for (episode of episodes; track trackFn($index, episode)) {  
  <li>{{ episode.title }}</li>  
}
```

```
trackFn(index: number, episode: { id: number; title: string; }){  
  return episode.id;  
}
```

Control flow

@For, la gestion d'un itérable vide

- ▶ Afin de gérer le cas où votre itérable est vide, nous avons le block `@empty`.
- ▶ Ce bloque n'est activé que si l'itérable sur lequel vous bouclez est vide.

```
<ol class="list-group">
  @for (player of players; track player.id) {
    <li class="list-group-item">{{player.name}}</li>
  }
  @empty {
    <li class="list-group-item list-group-item-danger">La liste ne nous est
pas encore parvenue</li>
  }
</ol>
```

Control flow

@switch

- ▶ Avec **@switch**, vous pouvez créer des switch très simplement.
- ▶ Vous avez les trois opérateurs : **@switch**, **@case**, **@default**
 - ▶ **@switch** : définir l'élément sur lequel switcher
 - ▶ **@case** : pour identifier le cas
 - ▶ **@default** : pour les valeurs par défaut

```
<div [ngSwitch]="streamingService">
  <div *ngSwitchCase="'AppleTV'">Ted Lasso</div>
  <div *ngSwitchCase="'Disney+'">Mandalorian</div>
  <div *ngSwitchDefault>Peaky Blinders</div>
</div>
@switch(streamingService) {
  @case ('Disney+') {
    <div>'Mandalorian'</div>
  } @case ('AppleTV') {
    <div>'Ted Lasso'</div>
  } @default {
    <div>'Peaky Blinders'</div>
  }
}
```

```
@Component({
  standalone: true,
  imports: [NgSwitch, NgSwitchCase, NgSwitchDefault],
})
export class SwitchComponent {
  streamingService = 'Netflix';
}
```

Control flow

Migration automatique

- ▶ Si vous voulez migrer votre ancien code et utilisez le nouveau work flow, angular vous fournie une commande qui le fait pour vous :

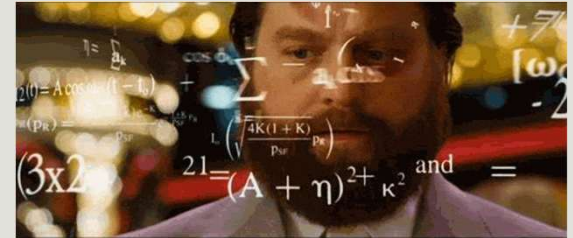
`ng g @angular/core:control-flow`

Cette fonctionnalité est encore en developper Preview (Angular 17.2)

Migrer vers le nouveau flux de control

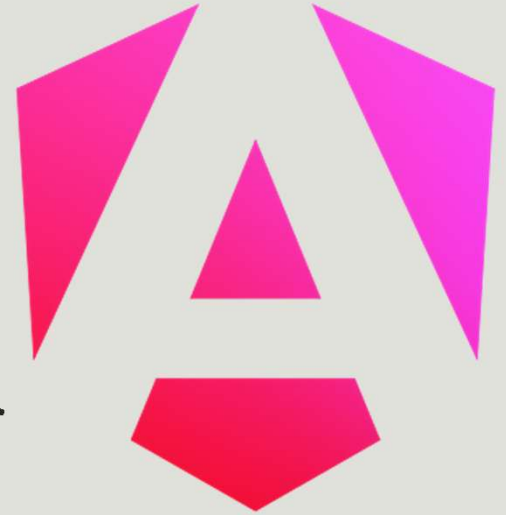
- Si vous voulez migrer votre ancien code et utilisez le nouveau work flow, angular vous fournie une commande qui le fait pour vous :
 - `ng g @angular/core:control-flow`
- Cette fonctionnalité est encore en developper Preview pour Angular 17.2

Exercice

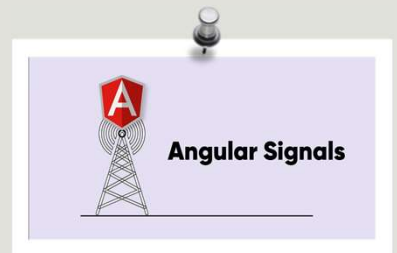


- Afin d'utiliser la nouvelle fonction inject, vous pouvez utiliser la commande : **ng generate @angular/core:inject**

Les Signaux



AYMEN SELLAOUTI



Qu'est ce qu'un Signal ?



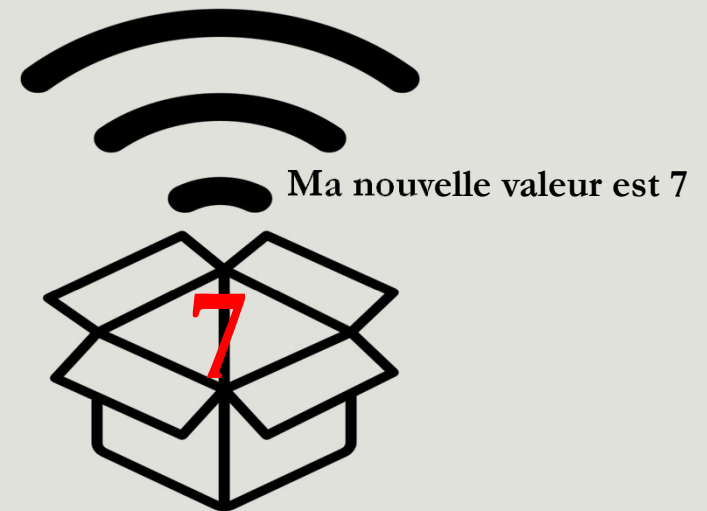
- Un signal agit comme une **enveloppe (wrapper) autour d'une valeur**, ayant la capacité **d'avertir les consommateurs lorsque la valeur change**.
- On peut modéliser ça en disant qu'un signal c'est une donnée qui a le pouvoir de notifier ses changements.



`value = signal(5)`



`value()`



`value.set(7)`

Qu'est ce qu'un Signal ?



- Un signal est une **primitive réactive** qui représente une valeur et qui nous permet de
 - **suivre ses changements** au fil du temps.
 - **modifier** cette même valeur en **notifiant tous ceux qui en dépendent**.
- Elle permet de définir l'état réactif de votre application et d'avoir une **identification précise de quels composants sont impactés par un changement**.

Qu'est ce qu'un Signal ?



- Les signaux sont un concept utilisé dans plusieurs frameworks (Qwik, SolidJs, Vue, KnockoutJs)
- Le concept du **Signal** dans Angular est une fonctionnalité introduite en **‘Developer Preview’** dans la version **16** de la bibliothèque `@angular/core` et **stable à partir de Angular 17**.
- Il a pour objectif **de simplifier le développement** en donnant une **alternative plus simple que RxJS** pour gérer **certains cas de réactivité** d’une façon plus simple.

Pourquoi intégrer les signaux



Reactivity Everywhere

Les signaux permettent de réagir aux changements d'état (State) n'importe où dans notre code et pas seulement au sein d'un composant.



Precision Updates

Les signaux boostent les performances de votre application en réduisant le travail que fait Angular pour garder le DOM à jour avec les valeurs des données

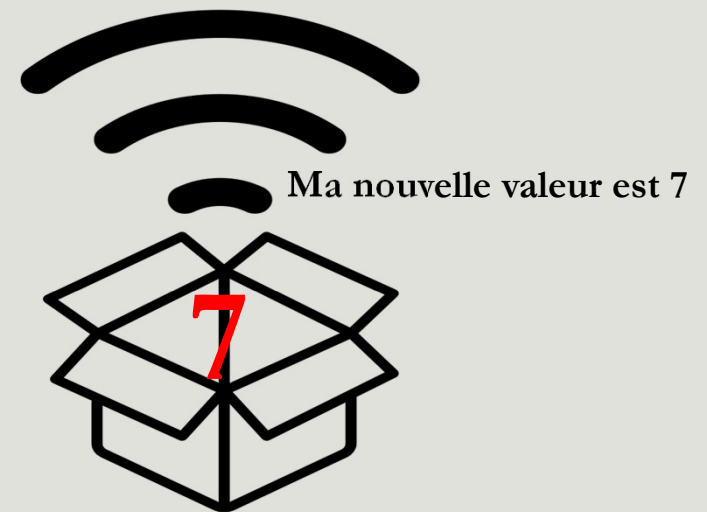


Lightweight Dependencies

Les signaux pèsent 2KB, n'ont pas besoin de charger des dépendances tierces et ne représentent aucun coût de démarrage lorsque votre application se charge.

API

- ▶ Angular propose trois principales primitives pour utiliser les signaux :
 - ▶ signal
 - ▶ computed
 - ▶ effect



Créer un signal via l'api signal()

- La fonction **signal** permet d'initialiser une variable et d'informer Angular et son contexte à chaque fois que sa valeur change.
- Elle retourne un objet de type **WritableSignal**.
- **Un signal doit avoir une valeur initiale**

```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  styleUrls: ['./signal-api.component.css'],
  template: ` <h1>Hello World</h1> `,
})
export class SignalApiComponent {
  lastname: WritableSignal<string> = signal('sellaouti');
}
```

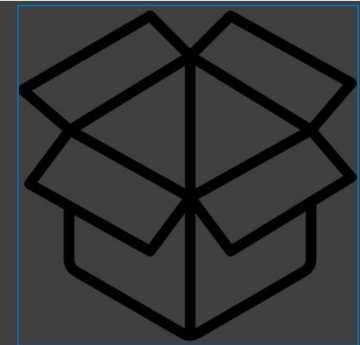


Récupérer la valeur d'un signal

- Afin de **récupérer la valeur d'un signal** il suffit de l'appeler comme une fonction.

```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  styleUrls: ['./signal-api.component.css'],
  template: ` <h1>Hello {{ lastname() }}</h1> `,
})
export class SignalApiComponent {
  lastname: WritableSignal<string> = signal('sellaouti');
}
```

sellaouti



Les méthodes de modifications de signal : set() et update()

- Pour modifier la valeur d'un **signal**, on peut passer par :
- La Méthode **set**, qui permet **d'affecter une nouvelle valeur au signal**.
- La méthode **update**, qui permet de **calculer une nouvelle valeur d'un signal en fonction de sa valeur précédente**.

```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  template: `
    <h1>Hello {{ lastname() }}</h1>
    <input type="number" #input
      (change)="setCounter(+input.value)"
    />
    <h2 (click)="increment()">
      Click here. You clicked {{ counter() }} times
    </h2>
  `,
})
export class SignalApiComponent {
  lastname = signal('aymen');
  counter = signal(0);
  increment() {
    this.counter.update((currentValue) => currentValue + 1);
  }
  setCounter(val: number) {
    this.counter.set(val);
  }
}
```


Quand utiliser un signal et non une variable standard ?

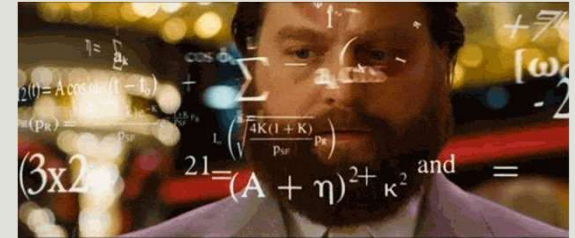
➤ Utilisez les signals

- pour chaque **variable d'état de votre composant dont la valeur change**.
- lorsque vous voulez **suivre les changements** de votre variable
- quand la **valeur de votre variable** doit **changer quand d'autres variables changent**

➤ N'utilisez pas les signals

- quand la **valeur ne change pas**
- Quand c'est une **variable locale** que vous **n'utilisez pas dans votre ui** ou dans dans **calculs réactifs**
- pour les **événements**
- pour les **opérations asynchrones**

Exercice



- Reprenez L'exercice 'ColorComponent' et rotatingCardComponent en utilisant les signaux.

Cas d'utilisation

- Soit le composant SumComponent. Que se passe-t-il si on incrémente x ou y ?

```
export class SumComponent {  
  x = 3;  
  y = 5;  
  z = this.x + this.y;  
}
```

```
<input type="number" [(ngModel)]="x"  
class="form-control">  
<input type="number" [(ngModel)]="y"  
class="form-control">  
{{ x }} + {{ y }} = {{ z }}
```

computed()

- L'api `computed()` permet de créer un **nouveau signal** dont la valeur **dépend d'autres signaux**.
- Lorsqu'un **signal** est mis à jour, **tous ses signaux dépendants seront alors automatiquement mis à jour**.
- On note que `computed()` retourne un objet de type **Signal** et non **WritableSignal**.
- Vous **ne pouvez pas modifier un computed**, il n'est pas Writable

```
lastname = signal('aymen');  
firstname = signal('sellaouti');  
fullname = computed(() => `${this.firstname()} ${this.lastname()}`)
```

computed()

Modifier un signal dans un computed

- computed() s'attend à ce que le calcul **soit léger et rapide**, et **n'entraîne aucun effet secondaire**.
- Vous **ne pouvez pas modifier un signal dans un computed**, ceci provoquera une **erreur**.
- Ne **modifiez aucune variable** dans les computed ; pas seulement les signaux, mais aussi les variables extérieures à la fonction.
- Ne **modifiez pas le DOM** et **n'exécutez pas de tâches asynchrones**.

```
fullname = computed(() => {  
  console.log('i am computing....');  
  this.firstname.set('ccc');  
  return `${this.firstname()} ${this.lastname()}`;  
});
```

```
✖ ▶ ERROR Error: NG0600: Writing to VM1270:1  
signals is not allowed in a `computed` or  
an `effect` by default. Use  
`allowSignalWrites` in the  
`CreateEffectOptions` to enable this inside  
effects.
```

computed()

- ▶ Pour identifier un signal qui a changé, et donc si on doit exécuter un computed, Angular utilise **Object.is**.
- ▶ Object.is() permet de déterminer si deux valeurs sont identiques. Deux valeurs sont considérées identiques si :
 - ▶ elles sont toutes les deux undefined
 - ▶ elles sont toutes les deux null
 - ▶ elles sont toutes les deux true ou toutes les deux false
 - ▶ elles sont des chaînes de caractères de la même longueur et avec les mêmes caractères (dans le même ordre)
 - ▶ **elles sont toutes les deux le même objet (même référence)**
 - ▶ elles sont des nombres et
 - ▶ sont toutes les deux égales à +0
 - ▶ sont toutes les deux égales à -0
 - ▶ sont toutes les deux égales à NaN

computed()

Comment ça marche ?

- ▶ L'arbre de dépendance est créé dynamiquement à chaque appel du computed.
- ▶ Il faut donc faire très attention dans la définition de vos computed lorsqu'il y a des traitements conditionnels.

```
@Component({
  template: `
    <h3>Counter value {{ $counter() }}</h3>
    <h3>Derived counter: {{ $derivedCounter() }}</h3>
    <button (click)="increment()">Increment</button>
    <button (click)="multiplier = 10">Set multiplier to 10</button>
  `,})
export class ComputedProblemComponent {
  $counter = signal(0);
  multiplier: number = 0;
  $derivedCounter = computed(() => {
    if (this.multiplier < 10) {
      return 0;
    } else {
      return this.$counter() * this.multiplier;
    }
  });
  increment() {
    console.log(`Updating counter...`);
    this.counter.set(this.$counter() + 1);
  }
}
```



computed()

Exclure un signal de l'arbre de dépendance

- ▶ Si, pour une raison ou une autre, vous voulez lire une valeur d'un signal dans un computed mais sans l'intégrer dans l'arbre de dépendance, vous pouvez utiliser l'Api **untracked**.
- ▶ Dans cet exemple le computed fullname ne sera mis à jour que si le signal firstname change

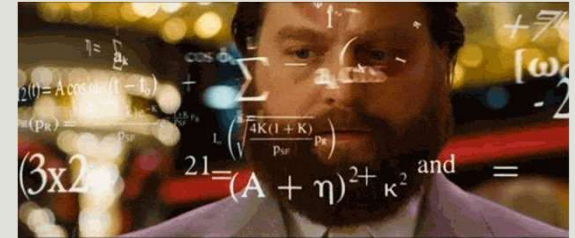
```
lastname = signal('sellaouti');  
firstname = signal('aymen');  
fullname = computed(() => `${this.firstname()} ${untracked(this.lastname)}`);
```


computed()

Les computed, autres propriétés

- Les computed sont paresseux (**lazy**), ce qui signifie que computed **n'est invoquée que lorsque quelqu'un s'intéresse (lit) à sa valeur**. Cela permet **d'optimiser les performances** en évitant les calculs inutiles.
- Les computed sont **automatiquement supprimés** lorsque la référence du signal calculée devient hors de portée.
- Cela garantit que les ressources **sont libérées et qu'aucune opération de nettoyage explicite n'est requise**.
- Ceci est due à l'utilisation des weakReference avec les signaux

Exercice



- Créer un composant TTC qui permet de calculer le prix TTC d'un produit selon le nombre de pièces et la TVA. Sachant que l'utilisateur peut changer toutes les valeurs et que par défaut la quantité est de 1 le prix est de 0 et la tva est de 18.
- Si le nombre de pièces est entre 10 et 15 une remise de 20% est appliquée.
- Si le nombre de pièces est supérieur à 15 une remise de 30% est appliquée.

TTC Calculator

100	10	18
Prix HT	Quantité	TVA
Prix unitaire TTC : \$118.00	Prix total TTC : \$1,180.00	Discount : \$0.00

L'API effect()

- Dans certains cas d'utilisation, vous avez besoin d'être notifié par le changement d'un signal pour **effectuer un effet de bord**, donc faire un **traitement sans pour autant créer un nouveau signal ou en modifier d'autres**.
- Pensez à un log, update du localStorage,...
- L'effect va être réexécuté si l'un des signaux qu'il utilise émet une nouvelle valeur.

L'API effect()

- Vous pouvez **créer un effet** via la fonction **effect**.
- Les effets s'exécutent toujours **au moins une fois**.
- Lorsqu'un effet est exécuté, il **suit tous les signaux qu'il contient**. Chaque fois que l'une de ces valeurs de **signal change**, **l'effet se reproduit**.
- L'effet est **semblable aux computed**, les effets gardent une **trace de leurs dépendances de manière dynamique** et ne suivent que les **signaux** qui ont été **lus lors de l'exécution la plus récente**.

```
constructor() {  
  effect(() => {  
    console.log(`count:${this.counter()}`);  
  });  
}
```

```
private logEffect = effect(() => {  
  console.log(`  
    The current count is:${this.counter()}`  
  `);  
});
```

L'API effect()

- Moment d'exécution : Un effect s'exécute pendant le cycle de détection des changements, après que les signaux suivis ont été mis à jour, mais avant que le DOM ne soit complètement rendu.
- Depuis Angular 19, les effets de composant (component effects) sont intégrés au processus de détection des changements, ce qui garantit un timing plus prévisible.
- Les effets seront exécutés le **nombre minimum de fois**. Si un effet **dépend** de **plusieurs signaux** et que **plusieurs d'entre eux changent simultanément**, une seule exécution de l'effect sera programmée.
- **Limitation** : Ne doit pas être utilisé pour des manipulations directes du DOM, car le DOM peut ne pas être encore mis à jour.

```
export class EffectComponent {  
  counter = signal(0);  
  constructor() {  
    this.counter.set(1);  
    this.counter.set(2);  
  }  
  private logEffect = effect(() => {  
    console.log(`  
      The current count is:  
      ${this.counter()}`);  
  });  
}
```

aymen.sellaouti@gmail.com